



二哥的Java进阶之路

javabetter.cn

面渣逆袭 JVM篇

三分恶|沉默王二修订版

15000字51张手绘图

内含54道面试八股

已成功帮助 7000 名球友拿到心仪 offer



前言

2.3 万字 115 张手绘图，详解 54 道 Java 虚拟机面试高频题（让天下没有难背的八股），面渣背会这些 JVM 八股文，这次吊打面试官，我觉得稳了（手动 dog）。整理：沉默王二，截[转载链接](#)，作者：三分恶，截[原文链接](#)。

亮白版本更适合拿出来打印，这也是很多学生党喜欢的方式，打印出来背诵的效率会更高。



2024 年 12 月 30 日开始着手第二版更新。

- 对于高频题，会标注在《[Java 面试指南（付费）](#)》中出现的位置，哪家公司，原题是什么，并且会加👉，目录一目了然；如果你想节省时间的话，可以优先背诵这些题目，尽快做到知彼知己，百战不殆。
- 区分八股精华回答版本和原理底层解释，让大家知其然知其所以然，同时又能做到面试时的高效回答。
- 结合项目（[技术派](#)、[pmhub](#)）来组织语言，让面试官最大程度感受到你的诚意，而不是机械化的背诵。
- 修复第一版中出现的问题，包括球友们的私信反馈，网站留言区的评论，以及 [GitHub 仓库](#)中的 issue，让这份面试指南更加完善。
- 增加[二哥编程星球](#)的球友们拿到的一些 offer，对面渣逆袭的感谢，以及对简历修改的一些认可，以此来激励大家，给大家更多信心。
- 优化排版，增加手绘图，重新组织答案，使其更加口语化，从而更贴近面试官的预期。

Current Repository
toBeBetterJavaer

Current Branch
master

Fetch origin
Last fetched 11 minutes ago

Changes 2

History

docs/src/sidebar/sanfene/jvm.md

2 changed files

docs/src/.../garbage-collector.md

docs/src/sidebar/sanf.../jvm.md

✓	✓	237	-	![JDK 1.8内存区域](https://cdn.tobebetterjavaer.com/tobebetterjavaer/images/sidebar/sanfene/jvm-8.png)
✓	✓	278	+	### 5.为什么使用元空间替代永久代?
		238		279
✓	✓	239	-	### 5.为什么使用元空间替代永久代作为方法区的实现?
✓	✓	280	+	客观上,永久代会导致 Java 应用程序更容易出现内存溢出的问题,因为它要受到 JVM 内存大小的限制。
		240		281
✓	✓	241	-	Java 虚拟机规范规定的方法区只是换种方式实现。有客观和主观两个原因。
✓	✓	282	+	HotSpot 虚拟机的永久代大小可以通过 <code>-XX:MaxPermSize</code> 参数来设置,32 位机器默认的大小为 64M,64 位的机器则为 85M。
		242		283
✓	✓	243	-	客观上使用永久代来实现方法区的决定的设计导致了 Java 应用更容易遇到内存溢出的问题(永久代有 <code>-XX:MaxPermSize</code> 的上限,即使不设置也有默认大小,而 J9 和 JRockit 只要没有触碰到进程可用内存的上限,例如 32 位系统中的 4GB 限制,就不会出问题),而且有极少数方法(例如 <code>String::intern()</code>)会因永久代的原因而导致不同虚拟机下有不同的表现。
✓	✓	284	+	而 J9 和 JRockit 虚拟机就不存在这种限制,只要没有触碰到进程可用的内存上限,例如 32 位系统中的 4GB 限制,就不会出问题。
		244		285
✓	✓	245	-	主观上当 Oracle 收购 BEA 获得了 JRockit 的所有权后,准备把 JRockit 中的优秀功能,譬如 Java Mission Control 管理工具,移植到 HotSpot 虚拟机时,但因为两者对方法区实现的差异而面临诸多困难。考虑到 HotSpot 未来的发展,在 JDK 6 的时候 HotSpot 开发团队就有放弃永久代,逐步改为采用本地内存(Native Memory)来实现方法区的计划了,到了 JDK 7 的 HotSpot,已经把原本放在永久代的字符串常量池、静态变量等移出,而到了 JDK 8,终于完全废弃了永久代的概念,改用与 JRockit、J9 一样在本地内存中实现的元空间(Meta-space)来代替,把 JDK 7 中永久代还剩余的内容(主要是类型信息)全部移到元空间中。
✓	✓	286	+	主观上,当 Oracle 收购 BEA 获得了 JRockit 的所有权后,就准备把 JRockit 中的优秀功能移植到 HotSpot 中。
		246		287
✓	✓	247	-	### 6.对象创建的过程了解吗?
✓	✓	288	+	如 Java Mission Control 管理工具。
✓	✓	289	+	
✓	✓	290	+	但因为两个虚拟机对方法区实现有差异,导致这项工作遇到了很多阻力。
		248		291
✓	✓	249	-	当我们使用 <code>new</code> 关键字创建一个对象的时候,JVM 首先会检查 <code>new</code> 指令的参数是否能在常量池中定位到一个类的符号引用,然后检查这个符号引用代表的类是否已被加载、解析和初始化过。如果没有,就先执行相应的类加载过程。
✓	✓	292	+	考虑到 HotSpot 虚拟机未来的发展,JDK 6 的时候,开发团队就打算放弃永久代了。
		250		293
✓	✓	251	-	如果已经加载,JVM 会为新生对象分配内存,内存分配完成之后,JVM 将分配到的内存空间初始化为零值(成员变量,数值类型是 0,布尔类型是 false,对象类型是 null),接下来设置对象头,对象头里包含了对象是哪个类的实例、对象的哈希码、对象的 GC 分代年龄等信息。
✓	✓	294	+	JDK 7 的时候 前讲了一小步 把原本放在永久代的字符串常量池 静态变量等移动到了堆中

Summary (required)

Description

Commit to master

由于 PDF 没办法自我更新,所以需要最新版的小伙伴,可以微信搜【沉默王二】,或者扫描/长按识别下面的二维码,关注二哥的公众号,回复【222】即可拉取最新版本。



当然了，请允许我的一点点私心，那就是星球的 PDF 版本会比公众号早一个月时间，毕竟星球用户都付费过了，我有必要让他们先享受到一点点福利。相信大家也都能理解，毕竟在线版是免费的，CDN、服务器、域名、OSS 等等都是需要成本的。

更别说我付出的时间和精力了，大家觉得有帮助还请给个口碑，让你身边的同事、同学都能受益到。

< 公众号

沉默王二

60*13薪别说新疆，外派到伊拉克都行

15:48

面渣逆袭+二哥的 Java 进阶之路 PDF，第二版，GitHub 星标 13000+

二哥的并发编程小册：<https://javabetter.cn/thread/>

二哥的 JVM 进阶之路：<https://javabetter.cn/jvm/>

222

< > wangpan

名称	修改日期
二哥的 JVM 进阶之路.epub	2024 年 7 月 11 日 14:0
二哥的 JVM 进阶之路.md	2024 年 7 月 11 日 14:0
二哥的 JVM 进阶之路.pdf	2024 年 7 月 11 日 14:0
面渣逆袭 Java 基础篇第二版.md	2024 年 12 月 31 日 08
面渣逆袭 JVM 篇 V2.0.epub	2025 年 1 月 21 日 14:2
面渣逆袭 JVM 篇 V2.0.pdf	2025 年 1 月 21 日 14:2
面渣逆袭 JVM 篇 V2.0 (暗黑版).pdf	2025 年 1 月 21 日 14:2
面渣逆袭 JVM 篇第二版.md	2025 年 1 月 21 日 14:2
面渣逆袭 MyBatis.md	2024 年 7 月 11 日 14:0
面渣逆袭 MyBatis.pdf	2024 年 7 月 11 日 14:0
面渣逆袭 Redis 篇.md	2024 年 7 月 11 日 14:0
面渣逆袭 Redis 篇.pdf	2024 年 7 月 11 日 14:0
面渣逆袭 Spring 篇.md	2024 年 7 月 11 日 14:0
面渣逆袭 Spring 篇.pdf	2024 年 7 月 11 日 14:0
面渣逆袭-分布式篇.md	2024 年 7 月 11 日 14:0
面渣逆袭-分布式篇.pdf	2024 年 7 月 11 日 14:0
面渣逆袭并发编程篇第二版.md	2025 年 2 月 26 日 11:3
面渣逆袭并发编程篇 V2.0.epub	2025 年 2 月 26 日 11:3
面渣逆袭并发编程篇 V2.0.pdf	2025 年 2 月 26 日 11:3
面渣逆袭并发编程篇 V2.0 (暗黑版).pdf	2025 年 2 月 26 日 11:3
面渣逆袭操作系统.md	2024 年 7 月 11 日 14:0
面渣逆袭操作系统.pdf	2024 年 7 月 11 日 14:0
面渣逆袭集合框架篇第二版.epub	2025 年 1 月 8 日 18:33
面渣逆袭集合框架篇第二版.md	2025 年 1 月 8 日 18:26
面渣逆袭集合框架篇 V2.0.pdf	2025 年 1 月 8 日 18:27
面渣逆袭集合框架篇 V2.0 (暗黑).pdf	2025 年 1 月 8 日 18:26
面渣逆袭计算机网络.md	2024 年 7 月 11 日 14:0
面渣逆袭计算机网络.pdf	2024 年 7 月 11 日 14:0
面渣逆袭微服务篇.md	2024 年 7 月 11 日 14:0
面渣逆袭微服务篇.pdf	2024 年 7 月 11 日 14:0
面渣逆袭 Java 基础篇 V2.0.epub	2024 年 12 月 31 日 09
面渣逆袭 Java 基础篇 V2.0.pdf	2024 年 12 月 30 日 20
面渣逆袭 Java 基础篇 V2.0 (暗黑).pdf	2024 年 12 月 30 日 20

我把二哥的 Java 进阶之路、JVM 进阶之路、并发编程进阶之路，以及所有面渣逆袭的版本都放进来了，涵盖 Java 基础、Java集合、Java并发、JVM、Spring、MyBatis、计算机网络、操作系统、MySQL、Redis、RocketMQ、分布式、微服务、设计模式、Linux 等 16 个大的主题，共有 40 多万字，2000+张手绘图，可以说是诚意满满。

展示一下暗黑版本的 PDF 吧，排版清晰，字体优雅，更加适合夜服，晚上看会更舒服一点。



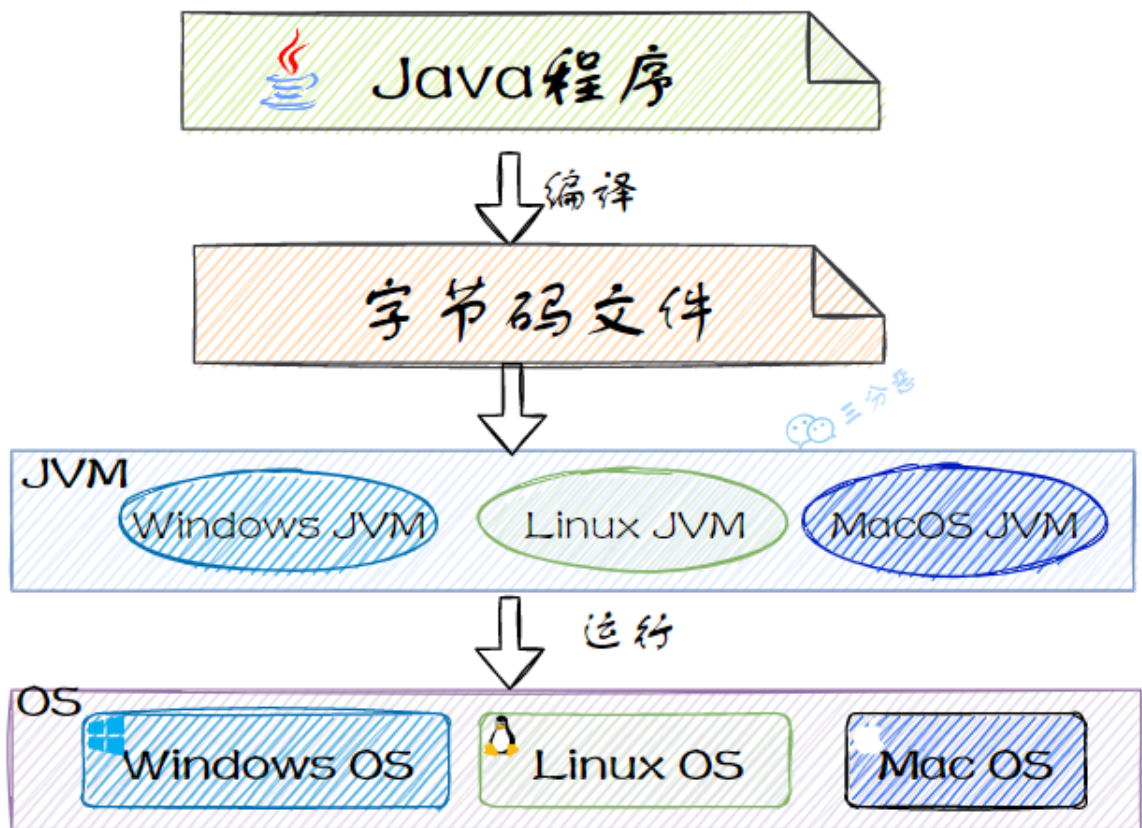
一、引言

1.什么是 JVM?

JVM，也就是 Java 虚拟机，它是 Java 实现跨平台的基石。

程序运行之前，需要先通过编译器将 Java 源代码文件编译成 Java 字节码文件；

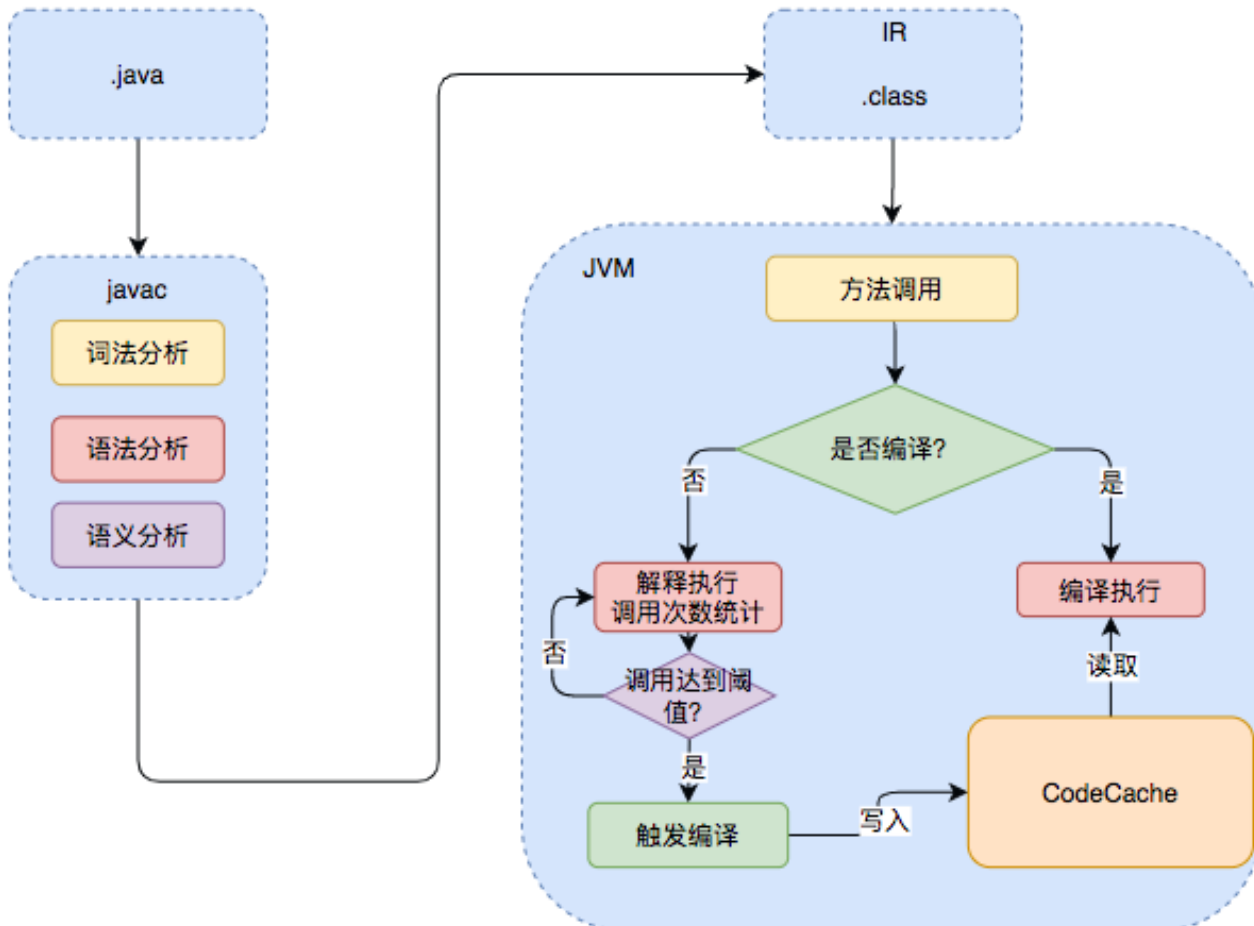
程序运行时，JVM 会对字节码文件进行逐行解释，翻译成机器码指令，并交给对应的操作系统去执行。



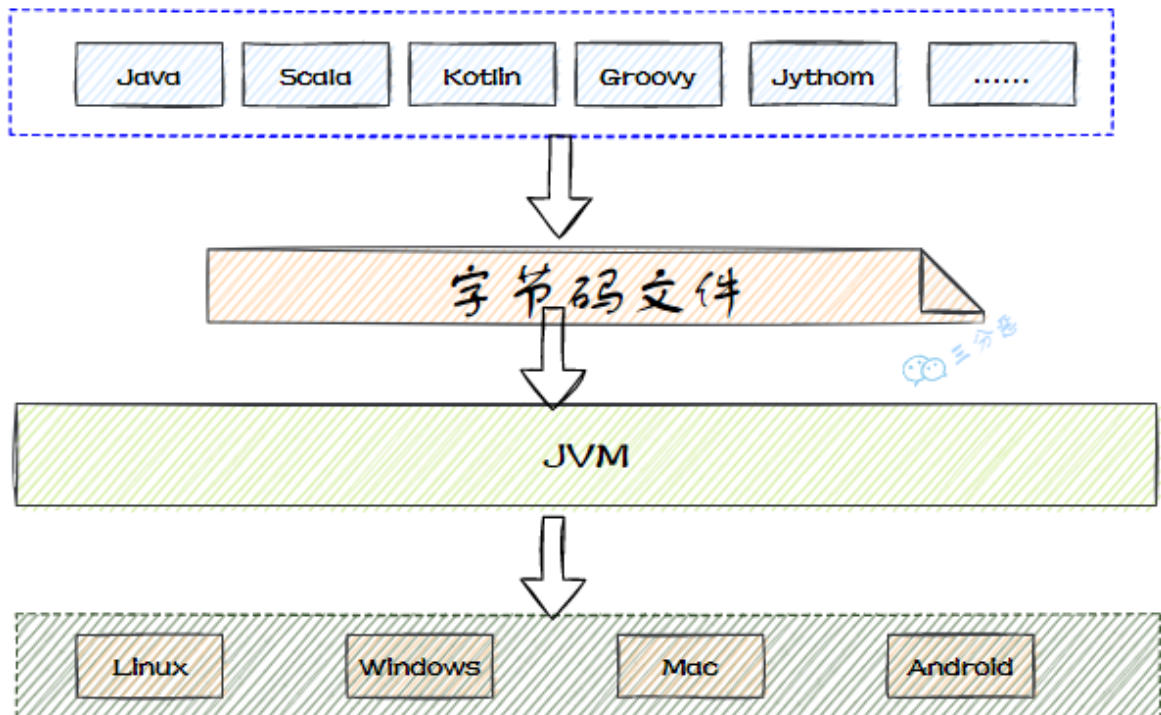
这样就实现了Java 一次编译，处处运行的特性。

说说JVM 的其他特性？

- ①、JVM 可以自动管理内存，通过垃圾回收器回收不再使用的对象并释放内存空间。
- ②、JVM 包含一个即时编译器 JIT，它可以在运行时将热点代码缓存到 codeCache 中，下次执行的时候不用再一行一行的解释，而是直接执行缓存后的机器码，执行效率会大幅提高。



③、任何可以通过 Java 编译的语言，比如说 Groovy、Kotlin、Scala 等，都可以在 JVM 上运行。



为什么要学习 JVM?

学习 JVM 可以帮助我们开发者更好地优化程序性能、避免内存问题。

比如了解 JVM 的内存模型和垃圾回收机制，可以帮助我们更合理地配置内存、减少 GC 停顿。

比如掌握 JVM 的类加载机制可以帮助我们排查类加载冲突或异常。

再比如说，JVM 还提供了很多调试和监控工具，可以帮助我们分析内存和线程的使用情况，从而解决内存溢出内存泄露等问题。

1. [Java 面试指南（付费）](#) 收录的京东同学 10 后端实习一面的原题：有了解 JVM 吗

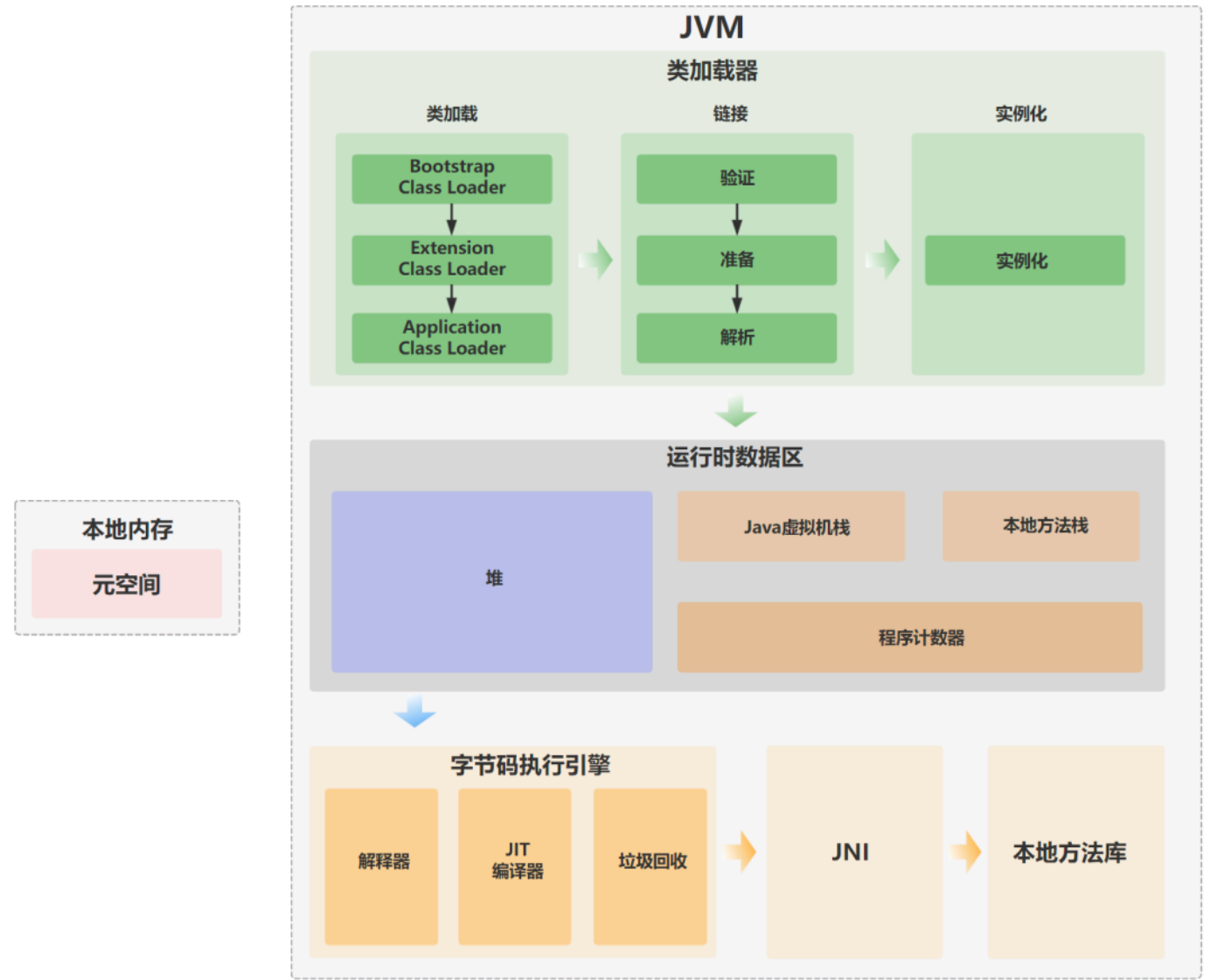
2. [Java 面试指南（付费）](#) 收录的字节跳动同学 20 测开一面的原题：了解过 JVM 么？讲一下 JVM 的特性

2.说说 JVM 的组织架构（补充）

增补于 2024 年 03 月 08 日。

推荐阅读：[大白话带你认识 JVM](#)

JVM 大致可以划分为三个部分：类加载器、运行时数据区和执行引擎。



① 类加载器，负责从文件系统、网络或其他来源加载 Class 文件，将 Class 文件中的二进制数据读入到内存当中。

② 运行时数据区，JVM 在执行 Java 程序时，需要在内存中分配空间来处理各种数据，这些内存区域按照 Java 虚拟机规范可以划分为方法区、堆、虚拟机栈、程序计数器和本地方法栈。

③ 执行引擎，也是 JVM 的心脏，负责执行字节码。它包括一个虚拟处理器、即时编译器 JIT 和垃圾回收器。

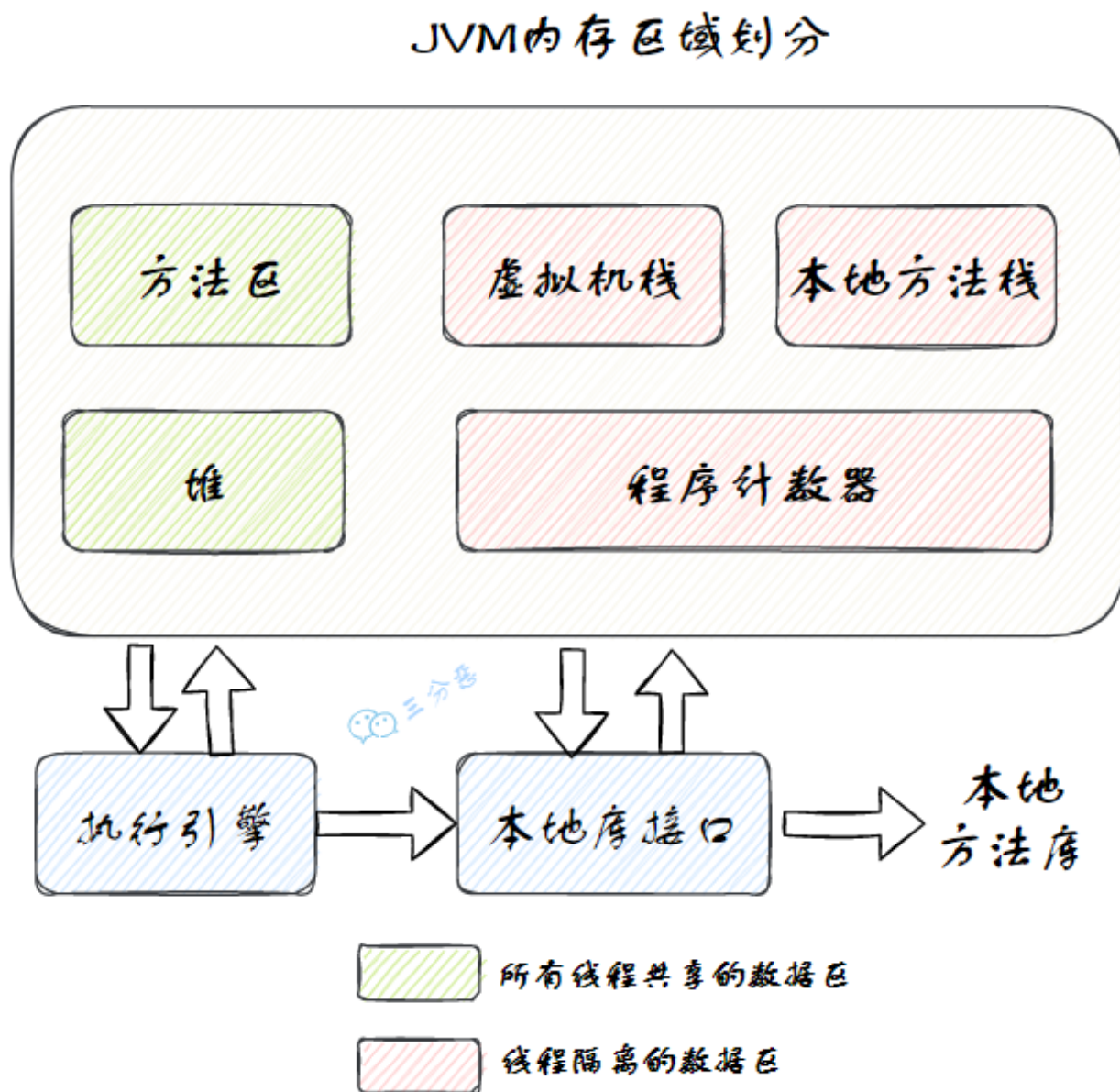
1. [Java 面试指南（付费）](#) 收录的腾讯 Java 后端实习一面原题：说说 JVM 的组织架构
2. [Java 面试指南（付费）](#) 收录的得物面经同学 9 面试题目原题：JVM 的架构，具体阐述一下各个部分的功能？

二、内存管理

3. 🌟 能说一下 JVM 的内存区域吗？

推荐阅读：[深入理解 JVM 的运行时数据区](#)

按照 Java 虚拟机规范，JVM 的内存区域可以细分为 程序计数器、虚拟机栈、本地方法栈、堆 和 方法区。



其中 方法区 和 堆 是线程共享的，虚拟机栈、本地方法栈 和 程序计数器 是线程私有的。

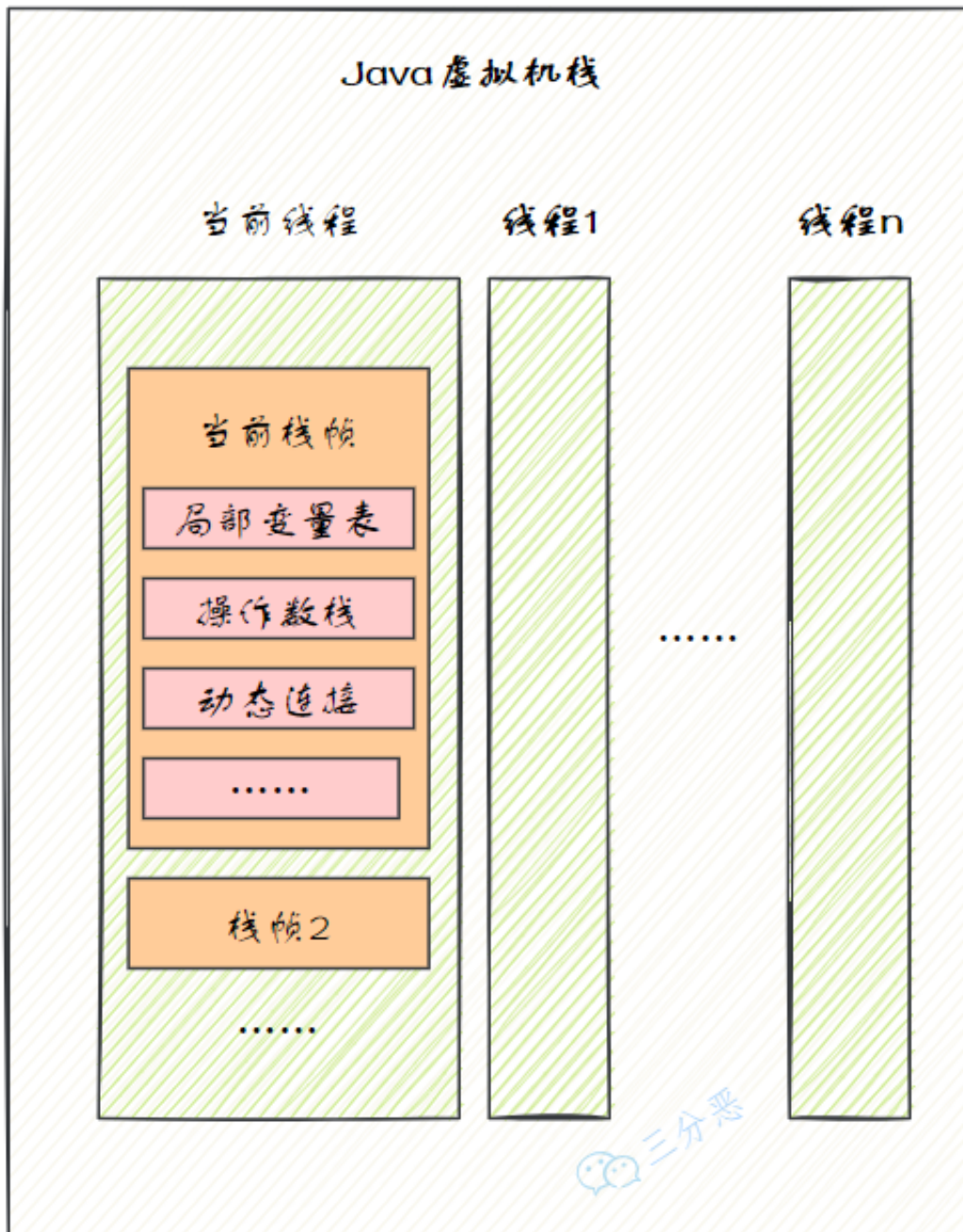
介绍一下程序计数器？

程序计数器也被称为 PC 寄存器，是一块较小的内存空间。它可以看作是当前线程所执行的字节码行号指示器。

介绍一下 Java 虚拟机栈？

Java 虚拟机栈的生命周期与线程相同。

当线程执行一个方法时，会创建一个对应的[栈帧](#)，用于存储局部变量表、操作数栈、动态链接、方法出口等信息，然后栈帧会被压入虚拟机栈中。当方法执行完毕后，栈帧会从虚拟机栈中移除。



一个什么都没有的空方法，空的参数都没有，那局部变量表里有没有变量？

对于[静态方法](#)，由于不需要访问实例对象 this，因此在局部变量表中不会有任何变量。

对于非静态方法，即使是一个完全空的方法，局部变量表中也会有一个用于存储 this 引用的变量。this 引用指向当前实例对象，在方法调用时被隐式传入。

详细解释一下：

比如说有这样一段代码：

```
public class VarDemo1 {
    public void emptyMethod() {
        // 什么都没有
    }

    public static void staticEmptyMethod() {
        // 什么都没有
    }
}
```

用 `javap -v VarDemo1` 命令查看编译后的字节码，就可以在 `emptyMethod` 中看到这样的内容：

```
public void emptyMethod();
  descriptor: ()V
  flags: ACC_PUBLIC
  Code:
    stack=0, locals=1, args_size=1
      0: return
  LineNumberTable:
    line 6: 0
  LocalVariableTable:
    Start   Length  Slot  Name   Signature
      0        1      0  this   Lcom/github/paicoding/forum/web/javabetter/jvm/VarDemo1;
```

这里的 `locals=1` 表示局部变量表有一个变量，即 `this`，Slot 0 位置存储了 `this` 引用。

而在静态方法 `staticEmptyMethod` 中，你会看到这样的内容：

```
public static void staticEmptyMethod();
  descriptor: ()V
  flags: ACC_PUBLIC, ACC_STATIC
  Code:
    stack=0, locals=0, args_size=0
      0: return
  LineNumberTable:
    line 10: 0
```

这里的 `locals=0` 表示局部变量表为空，因为静态方法属于类级别方法，不需要 `this` 引用，也就没有局部变量。

介绍一下本地方法栈？

本地方法栈与虚拟机栈相似，区别在于虚拟机栈是为 JVM 执行 Java 编写的方法服务的，而本地方法栈是为 Java 调用[本地 native 方法](#)服务的，通常由 C/C++ 编写。

在本地方法栈中，主要存放了 native 方法的局部变量、动态链接和方法出口等信息。当一个 Java 程序调用一个 native 方法时，JVM 会切换到本地方法栈来执行这个方法。

介绍一下本地方法栈的运行场景？

当 Java 应用需要与操作系统底层或硬件交互时，通常会用到本地方法栈。

比如调用操作系统的特定功能，如内存管理、文件操作、系统时间、系统调用等。

详细说明一下：

比如说获取系统时间的 `System.currentTimeMillis()` 方法就是调用本地方法，来获取操作系统当前时间的。

```
public final class System {  
    See Also: setSecurityManager  
    public static SecurityManager getSecurityManager() { return security; }  
  
    Returns the current time in milliseconds. Note that while the unit of time of the return value is a millisecond, the granularity of the value depends on the underlying operating system and may be larger. For example, many operating systems measure time in units of tens of milliseconds.  
    See the description of the class Date for a discussion of slight discrepancies that may arise between "computer time" and coordinated universal time (UTC).  
  
    Returns: the difference, measured in milliseconds, between the current time and midnight, January 1, 1970 UTC.  
    See Also: java.util.Date  
    public static native long currentTimeMillis();  
}
```

再比如 JVM 自身的一些底层功能也需要通过本地方法来实现。像 `Object` 类中的 `hashCode()` 方法、`clone()` 方法等。

```
public class Object {  
    However, the programmer should be aware that producing distinct integer results for unequal  
    objects may improve the performance of hash tables.  
  
    As much as is reasonably practical, the hashCode method defined by class Object does return  
    distinct integers for distinct objects. (This is typically implemented by converting the internal address  
    of the object into an integer, but this implementation technique is not required by the Java™  
    programming language.)  
  
    Returns: a hash code value for this object.  
  
    See Also: equals(Object),  
             System.identityHashCode  
  
    public native int hashCode();  
}
```

native 方法解释一下？

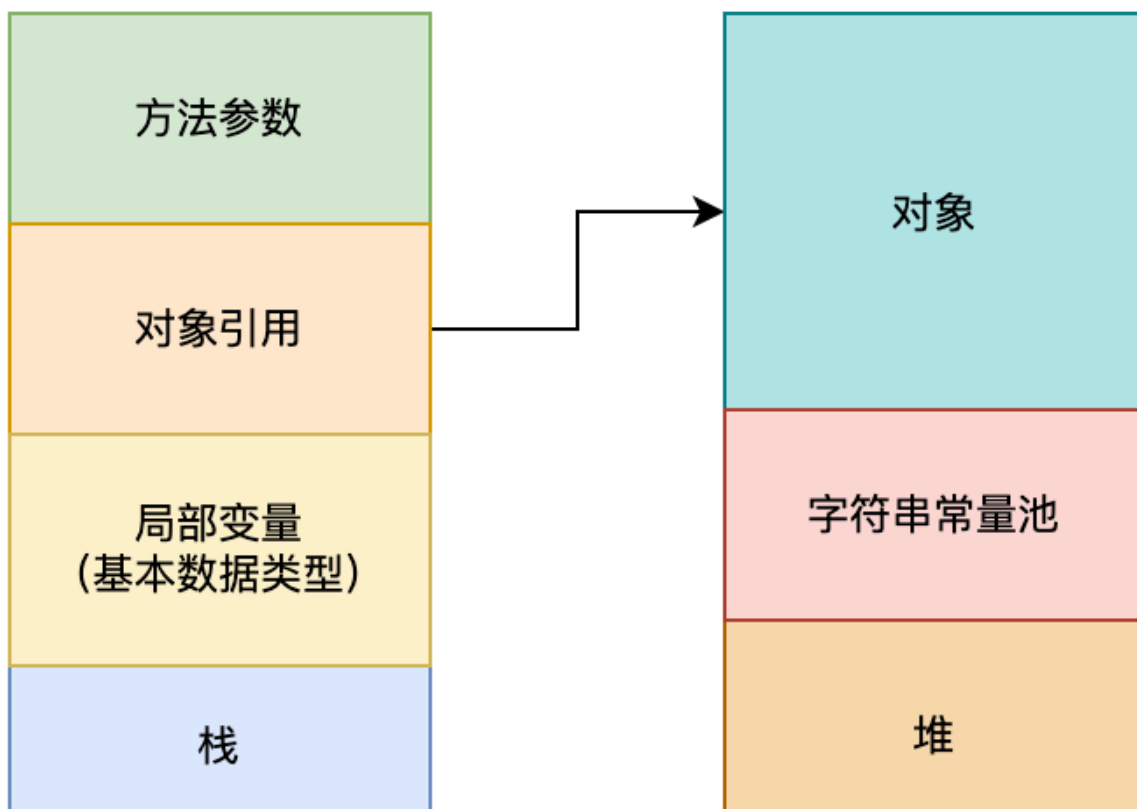
推荐阅读：[手把手教你用 C 语言实现 Java native 本地方法](#)

native 方法是在 Java 中通过 [native 关键字](#) 声明的，用于调用非 Java 语言，如 C/C++ 编写的代码。Java 可以通过 JNI，也就是 Java Native Interface 与底层系统、硬件设备、或者本地库进行交互。

介绍一下 Java 堆？

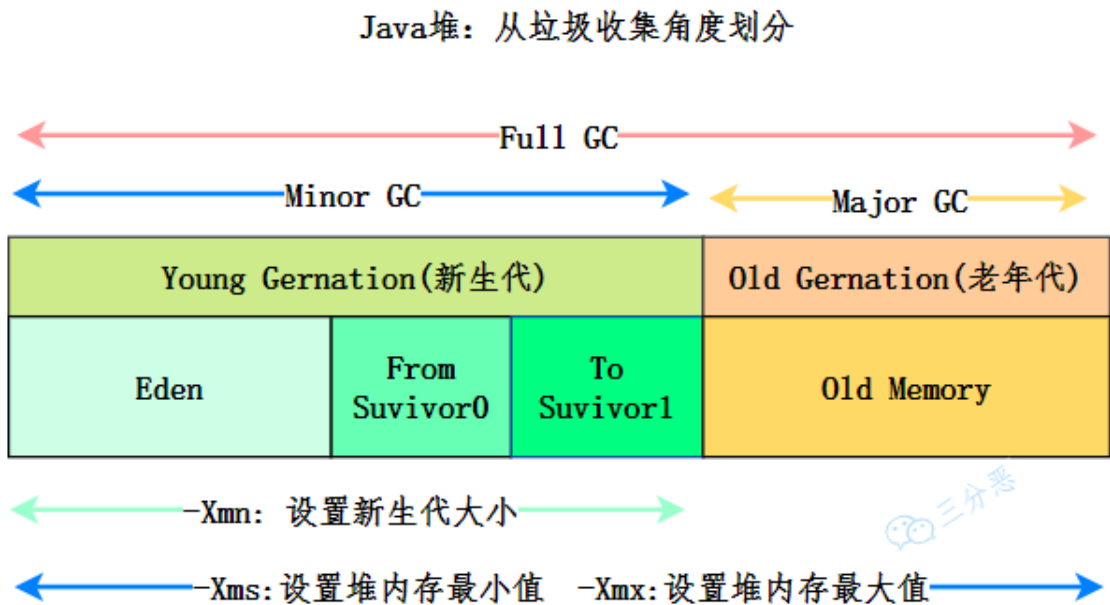
堆是 JVM 中最大的一块内存区域，被所有线程共享，在 JVM 启动时创建，主要用来存储 new 出来的对象。

二哥的 JVM 进阶之路



Java 中“几乎”所有的对象都会在堆中分配，堆也是[垃圾收集器](#)管理的目标区域。

从内存回收的角度来看，由于垃圾收集器大部分都是基于分代收集理论设计的，所以堆又被细分为 新生代、老年代、Eden空间、From Survivor空间、To Survivor空间等。



随着 [JIT 编译器](#)的发展和逃逸技术的逐渐成熟，“所有的对象都会分配到堆上”就不再那么绝对了。

从JDK 7 开始，JVM 默认开启了逃逸分析，意味着如果某些方法中的对象引用没有被返回或者没有在方法体外使用，也就是未逃逸出去，那么对象可以直接在栈上分配内存。

堆和栈的区别是什么？

堆属于线程共享的内存区域，几乎所有 new 出来的对象都会堆上分配，生命周期不由单个方法调用所决定，可以在方法调用结束后继续存在，直到不再被任何变量引用，最后被垃圾收集器回收。

栈属于线程私有的内存区域，主要存储局部变量、方法参数、对象引用等，通常随着方法调用的结束而自动释放，不需要垃圾收集器处理。

介绍一下方法区？

方法区并不真实存在，属于 Java 虚拟机规范中的一个逻辑概念，用于存储已被 JVM 加载的类信息、常量、静态变量、即时编译器编译后的代码缓存等。

在 HotSpot 虚拟机中，方法区的实现称为永久代 PermGen，但在 Java 8 及之后的版本中，已经被元空间 Metaspace 所替代。

变量存在堆栈的什么位置？

对于局部变量，它存储在当前方法栈帧中的局部变量表中。当方法执行完毕，栈帧被回收，局部变量也会被释放。

```
public void method() {  
    int localVar = 100; // 局部变量，存储在栈帧中的局部变量表里  
}
```


对于静态变量来说，它存储在 Java 虚拟机规范中的方法区中，在 Java 7 中是永久带，在 Java8 及以后 是元空间。

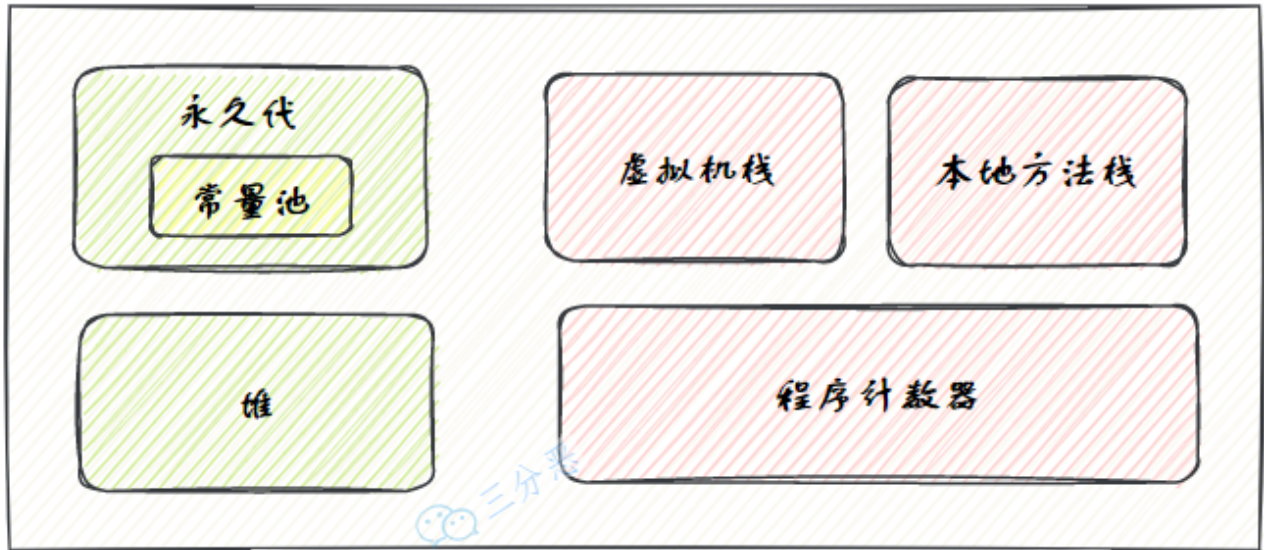
```
public class StaticVarDemo {
    public static int staticVar = 100; // 静态变量，存储在方法区中
}
```

1. [Java 面试指南（付费）](#) 收录的京东同学 10 后端实习一面的原题：堆和栈的区别是什么
2. [Java 面试指南（付费）](#) 收录的比亚迪面经同学 3 Java 技术一面面试原题：介绍一下 JVM 运行时数据区
3. [Java 面试指南（付费）](#) 收录的字节跳动面经同学 1 Java 后端技术一面面试原题：讲一下 JVM 内存结构？
4. [Java 面试指南（付费）](#) 收录的京东面经同学 1 Java 技术一面面试原题：说说 JVM 运行时数据区
5. [Java 面试指南（付费）](#) 收录的美团面经同学 2 Java 后端技术一面面试原题：JVM 内存结构了解吗？
6. [Java 面试指南（付费）](#) 收录的快手面经同学 1 部门主站技术部面试原题：请说一下 Java 的内存区域，程序计数器等？
7. [Java 面试指南（付费）](#) 收录的字节跳动面经同学 8 Java 后端实习一面面试原题：jvm 内存分布，有垃圾回收的是哪些地方
8. [Java 面试指南（付费）](#) 收录的得物面经同学 8 一面面试原题：说一说 jvm 内存区域
9. [Java 面试指南（付费）](#) 收录的美团面经同学 3 Java 后端技术一面面试原题：jmm 内存模型 栈 方法区存放的是什么
10. [Java 面试指南（付费）](#) 收录的收钱吧面经同学 1 Java 后端一面面试原题：你提到了栈帧，那局部变量表除了栈帧还有什么？一个什么都没有的空方法，完全空的参数什么都没有，那局部变量表里有没有变量？
11. [Java 面试指南（付费）](#) 收录的招银网络科技面经同学 9 Java 后端技术一面面试原题：Java堆内存和栈内存的区别
12. [Java 面试指南（付费）](#) 收录的 OPPO 面经同学 1 面试原题：说一下JVM内存模型
13. [Java 面试指南（付费）](#) 收录的深信服面经同学 3 Java 后端线下一面面试原题：JVM变量存在堆栈的位置？
14. [Java 面试指南（付费）](#) 收录的TP联洲同学 5 Java 后端一面的原题：jvm内存区域，本地方法栈的运行场景，Native方法解释一下
15. [Java 面试指南（付费）](#) 收录的字节跳动同学 17 后端技术面试原题：jvm结构 运行时数据区有什么结构 堆存什么
16. [Java 面试指南（付费）](#) 收录的腾讯面经同学 29 Java 后端一面原题：new一个对象存放在哪里？（运行时数据区），局部变量存在JVM哪里

4.说一下 JDK 1.6、1.7、1.8 内存区域的变化？

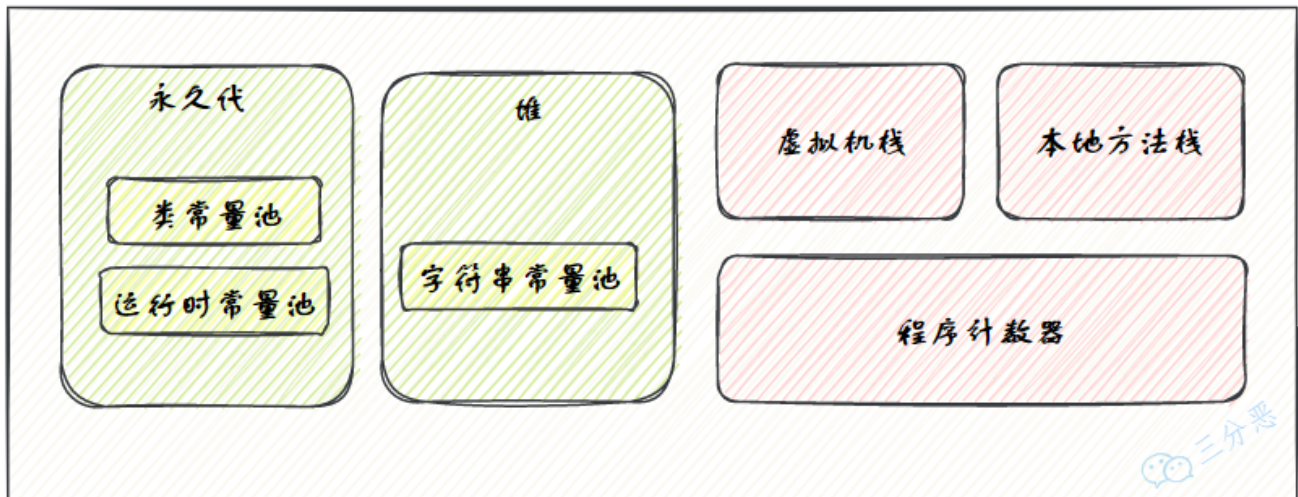
JDK 1.6 使用永久代来实现方法区：

JDK1.6



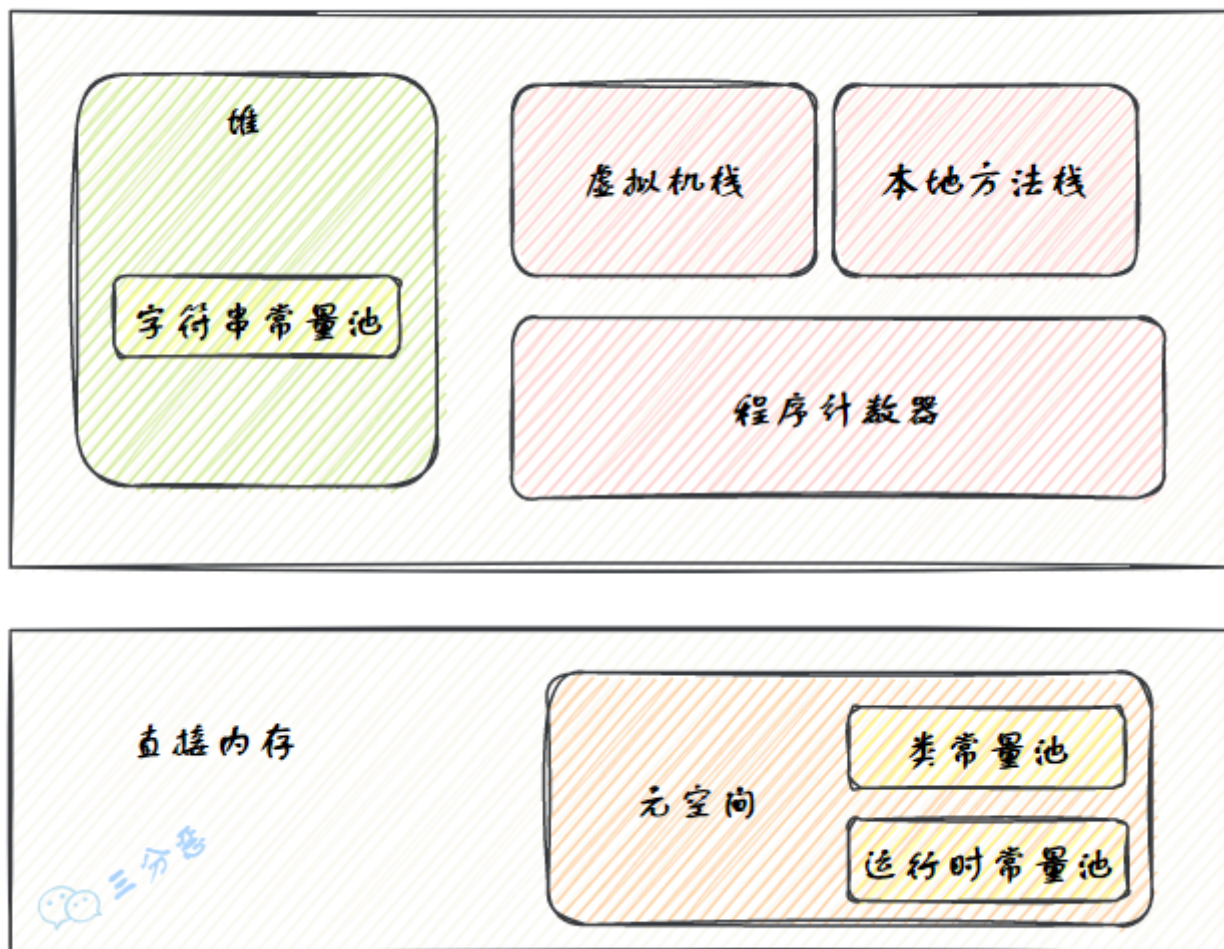
JDK 1.7 时仍然是永久带，但发生了一些细微变化，比如将字符串常量池、静态变量存放到了堆上。

JDK1.7



在 JDK 1.8 时，直接在内存中划出了一块区域，叫元空间，来取代之之前放在 JVM 内存中的永久代，并将运行时常量池、类常量池都移动到了元空间。

JDK1.8



5.为什么使用元空间替代永久代？

客观上，永久代会导致 Java 应用程序更容易出现内存溢出的问题，因为它要受到 JVM 内存大小的限制。

HotSpot 虚拟机的永久代大小可以通过 `-XX:MaxPermSize` 参数来设置，32 位机器默认的大小为 64M，64 位的机器则为 85M。

而 J9 和 JRockit 虚拟机就不存在这种限制，只要没有触碰到进程可用的内存上限，例如 32 位系统中的 4GB 限制，就不会出问题。

主观上，当 Oracle 收购 BEA 获得了 JRockit 的所有权后，就准备把 JRockit 中的优秀功能移植到 HotSpot 中。

如 Java Mission Control 管理工具。

但因为两个虚拟机对方法区实现有差异，导致这项工作遇到了很多阻力。

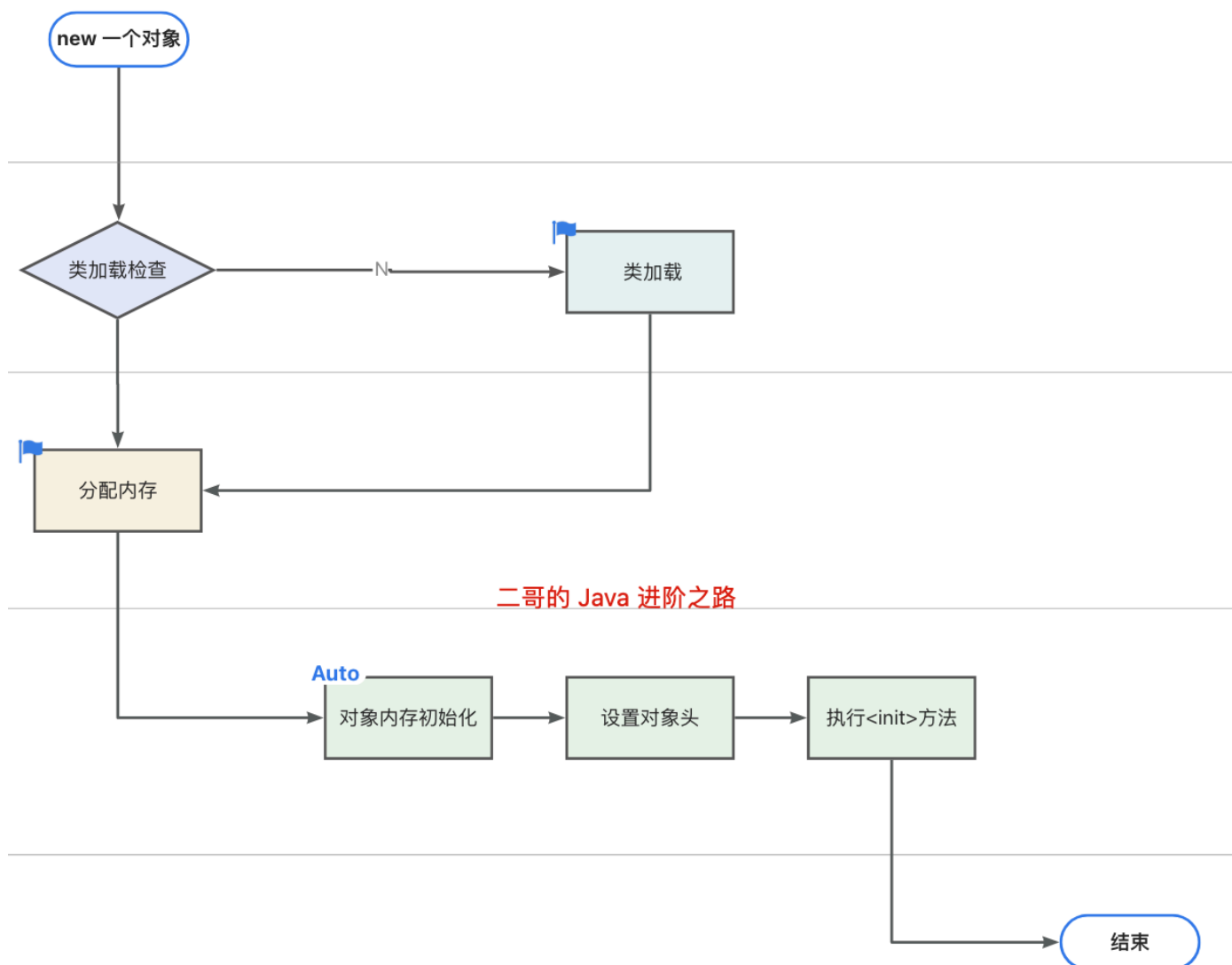
考虑到 HotSpot 虚拟机未来的发展，JDK 6 的时候，开发团队就打算放弃永久代了。

JDK 7 的时候，前进了一小步，把原本放在永久代的字符串常量池、静态变量等移动到了堆中。

JDK 8 就终于完成了这项移出工作，这样的好处就是，元空间的大小不再受到 JVM 内存的限制，而是可以像 J9 和 JRockit 那样，只要系统内存足够，就可以一直用。

6. 🌟对象创建的过程了解吗？

当我们使用 `new` 关键字创建一个对象时，JVM 首先会检查 `new` 指令的参数是否能在常量池中定位到类的符号引用，然后检查这个符号引用代表的类是否已被加载、解析和初始化。如果没有，就先执行类加载。



如果已经加载，JVM 会为对象分配内存完成初始化，比如数值类型的成员变量初始值是 0，布尔类型是 false，对象类型是 null。

接下来会设置对象头，里面包含了对象是哪个类的实例、对象的哈希码、对象的 GC 分代年龄等信息。

最后，JVM 会执行构造方法 `<init>` 完成赋值操作，将成员变量赋值为预期的值，比如 `int age = 18`，这样一个对象就创建完成了。

对象的销毁过程了解吗？

当对象不再被任何引用指向时，就会变成垃圾。垃圾收集器会通过可达性分析算法判断对象是否存活，如果对象不可达，就会被回收。

垃圾收集器通过标记清除、标记复制、标记整理等算法来回收内存，将对象占用的内存空间释放出来。

可以通过 `java -XX:+PrintCommandLineFlags -version` 和 `java -XX:+PrintGCDetails -version` 命令查看 JVM 的 GC 收集器。

```
~/Downloads/nacos2/nacos/bin (0.081s)
java -XX:+PrintCommandLineFlags -version
-XX:InitialHeapSize=2147483648 -XX:MaxHeapSize=32203866112 -XX:+PrintCommandLineFlags -XX:+UseCompressedC
lassPointers -XX:+UseCompressedOops -XX:+UseParallelGC
openjdk version "1.8.0_412"
OpenJDK Runtime Environment Corretto-8.412.08.1 (build 1.8.0_412-b08)
OpenJDK 64-Bit Server VM Corretto-8.412.08.1 (build 25.412-b08, mixed mode)

~/Downloads/nacos2/nacos/bin (0.075s)
java -XX:+PrintGCDetails -version
openjdk version "1.8.0_412"
OpenJDK Runtime Environment Corretto-8.412.08.1 (build 1.8.0_412-b08)
OpenJDK 64-Bit Server VM Corretto-8.412.08.1 (build 25.412-b08, mixed mode)
Heap
PSYoungGen      total 611840K, used 20992K [0x000000008003000000, 0x0000000082ad80000, 0x00000000a80000000)
 eden space 524800K, 4% used [0x000000008003000000, 0x00000000801780188, 0x00000000820380000)
  from space 87040K, 0% used [0x00000000825880000, 0x00000000825880000, 0x0000000082ad80000)
 to   space 87040K, 0% used [0x00000000820380000, 0x00000000820380000, 0x00000000825880000)
ParOldGen       total 1398272K, used 0K [0x000000003008000000, 0x00000000355d80000, 0x00000000800300000)
 object space 1398272K, 0% used [0x000000003008000000, 0x00000000300800000, 0x00000000355d80000)
Metaspace       used 2227K, capacity 4480K, committed 4480K, reserved 1056768K
 class space    used 241K, capacity 384K, committed 384K, reserved 1048576K
```

可以看到，我本机安装的JDK 8 默认使用的是 `Parallel Scavenge + Parallel Old`。

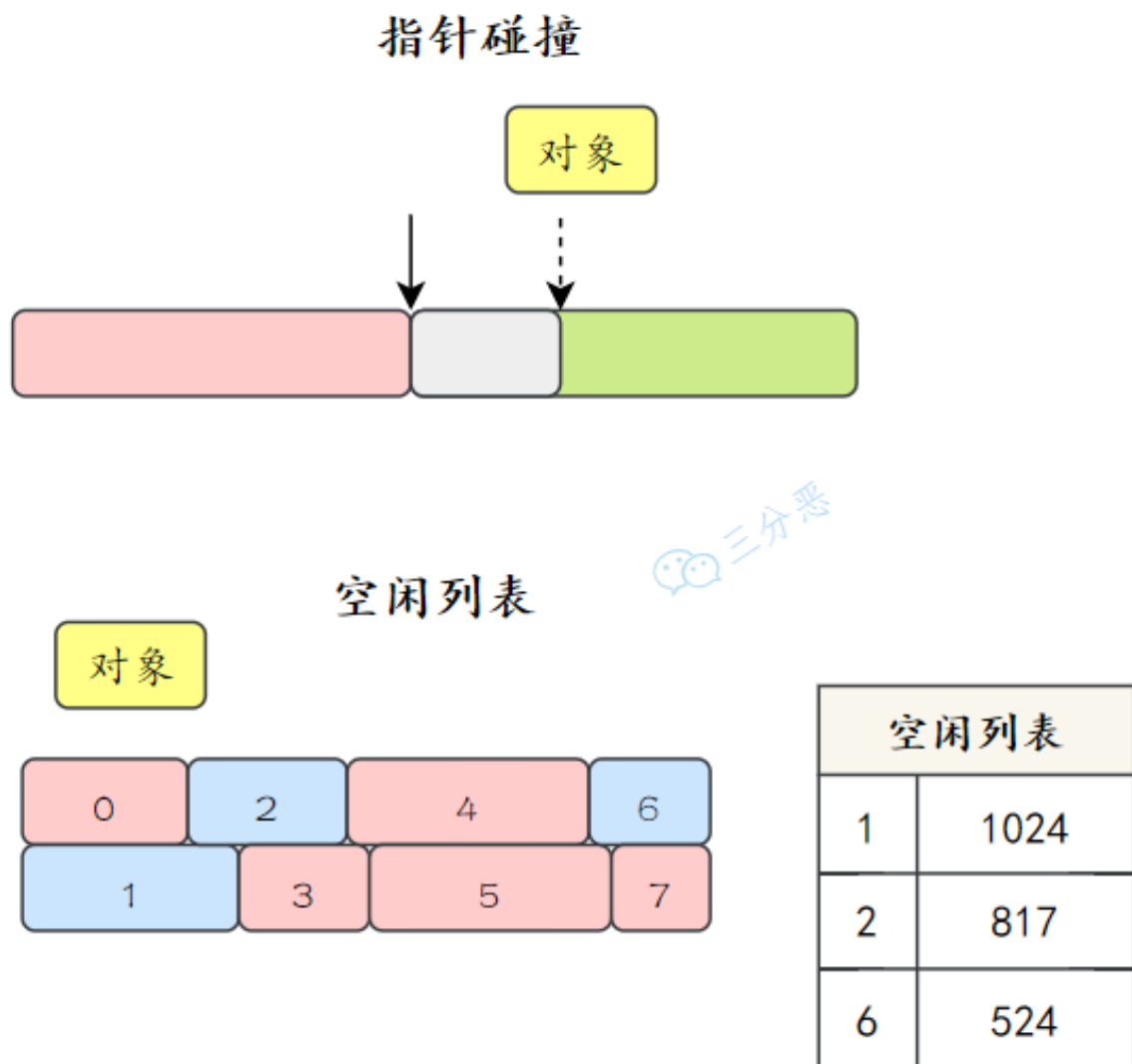
不同参数代表对应的垃圾收集器表单：

新生代	老年代	JVM参数
Serial	Serial	-XX:+UseSerialGC
Parallel Scavenge	Serial	-XX:+UseParallelGC -XX:-UseParallelOldGC
Parallel Scavenge	Parallel Old	-XX:+UseParallelGC -XX:+UseParallelOldGC
Parallel New	CMS	-XX:+UseParNewGC -XX:+UseConcMarkSweepGC
G1		-XX:+UseG1GC

1. [Java 面试指南（付费）](#) 收录的比亚迪面经同学 3 Java 技术一面面试原题：对象创建到销毁的流程
2. [Java 面试指南（付费）](#) 收录的美团面经同学 2 Java 后端技术一面面试原题：说说创建对象的流程？
3. [Java 面试指南（付费）](#) 收录的携程面经同学 1 Java 后端技术一面面试原题：对象创建到销毁，内存如何分配的，（类加载和对象创建过程，CMS，G1 内存清理和分配）

7.堆内存是如何分配的？

在堆中为对象分配内存时，主要使用两种策略：指针碰撞和空闲列表。



三分恶

指针碰撞适用于管理简单、碎片化较少的内存区域，如年轻代；而空闲列表适用于内存碎片化较严重或对象大小差异较大的场景如老年代。

什么是指针碰撞？

假设堆内存是一个连续的空间，分为两个部分，一部分是已经被使用的内存，另一部分是未被使用的内存。

在分配内存时，Java 虚拟机会维护一个指针，指向下一个可用的内存地址，每次分配内存时，只需要将指针向后移动一段距离，如果没有发生碰撞，就将这段内存分配给对象实例。

什么是空闲列表？

JVM 维护一个列表，记录堆中所有未占用的内存块，每个内存块都记录有大小和地址信息。

当有新的对象请求内存时，JVM 会遍历空闲列表，寻找足够大的空间来存放新对象。

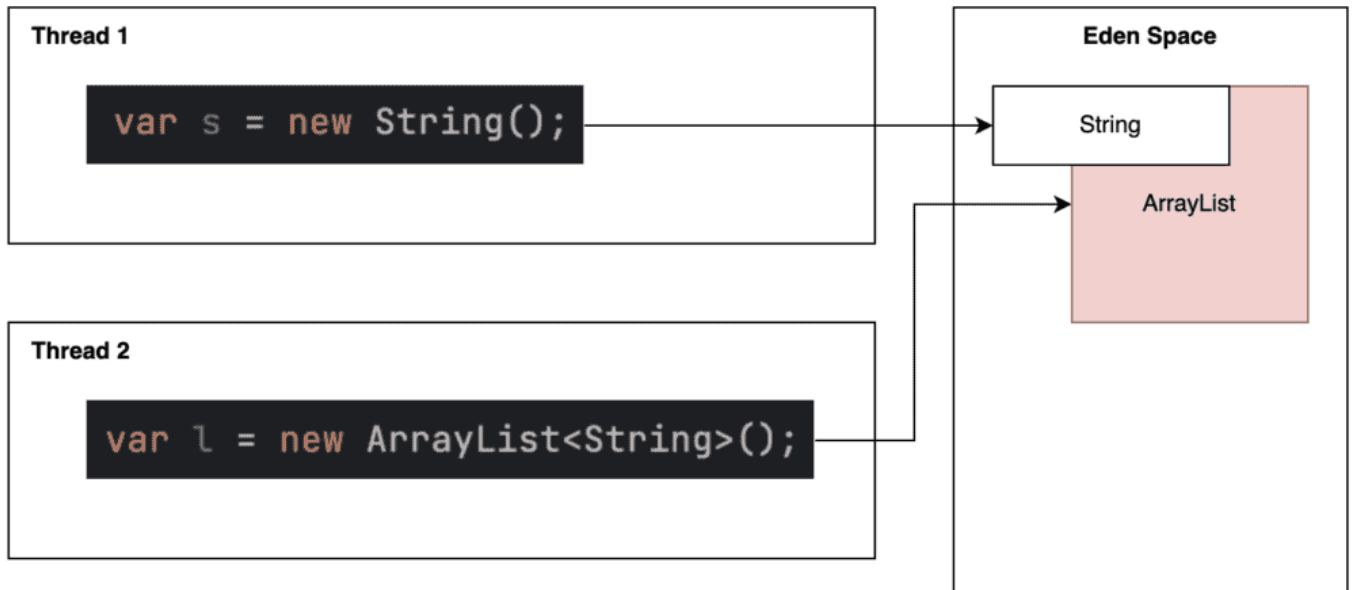
分配后，如果选中的内存块未被完全利用，剩余的部分会作为一个新的内存块加入到空闲列表中。

1. [Java 面试指南（付费）](#) 收录的携程面经同学 1 Java 后端技术一面面试原题：对象创建到销毁，内存如何分配的，（类加载和对象创建过程，CMS，G1 内存清理和分配）

memo: 2025 年 1 月 10 日修改到此

8.new 对象时，堆会发生抢占吗？

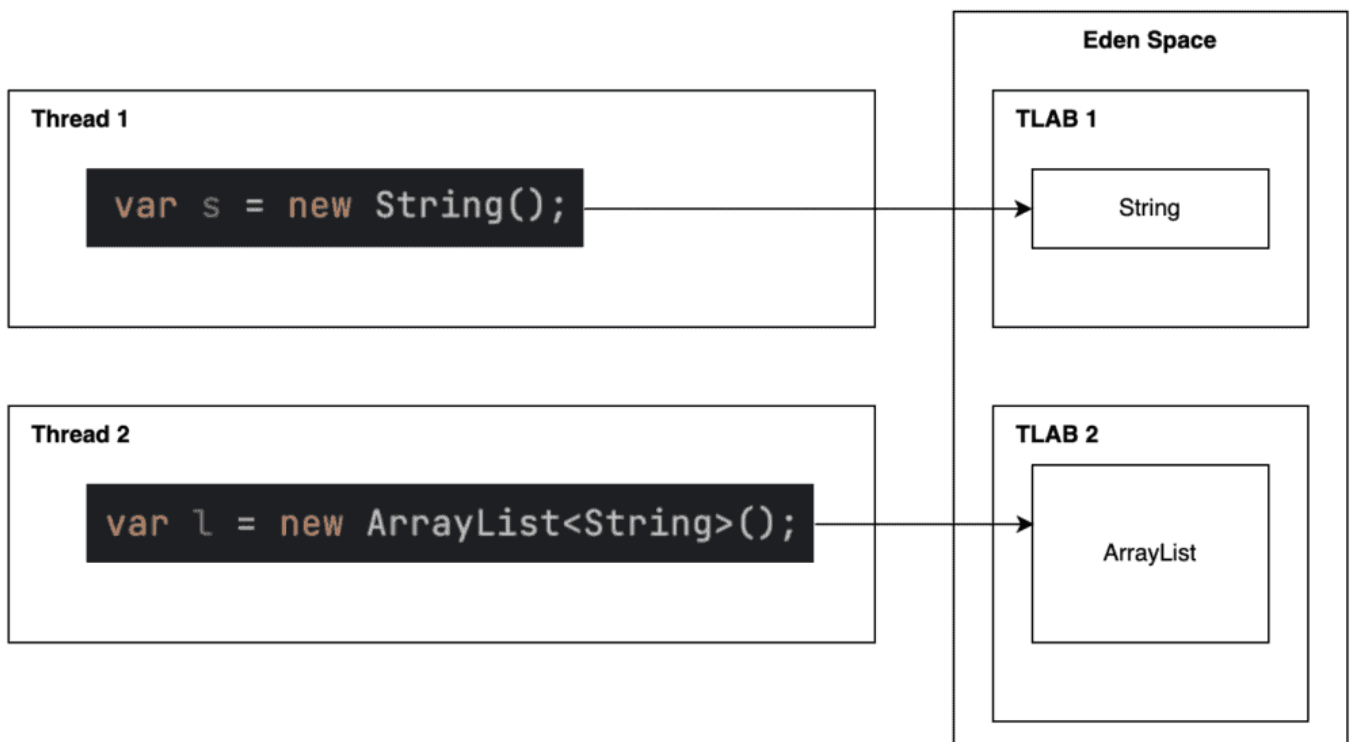
会。



new 对象时，指针会向右移动一个对象大小的距离，假如一个线程 A 正在给字符串对象 s 分配内存，另外一个线程 B 同时为 ArrayList 对象 l 分配内存，两个线程就发生了抢占。

JVM 怎么解决堆内存分配的竞争问题？

为了解决堆内存分配的竞争问题，JVM 为每个线程保留了一小块内存空间，被称为 TLAB，也就是线程本地分配缓冲区，用于存放该线程分配的对象。



当线程需要分配对象时，直接从 TLAB 中分配。只有当 TLAB 用尽或对象太大需要直接在堆中分配时，才会使用全局分配指针。

这里简单测试一下 TLAB。

可以通过 `java -XX:+PrintFlagsFinal -version | grep TLAB` 命令查看当前 JVM 是否开启了 TLAB。

```
java -XX:+PrintFlagsFinal -version | grep TLAB
    bool FastTLABRefill                = true                {product}
    uintx MinTLABSize                  = 2048                {product}
    bool PrintTLAB                      = false               {product}
    bool ResizeTLAB                    = true                 {pd product}
    uintx TLABAllocationWeight          = 35                  {product}
    uintx TLABRefillWasteFraction       = 64                  {product}
    uintx TLABSize                      = 0                   {product}
    bool TLABStats                      = true                 {product}
    uintx TLABWasteIncrement            = 4                   {product}
    uintx TLABWasteTargetPercent        = 1                   {product}
    bool UseTLAB                        = true                 {pd product}
    bool ZeroTLAB                      = false                {product}
openjdk version "1.8.0_412"
OpenJDK Runtime Environment Corretto-8.412.08.1 (build 1.8.0_412-b08)
OpenJDK 64-Bit Server VM Corretto-8.412.08.1 (build 25.412-b08, mixed mode)
```

如果开启了 TLAB，会看到类似以下的输出，其中 `bool UseTLAB` 的值为 `true`。

我们编写一个简单的测试类，创建大量对象并强制触发垃圾回收，查看 TLAB 的使用情况。

```
class TLABDemo {
    public static void main(String[] args) {
        for (int i = 0; i < 10_000_000; i++) {
            allocate(); // 创建大量对象
        }
        System.gc(); // 强制触发垃圾回收
    }

    private static void allocate() {
        // 小对象分配，通常会使用 TLAB
        byte[] bytes = new byte[64];
    }
}
```

在 VM 参数中添加 `-XX:+UseTLAB -XX:+PrintTLAB -XX:+PrintGCDetails -XX:+PrintGCDateStamps`，运行后可以看到这样的内容：

```
TLAB: gc thread: 0x000000012d0e7800 [id: 30723] desired_size: 10496KB slow allocs: 0  refill waste: 167936B alloc: 1.00000  52494KB ref
TLAB: gc thread: 0x000000012d0e6800 [id: 22019] desired_size: 10496KB slow allocs: 0  refill waste: 167936B alloc: 1.00000  52494KB ref
TLAB: gc thread: 0x000000013c00a000 [id: 6915] desired_size: 10496KB slow allocs: 0  refill waste: 167936B alloc: 1.00000  52494KB ref
TLAB totals: thrds: 3  refills: 5 max: 3 slow allocs: 0 max 0 waste: 55.6% gc: 29877480B max: 10747880B slow: 3464B max: 3464B fast: 8B m
2025-01-11T11:16:30.800+0800: [GC (System.gc()) [PSYoungGen: 52494K->2396K(611840K)] 52494K->2404K(2010112K), 0.0012334 secs] [Times: use
2025-01-11T11:16:30.801+0800: [Full GC (System.gc()) [PSYoungGen: 2396K->0K(611840K)] [ParOldGen: 8K->2112K(1398272K)] 2404K->2112K(20101
Heap
PSYoungGen      total 611840K, used 26240K [0x0000000800300000, 0x000000082ad80000, 0x0000000a80000000)
  eden space 524800K, 5% used [0x0000000800300000,0x0000000801ca01a8,0x0000000820380000)
  from space 87040K, 0% used [0x0000000820380000,0x0000000820380000,0x0000000825880000)
  to   space 87040K, 0% used [0x0000000825880000,0x0000000825880000,0x000000082ad80000)
ParOldGen       total 1398272K, used 2112K [0x0000000300800000, 0x0000000355d80000, 0x0000000800300000)
  object space 1398272K, 0% used [0x0000000300800000,0x0000000300a10280,0x0000000355d80000)
Metaspace       used 3438K, capacity 4564K, committed 4864K, reserved 1056768K
class space     used 367K, capacity 388K, committed 512K, reserved 1048576K
```

- waste: 未使用的 TLAB 空间。
- alloc: 分配到 TLAB 的空间。
- refills: TLAB 被重新填充的次数。

可以看到，当前线程的 TLAB 目标大小为 10,496 KB（desired_size: 10496KB）；未发生慢分配（slow allocs: 0）；分配效率直接拉满（alloc: 1.00000 52494KB）。

当使用 `-XX:-UseTLAB -XX:+PrintGCDetails` 关闭 TLAB 时，会看到类似以下的输出：

```
/Users/itwanger/Library/Java/JavaVirtualMachines/corretto-1.8.0_412/Contents/Home/bin/java ...
[GC (System.gc()) [PSYoungGen: 22682K->2420K(611840K)] 22682K->2428K(2010112K), 0.0010365 secs] [Times: user=0.01 sys=0.00, real=0.00 sec]
[Full GC (System.gc()) [PSYoungGen: 2420K->0K(611840K)] [ParOldGen: 8K->2088K(1398272K)] 2428K->2088K(2010112K), [Metaspace: 3410K->3410K]
Heap
PSYoungGen      total 611840K, used 33K [0x0000000080030000, 0x0000000082ad8000, 0x0000000a80000000)
  eden space 524800K, 0% used [0x0000000080030000, 0x0000000080030ad8, 0x0000000820380000)
  from space 87040K, 0% used [0x0000000082038000, 0x0000000082038000, 0x0000000825880000)
  to   space 87040K, 0% used [0x0000000825880000, 0x0000000825880000, 0x000000082ad80000)
ParOldGen       total 1398272K, used 2088K [0x0000000300800000, 0x0000000355d80000, 0x0000000800300000)
  object space 1398272K, 0% used [0x0000000300800000, 0x0000000300a0a3f0, 0x0000000355d80000)
Metaspace       used 3438K, capacity 4564K, committed 4864K, reserved 1056768K
  class space   used 367K, capacity 388K, committed 512K, reserved 1048576K
```

直接出现了两次 GC，因为没有 TLAB，Eden 区更快被填满，导致年轻代 GC。年轻代 GC 频繁触发，一部分长生命周期对象被晋升到老年代，间接导致老年代 GC 触发。

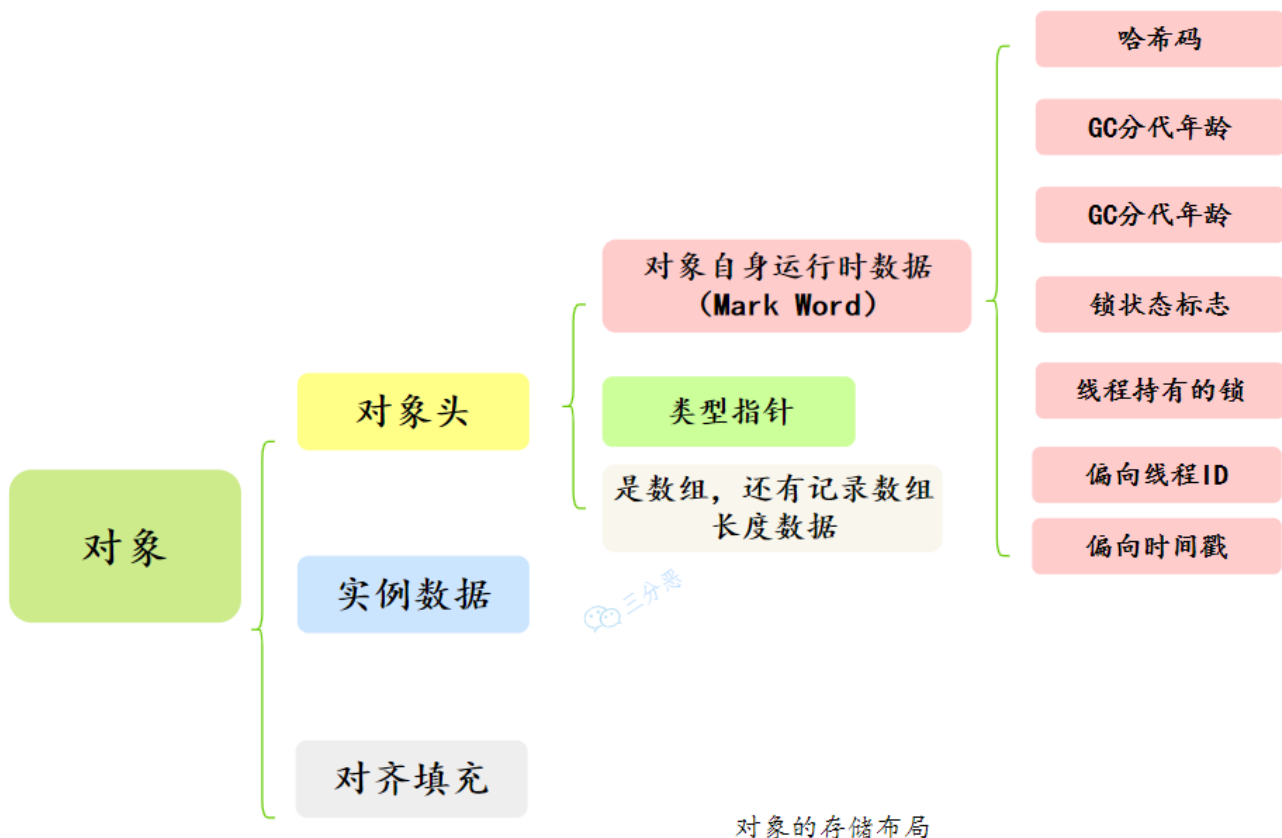
9.能说一下对象的内存布局吗？

好的。

对象的内存布局是由 Java 虚拟机规范定义的，但具体的实现细节各有不同，如 HotSpot 和 OpenJ9 就不一样。

就拿我们常用的 HotSpot 来说吧。

对象在内存中包括三部分：对象头、实例数据和对齐填充。



说说对象头的作用？

对象头是对象存储在内存中的元信息，包含了Mark Word、类型指针等信息。

Mark Word 存储了对象的运行时状态信息，包括锁、哈希值、GC 标记等。在 64 位操作系统下占 8 个字节，32 位操作系统下占 4 个字节。

类型指针指向对象所属类的元数据，也就是 Class 对象，用来支持多态、方法调用等功能。

除此之外，如果对象是数组类型，还会有一个额外的数组长度字段。占 4 个字节。

类型指针会被压缩吗？

类型指针可能会被压缩，以节省内存空间。比如说在开启压缩指针的情况下占 4 个字节，否则占 8 个字节。在 JDK 8 中，压缩指针默认是开启的。

可以通过 `java -XX:+PrintFlagsFinal -version | grep UseCompressedOops` 命令来查看 JVM 是否开启了压缩指针。

```

/usr/local/mysql/bin (0.341s)
[j] java -XX:+PrintFlagsFinal -version | grep UseCompressedOops

    bool UseCompressedOops               := true
           {lp64_product}
java version "1.8.0_301"
Java(TM) SE Runtime Environment (build 1.8.0_301-b09)
Java HotSpot(TM) 64-Bit Server VM (build 25.301-b09, mixed mode)
  
```


如果压缩指针开启，输出结果中的 `bool UseCompressedOops` 值为 `true`。

实例数据了解吗？

了解一些。

实例数据是对象实际的字段值，也就是成员变量的值，按照字段在类中声明的顺序存储。

```
class ObjectDemo {  
    int age;  
    String name;  
}
```

JVM 会对这些数据进行对齐/重排，以提高内存访问速度。

对齐填充了解吗？

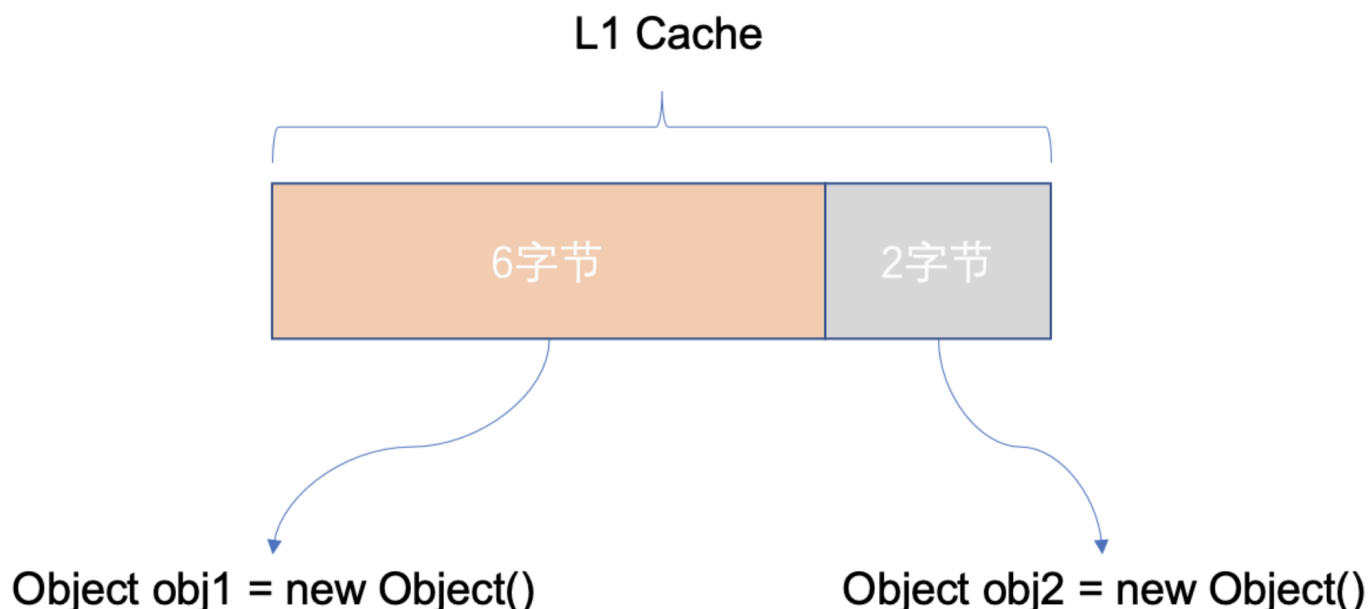
由于 JVM 的内存模型要求对象的起始地址是 8 字节对齐（64 位 JVM 中），因此对象的总大小必须是 8 字节的倍数。

如果对象头和实例数据的总长度不是 8 的倍数，JVM 会通过填充额外的字节来对齐。

比如说，如果对象头 + 实例数据 = 14 字节，则需要填充 2 个字节，使总长度变为 16 字节。

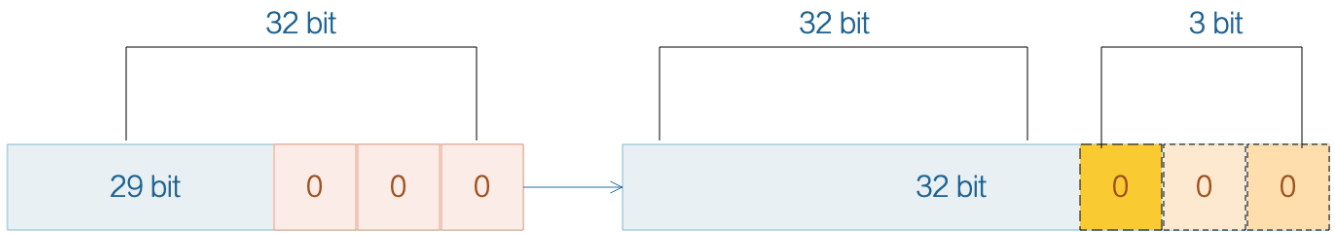
为什么非要进行 8 字节对齐呢？

因为 CPU 进行内存访问时，一次寻址的指针大小是 8 字节，正好是 L1 缓存行的大小。如果不进行内存对齐，则可能出现跨缓存行访问，导致额外的缓存行加载，CPU 的访问效率就会降低。



比如说上图中 `obj1` 占 6 个字节，由于没有对齐，导致这一行缓存中多了 2 个字节 `obj2` 的数据，当 CPU 访问 `obj2` 的时候，就会导致缓存行刷新。

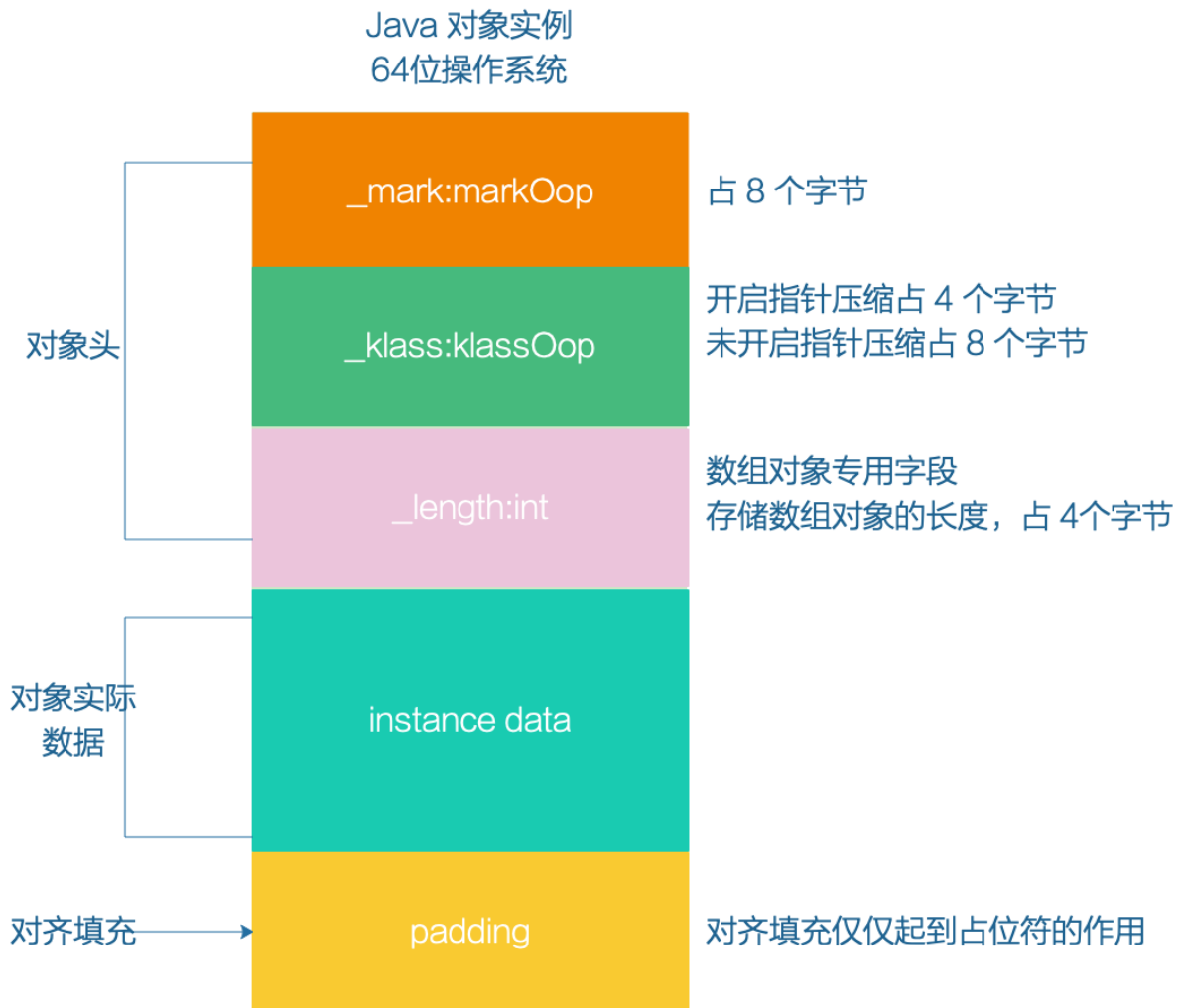
也就说，8 字节对齐，是为了效率的提高，以空间换时间的一种方案。



new Object() 对象的内存大小是多少?

推荐阅读: [高端面试必备: 一个 Java 对象占用多大内存](#)

一般来说, 目前的操作系统都是 64 位的, 并且 JDK 8 中的压缩指针是默认开启的, 因此在 64 位的 JVM 上, `new Object()` 的大小是 16 字节 (12 字节的对象头 + 4 字节的对齐填充)。



对象头的大小是固定的, 在 32 位 JVM 上是 8 字节, 在 64 位 JVM 上是 16 字节; 如果开启了压缩指针, 就是 12 字节。

实例数据的大小取决于对象的成员变量和它们的类型。对于 `new Object()` 来说, 由于默认没有成员变量, 因此我们可以认为此时的实例数据大小是 0。

假如 MyObject 对象有三个成员变量，分别是 int、long 和 byte 类型，那么它们占用的内存大小分别是 4 字节、8 字节和 1 字节。

```
class MyObject {  
    int a;        // 4 字节  
    long b;       // 8 字节  
    byte c;       // 1 字节  
}
```

考虑到对齐填充，MyObject 对象的总大小为 12（对象头） + 4（a） + 8（b） + 1（c） + 7（填充） = 32 字节。

用过 JOL 查看对象的内存布局吗？

用过。

[JOL](#) 是一款分析 JVM 对象布局的工具。

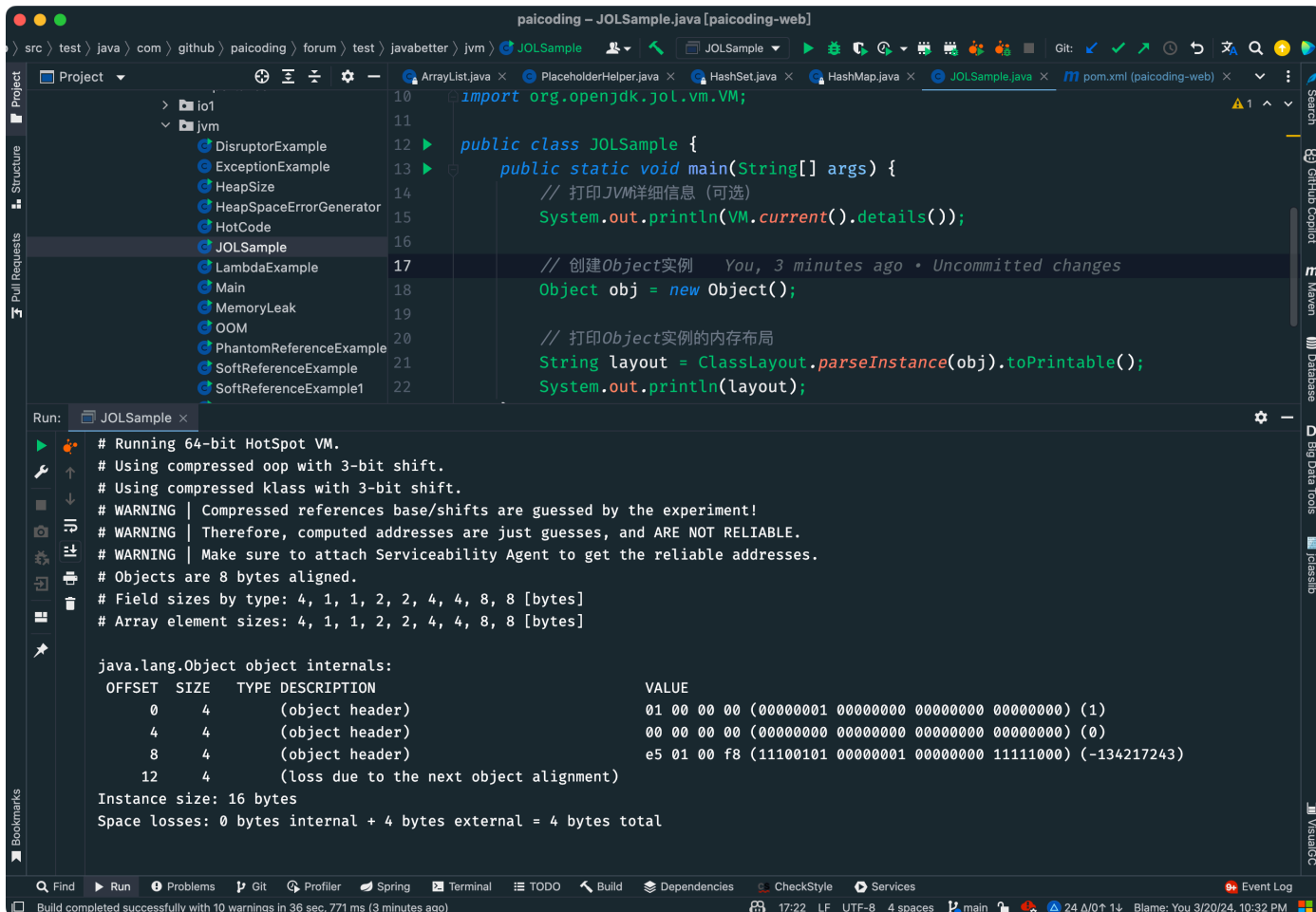
第一步，在 pom.xml 中引入 JOL 依赖：

```
<dependency>  
    <groupId>org.openjdk.jol</groupId>  
    <artifactId>jol-core</artifactId>  
    <version>0.9</version>  
</dependency>
```

第二步，使用 JOL 编写代码示例：

```
public class JOLSample {  
    public static void main(String[] args) {  
        // 打印JVM详细信息（可选）  
        System.out.println(VM.current().details());  
  
        // 创建Object实例  
        Object obj = new Object();  
  
        // 打印Object实例的内存布局  
        String layout = ClassLayout.parseInstance(obj).toPrintable();  
        System.out.println(layout);  
    }  
}
```

第三步，运行代码，查看输出结果：



可以看到有 OFFSET、SIZE、TYPE DESCRIPTION、VALUE 这几个信息。

- OFFSET：偏移地址，单位字节；
- SIZE：占用的内存大小，单位字节；
- TYPE DESCRIPTION：类型描述，其中 object header 为对象头；
- VALUE：对应内存中当前存储的值，二进制 32 位；

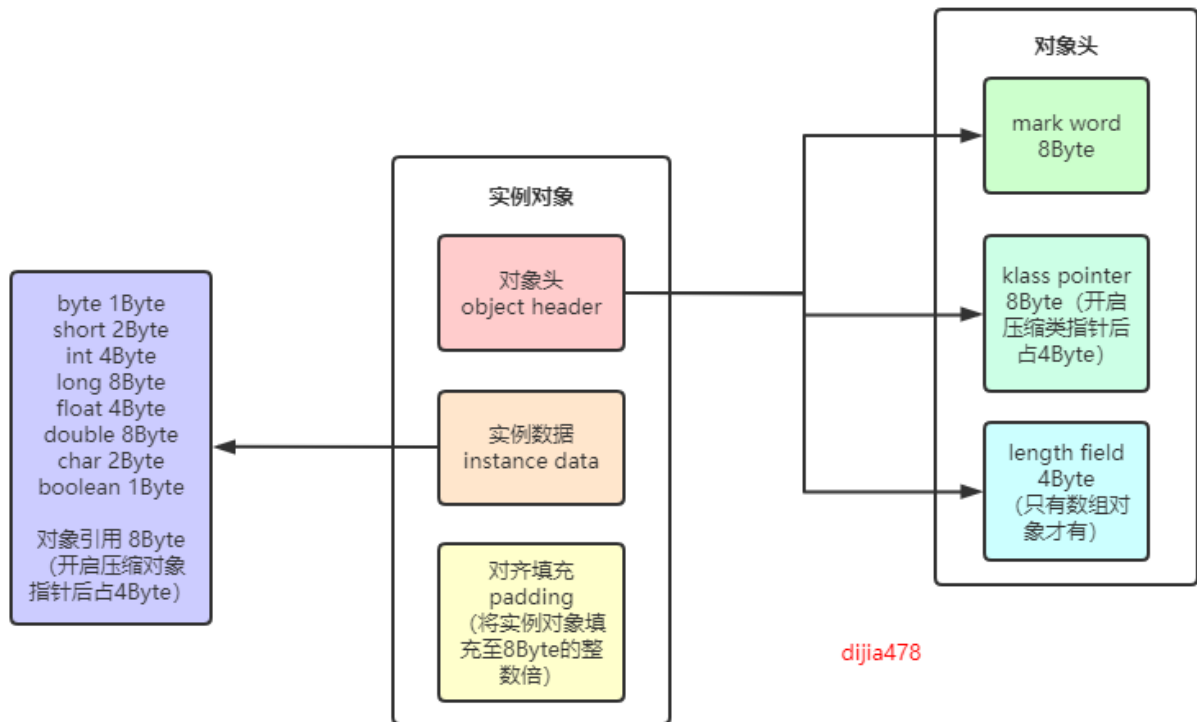
从上面的结果能看到，对象头是 12 个字节，还有 4 个字节的 padding，`new Object()` 一共 16 个字节。

对象的引用大小了解吗？

推荐阅读：[Object o = new Object\(\)占多少个字节？](#)

在 64 位 JVM 上，未开启压缩指针时，对象引用占用 8 字节；开启压缩指针时，对象引用会被压缩到 4 字节。HotSpot 虚拟机默认是开启压缩指针的。

hotspot实现的64位虚拟机

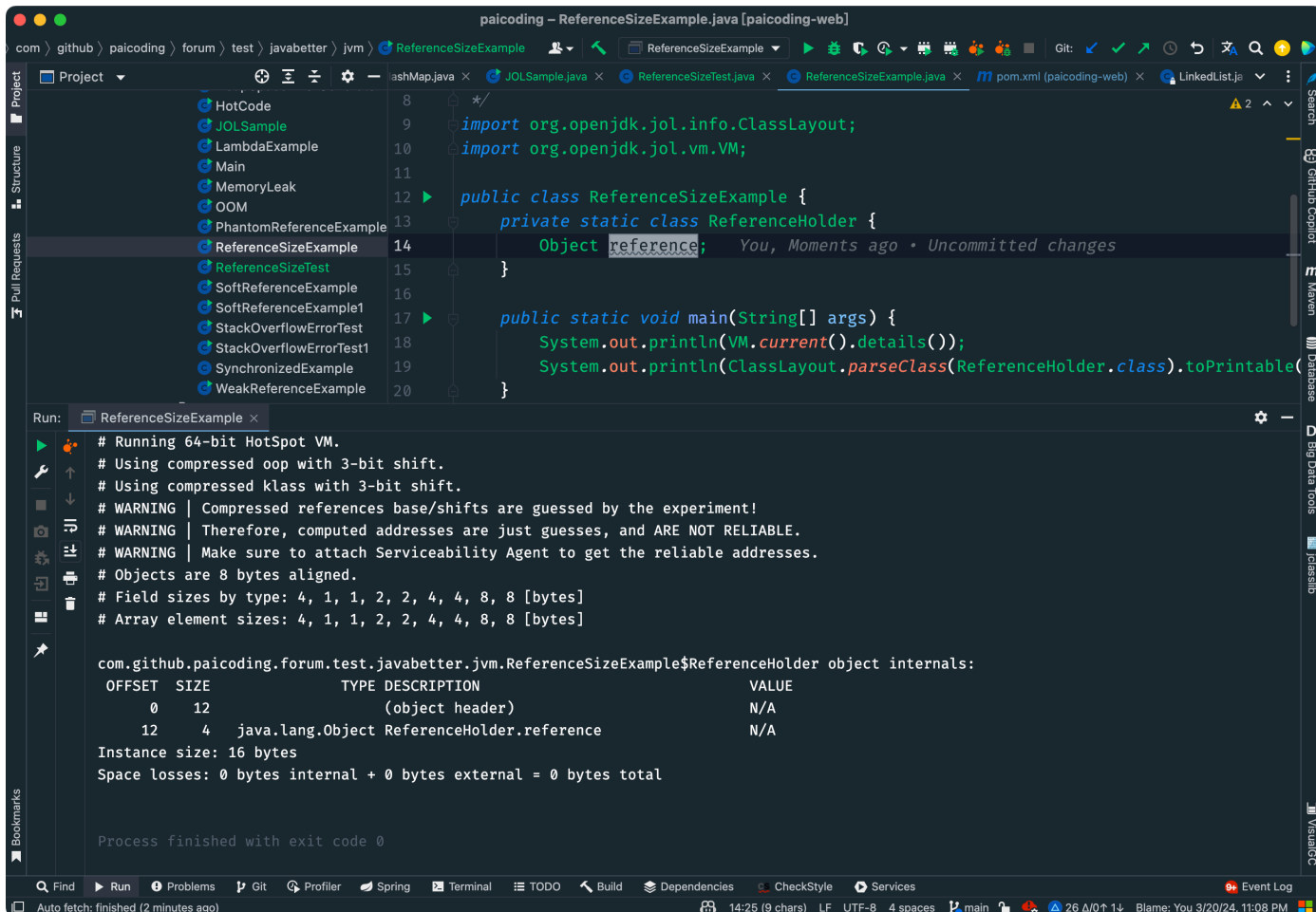


我们来验证一下：

```
class ReferenceSizeExample {
    private static class ReferenceHolder {
        Object reference;
    }

    public static void main(String[] args) {
        System.out.println(VM.current().details());
        System.out.println(ClassLayout.parseClass(ReferenceHolder.class).toPrintable());
    }
}
```

运行代码，查看输出结果：



ReferenceHolder.reference 的大小为 4 字节。

1. [Java 面试指南（付费）](#) 收录的帆软同学 3 Java 后端一面的原题：Object a = new object()的大小，对象引用占多少大小？
2. [Java 面试指南（付费）](#) 收录的去哪儿面经同学 1 技术二面面试原题：Object 底层的数据结构（蒙了）

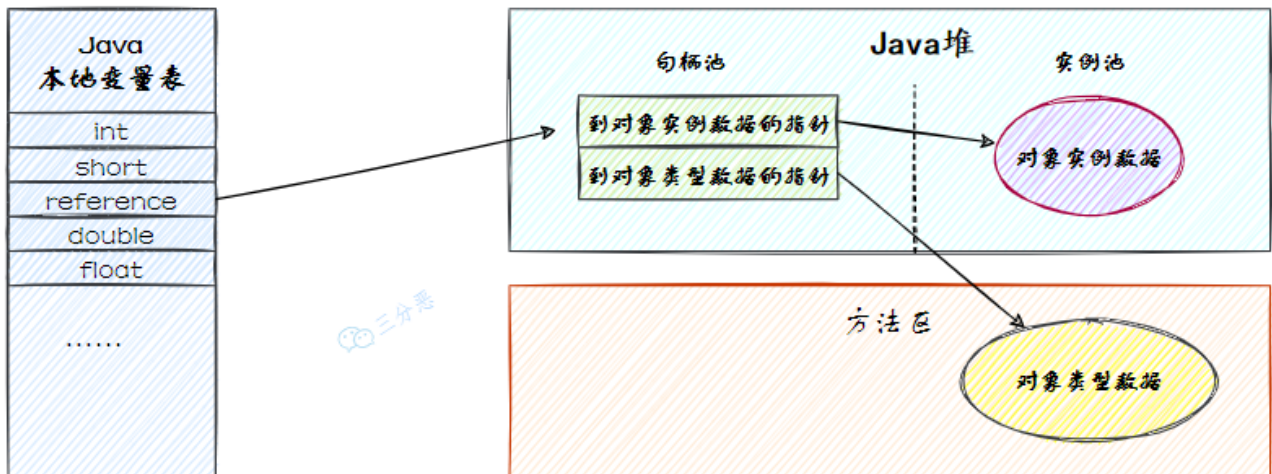
memo：2025 年 1 月 11 日修改到此

10.JVM 怎么访问对象的？

主流的方式有两种：句柄和直接指针。

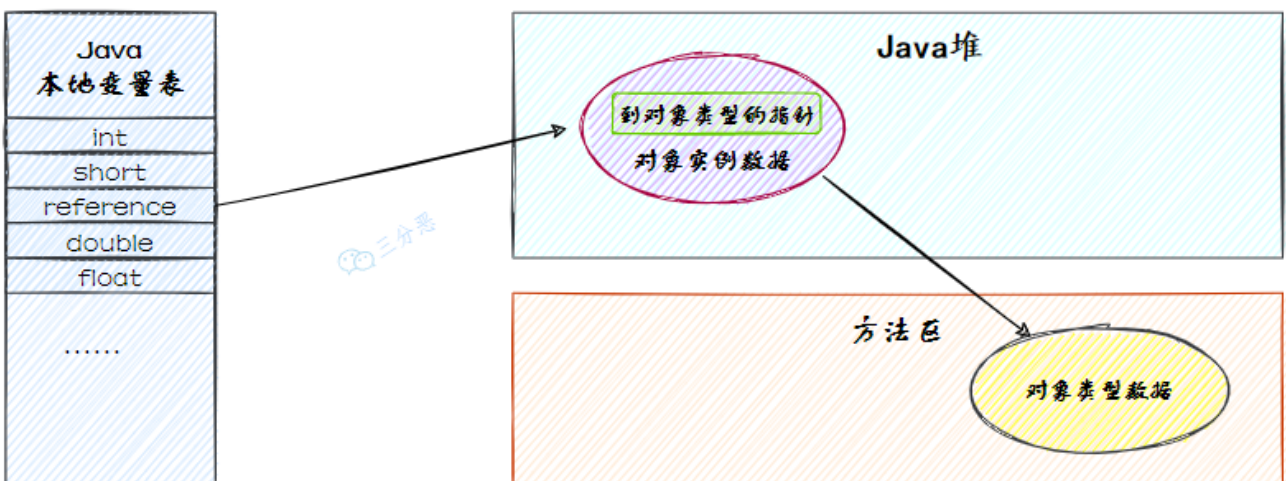
两种方式的区别在于，句柄是通过一个中间的句柄表来定位对象的，而直接指针则是通过引用直接指向对象的内存地址。

优点是，对象被移动时只需要修改句柄表中的指针，而不需要修改对象引用本身。



通过句柄访问对象

在直接指针访问中，引用直接存储对象的内存地址；对象的实例数据和类型信息都存储在堆中固定的内存区域。优点是访问速度更快，因为少了一次句柄的寻址操作。缺点是如果对象在内存中移动，引用需要更新为新的地址。

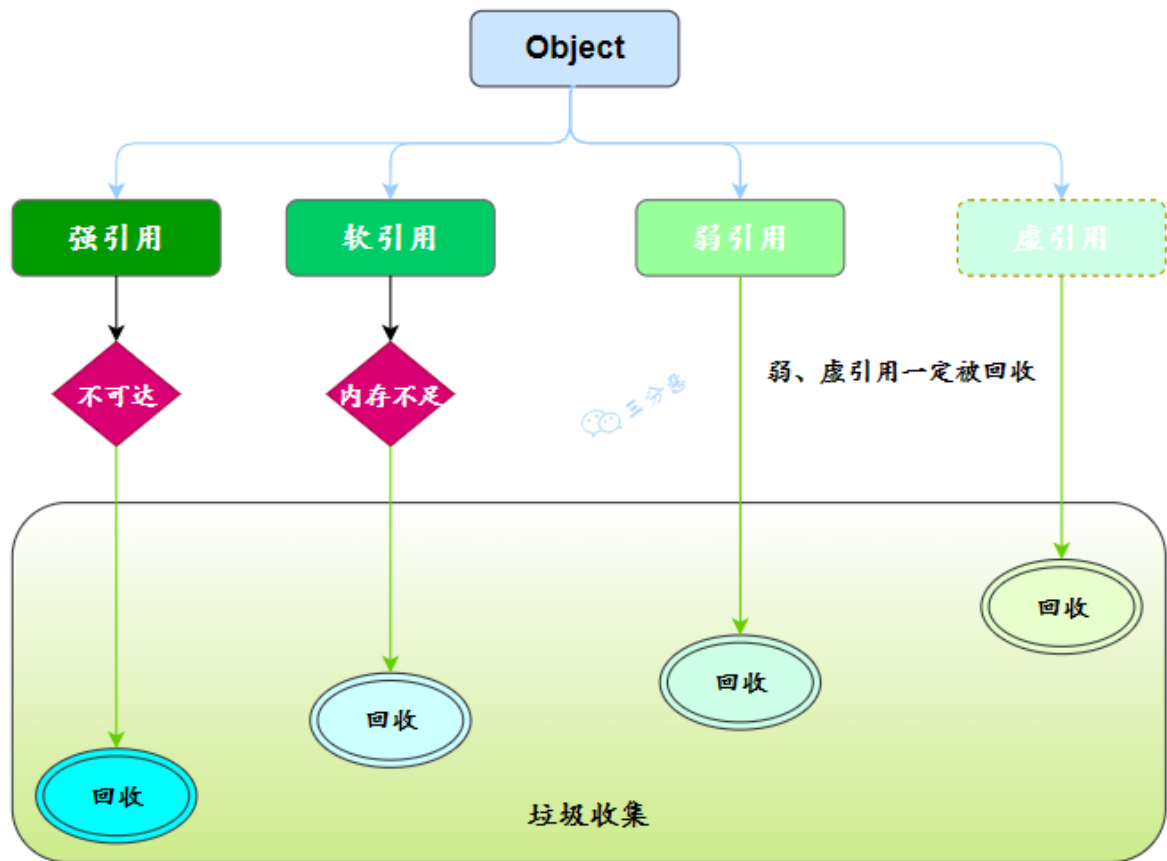


通过直接指针访问对象

HotSpot 虚拟机主要使用直接指针来进行对象访问。

11.说一下对象有哪几种引用？

四种，分别是强引用、软引用、弱引用和虚引用。



强引用是 Java 中最常见的引用类型。使用 `new` 关键字赋值的引用就是强引用，只要强引用关联着对象，垃圾收集器就不会回收这部分对象，即使内存不足。

```
// str 就是一个强引用
String str = new String("沉默王二");
```

软引用用于描述一些非必须对象，通过 `SoftReference` 类实现。软引用的对象在内存不足时会被回收。

```
// softRef 就是一个软引用
SoftReference<String> softRef = new SoftReference<>(new String("沉默王二"));
```

弱引用用于描述一些短生命周期的非必须对象，如 `ThreadLocal` 中的 `Entry`，就是通过 `WeakReference` 类实现的。弱引用的对象会在下一次垃圾回收时会被回收，不论内存是否充足。

```
static class Entry extends WeakReference<ThreadLocal<?>> {
    /** The value associated with this ThreadLocal. */
    Object value;

    //节点类
    Entry(ThreadLocal<?> k, Object v) {
        //key赋值
        super(k);
        //value赋值
        value = v;
    }
}
```

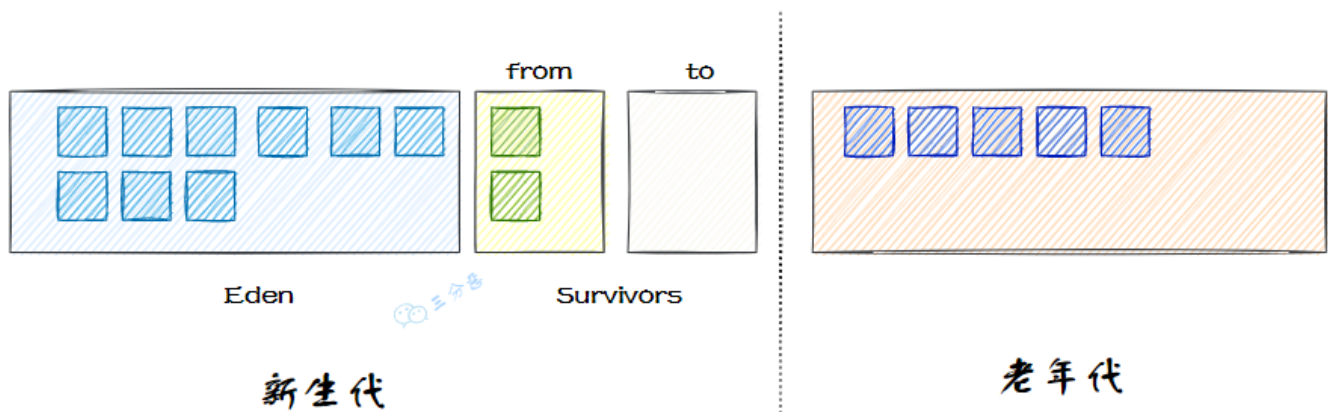
虚引用主要用来跟踪对象被垃圾回收的过程，通过 PhantomReference 类实现。虚引用的对象在任何时候都可能被回收。

```
// phantomRef 就是一个虚引用
PhantomReference<String> phantomRef = new PhantomReference<>(new String("沉默王二"), new
ReferenceQueue<>());
```

1. [Java 面试指南（付费）](#) 收录的京东同学 4 云实习面试原题：四个引用(强软弱虚)

12.Java 堆的内存分区了解吗？

了解。Java 堆被划分为新生代和老年代两个区域。



新生代又被划分为 Eden 空间和两个 Survivor 空间（From 和 To）。

新创建的对象会被分配到 Eden 空间。当 Eden 区填满时，会触发一次 Minor GC，清除不再使用的对象。存活下来的对象会从 Eden 区移动到 Survivor 区。

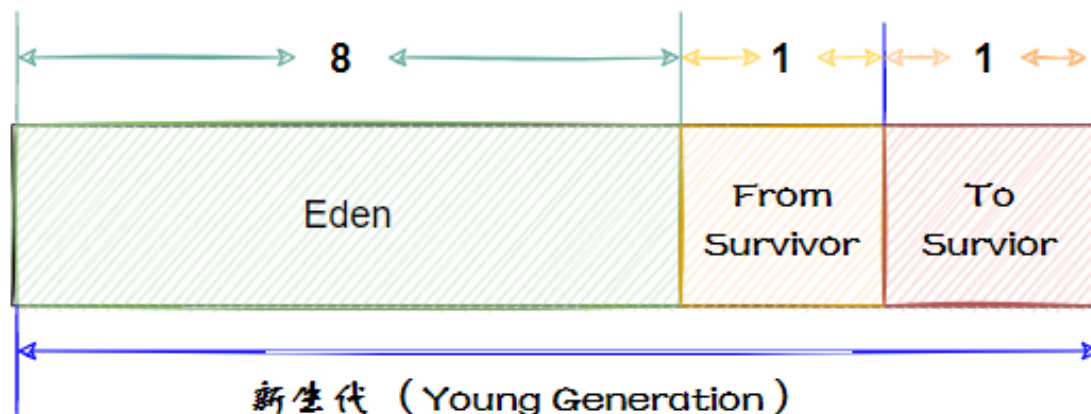
对象在新生代中经历多次 GC 后，如果仍然存活，会被移动到老年代。当老年代内存不足时，会触发 Major GC，对整个堆进行垃圾回收。

1. [Java 面试指南（付费）](#) 收录的得物面经同学 8 一面面试原题：Java 中堆内存怎么组织的
2. [Java 面试指南（付费）](#) 收录的腾讯面经同学 27 云后台技术一面面试原题：怎么来区分对象是属于哪个代的？

13.说一下新生代的区域划分？

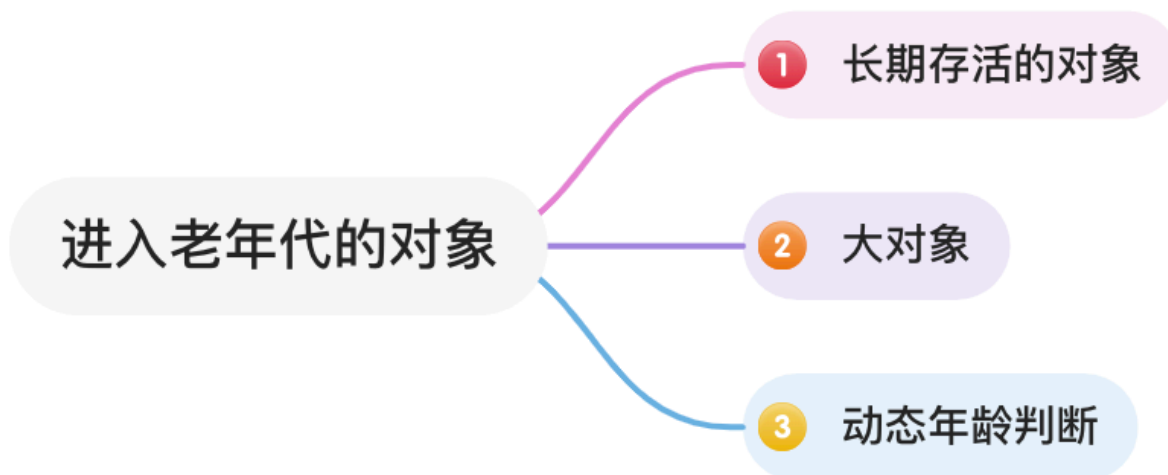
新生代的垃圾收集主要采用标记-复制算法，因为新生代的存活对象比较少，每次复制少量的存活对象效率比较高。

基于这种算法，虚拟机将内存分为一块较大的 Eden 空间和两块较小的 Survivor 空间，每次分配内存只使用 Eden 和其中一块 Survivor。发生垃圾收集时，将 Eden 和 Survivor 中仍然存活的对象一次性复制到另外一块 Survivor 空间上，然后直接清理掉 Eden 和已用过的那块 Survivor 空间。默认 Eden 和 Survivor 的大小比例是 8:1。



14.🌟对象什么时候会进入老年代？

对象通常会在年轻代中分配，随着时间的推移和垃圾收集的进程，某些满足条件的对象会进入到老年代中，如长期存活的对象。



长期存活的对象如何判断？

JVM 会为对象维护一个“年龄”计数器，记录对象在新生代中经历 Minor GC 的次数。每次 GC 未被回收的对象，其年龄会加 1。

当超过一个特定阈值，默认值是 15，就会被认为老对象了，需要重点关照。这个年龄阈值可以通过 JVM 参数 `-XX:MaxTenuringThreshold` 来设置。

可以通过 `jinfo -flag MaxTenuringThreshold $(jps | grep -i nacos | awk '{print $1}')` 来查看当前 JVM 的年龄阈值。

```
~/Downloads/nacos2/nacos/bin (0.433s)
jinfo -flag MaxTenuringThreshold $(jps | grep -i nacos | awk '{print $1}')
```

`-XX:MaxTenuringThreshold=15`

1. 如果应用中的对象存活时间较短，可以适当调大这个值，让对象在新生代多待一会儿
2. 如果对象存活时间较长，可以适当调小这个值，让对象更快进入老年代，减少在新生代的复制次数

大对象如何判断？

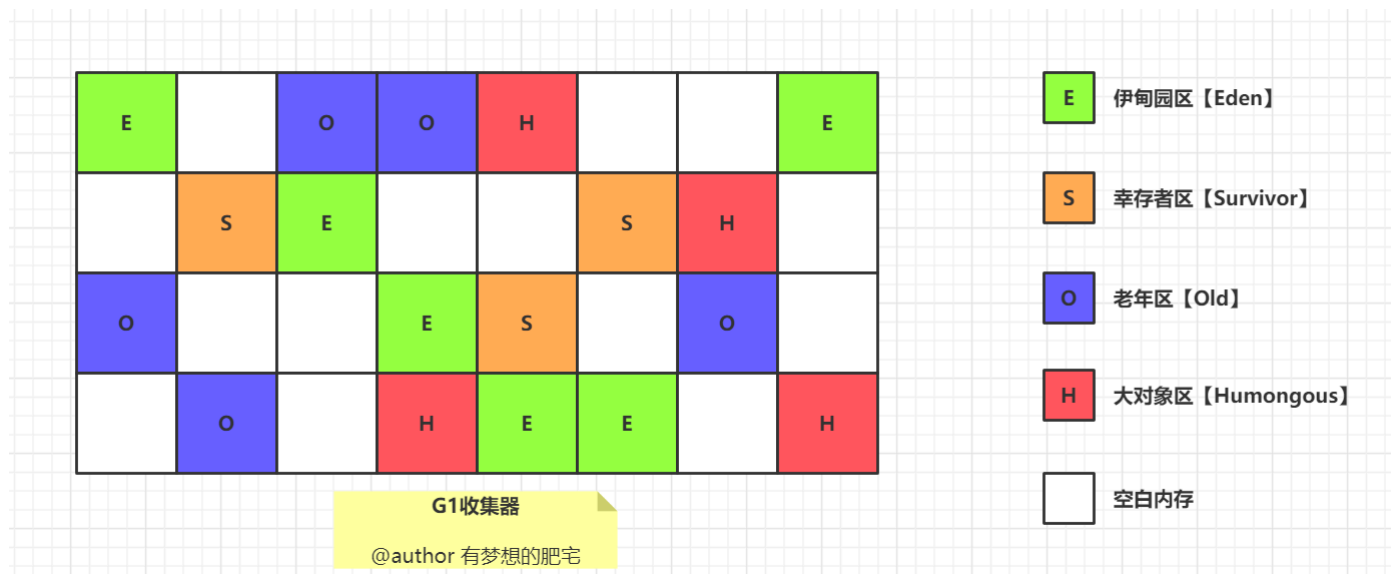
大对象是指占用内存较大的对象，如大数组、长字符串等。

```
int[] array = new int[1000000];
String str = new String(new char[1000000]);
```

其大小由 JVM 参数 `-XX:PretenureSizeThreshold` 控制，但在 JDK 8 中，默认值为 0，也就是说默认情况下，对象仅根据 GC 存活的次数来判断是否进入老年代。

```
[root@VM-8-2-opencloudos ~]# java -XX:+PrintFlagsFinal -version | grep PretenureSizeThreshold
uintx PretenureSizeThreshold = 0 {product}
openjdk version "1.8.0_432"
OpenJDK Runtime Environment (Tencent Kona 8.0.20) (build 1.8.0_432-1)
OpenJDK 64-Bit Server VM (Tencent Kona 8.0.20) (build 25.432-b1, mixed mode, sharing)
```

G1 垃圾收集器中，大对象会直接分配到 HUMONGOUS 区域。当对象大小超过一个 Region 容量的 50% 时，会被认为是大对象。



Region 的大小可以通过 JVM 参数 `-XX:G1HeapRegionSize` 来设置，默认情况下从 1MB 到 32MB 不等，会根据堆内存大小动态调整。

可以通过 `java -XX:+UseG1GC -XX:+PrintGCDetails -version` 查看 G1 垃圾收集器的相关信息。


```

java -XX:+UseG1GC -XX:+PrintGCDetails -version
openjdk version "1.8.0_412"
OpenJDK Runtime Environment Corretto-8.412.08.1 (build 1.8.0_412-b08)
OpenJDK 64-Bit Server VM Corretto-8.412.08.1 (build 25.412-b08, mixed mode)
Heap
  garbage-first heap    total 2097152K, used 0K [0x0000000030080000, 0x00000000300c0100, 0x00000000a7e80000)
    region size 4096K, 1 young (4096K), 0 survivors (0K)
  Metaspace            used 2227K, capacity 4480K, committed 4480K, reserved 1056768K
    class space        used 241K, capacity 384K, committed 384K, reserved 1048576K

```

从结果上来看，我本机上 G1 的堆大小为 2GB，Region 的大小为 4MB。

动态年龄判定了解吗？

如果 Survivor 区中所有对象的总大小超过了一定比例，通常是 Survivor 区的一半，那么年龄较小的对象也可能被提前晋升到老年代。

这是因为如果年龄较小的对象在 Survivor 区中占用了较大的空间，会导致 Survivor 区中的对象复制次数增多，影响垃圾回收的效率。

1. [Java 面试指南（付费）](#) 收录的阿里面经同学 5 阿里妈妈 Java 后端技术一面面试原题：哪些情况下对象会进入老年代？
2. [Java 面试指南（付费）](#) 收录的京东面经同学 7 Java 后端技术一面面试原题：新生代对象转移到老年代的条件
3. [Java 面试指南（付费）](#) 收录的拼多多面经同学 4 技术一面面试原题：对象什么时候进入老年代

memo：2025 年 1 月 13 日修改到此

15.STW 了解吗？

了解。

JVM 进行垃圾回收的过程中，会涉及到对象的移动，为了保证对象引用在移动过程中不被修改，必须暂停所有的用户线程，像这样的停顿，我们称之为 `Stop The World`。简称 STW。

如何暂停线程呢？

JVM 会使用一个名为安全点（Safe Point）的机制来确保线程能够被安全地暂停，其过程包括四个步骤：

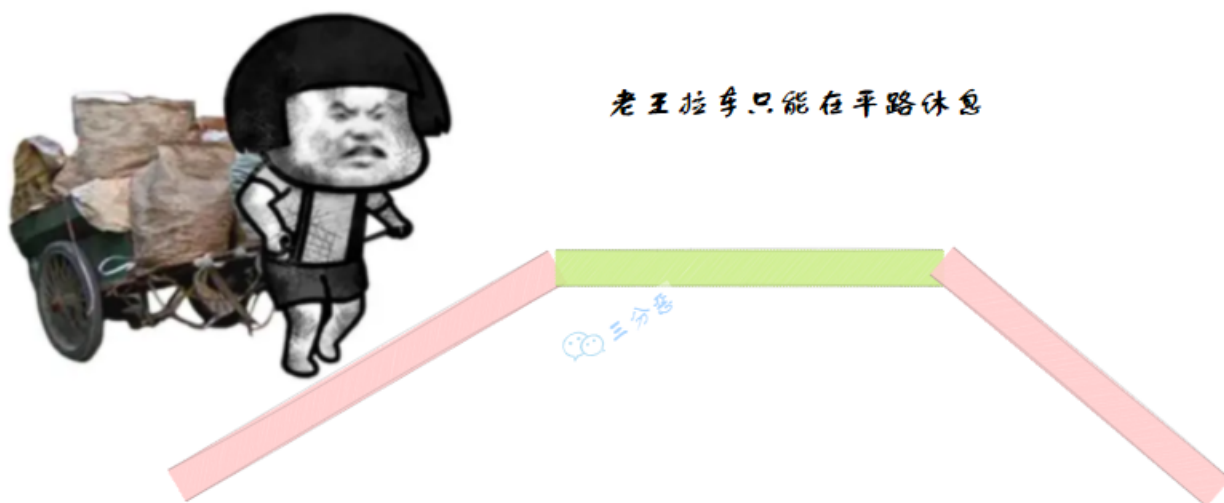
- JVM 发出暂停信号；
- 线程执行到安全点后，挂起自身并等待垃圾收集完成；
- 垃圾回收器完成 GC 操作；
- 线程恢复执行。

什么是安全点？

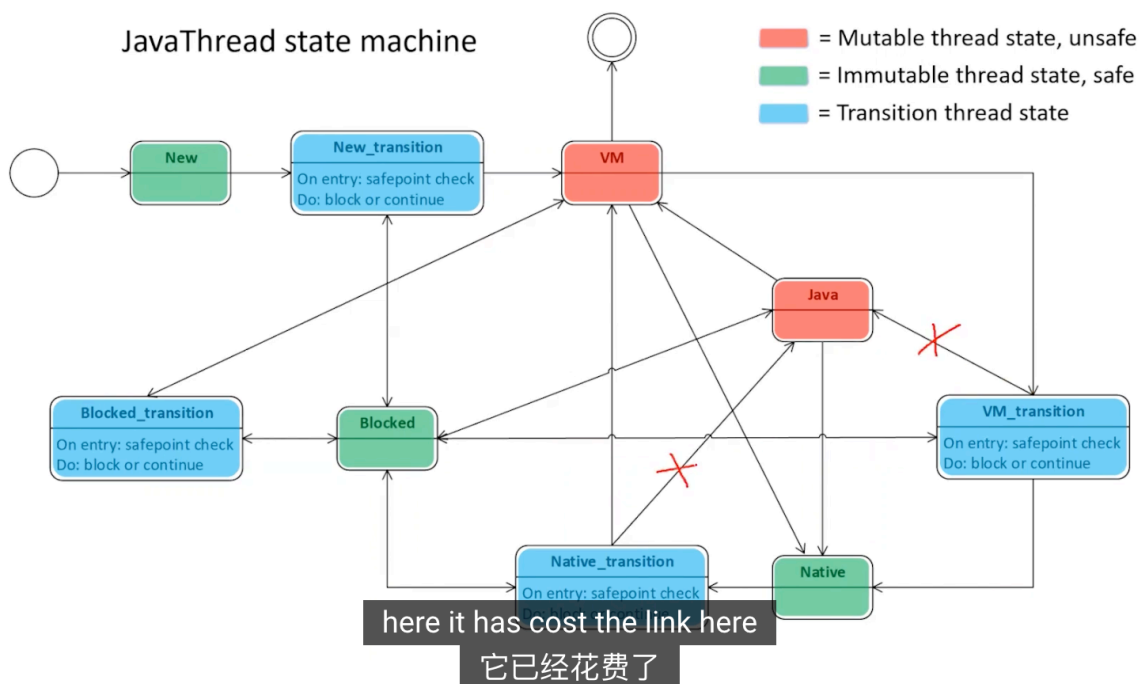
安全点是 JVM 的一种机制，常用于垃圾回收的 STW 操作，用于让线程在执行到某些特定位置时，可以被安全地暂停。

通常位于方法调用、循环跳转、异常处理等位置，以保证线程暂停时数据的一致性。

用个通俗的比喻，老王去拉车，车上的东西很重，老王累的汗流浹背，但是老王不能在上坡或者下坡时休息，只能在平地上停下来擦擦汗，喝口水。



推荐大家看看这个[HotSpot JVM Deep Dive - Safepoint](#)，对 safe point 有一个比较深入地解释。



HotSpot JVM Deep Dive - Safepoint

16.对象一定分配在堆中吗？

不一定。

默认情况下，Java 对象是在堆中分配的，但 JVM 会进行逃逸分析，来判断对象的生命周期是否只在方法内部，如果是的话，这个对象可以在栈上分配。

举例来说，下面的代码中，对象 `new Person()` 的生命周期只在 `testStackAllocation` 方法内部，因此 JVM 会将这个对象分配在栈上。

```
public void testStackAllocation() {
    Person p = new Person(); // 对象可能分配在栈上
    p.name = "沉默王二是只狗";
    p.age = 18;
    System.out.println(p.name);
}
```

什么是逃逸分析？

逃逸分析是一种 JVM 优化技术，用来分析对象的作用域和生命周期，判断对象是否逃逸出方法或线程。

可以通过分析对象的引用流向，判断对象是否被方法返回、赋值到全局变量、传递到其他线程等，来确定对象是否逃逸。

如果对象没有逃逸，就可以进行栈上分配、同步消除、标量替换等优化，以提高程序的性能。

可以通过 `java -XX:+PrintFlagsFinal -version | grep DoEscapeAnalysis` 来确认 JVM 是否开启了逃逸分析。

```
~/Downloads/sentinel (0.093s)
java -XX:+PrintFlagsFinal -version | grep DoEscapeAnalysis
    bool DoEscapeAnalysis                = true
                                {C2 product}
openjdk version "1.8.0_412"
OpenJDK Runtime Environment Corretto-8.412.08.1 (build 1.8.0_412-b08)
OpenJDK 64-Bit Server VM Corretto-8.412.08.1 (build 25.412-b08, mixed mode)
```

逃逸具体是指什么？

根据对象逃逸的范围，可以分为方法逃逸和线程逃逸。

当对象被方法外部的代码引用，生命周期超出了方法的范围，那么对象就必须分配在堆中，由垃圾收集器管理。

```
public Person createPerson() {
    return new Person(); // 对象逃逸出方法
}
```

比如说 `new Person()` 创建的对象被返回，那么这个对象就逃逸出当前方法了。

未逃逸

```

public static void returnStr() {
    User user = new User();
    user.setId(1);
    user.setName("张三");
    user.setAge(18);
}

public static String returnStr() {
    User user = new User();
    user.setId(1);
    user.setName("张三");
    user.setAge(18);
    return user.toString(); // 这里User要实现get, set方法, 还要实现toString方法
}

```

局部对象user未被外部调用

方法逃逸

```

public static User returnStr() {
    User user = new User();
    user.setId(1);
    user.setName("张三");
    user.setAge(18);
    return user; // 这里User要实现get, set方法
}

```

局部对象user可能会被外部调用

再比如说, 对象被另外一个线程引用, 生命周期超出了当前线程, 那么对象就必须分配在堆中, 并且线程之间需要同步。

```

public void threadEscapeExample() {
    Person p = new Person(); // 对象逃逸到另一个线程
    new Thread(() -> {
        System.out.println(p);
    }).start();
}

```

对象 `new Person()` 被另外一个线程引用了, 发生了线程逃逸。

逃逸分析会带来什么好处?

主要有三个。

第一, 如果确定一个对象不会逃逸, 那么就可以考虑栈上分配, 对象占用的内存随着栈帧出栈后销毁, 这样一来, 垃圾收集的压力就降低很多。

第二, 线程同步需要加锁, 加锁就要占用系统资源, 如果逃逸分析能够确定一个对象不会逃逸出线程, 那么这个对象就不用加锁, 从而减少线程同步的开销。

第三，如果对象的字段在方法中独立使用，JVM 可以将对象分解为标量变量，避免对象分配。

```
public void scalarReplacementExample() {
    Point p = new Point(1, 2);
    System.out.println(p.getX() + p.getY());
}
```

如果 Point 对象未逃逸，JVM 可以优化为：

```
int x = 1;
int y = 2;
System.out.println(x + y);
```

1. [Java 面试指南（付费）](#) 收录的收钱吧面经同学 1 Java 后端一面面试原题：所有对象都在堆上对不对？

17.内存溢出和内存泄漏了解吗？

内存溢出，俗称 OOM，是指当程序请求分配内存时，由于没有足够的内存空间，从而抛出 OutOfMemoryError。

```
List<String> list = new ArrayList<>();
while (true) {
    list.add("OutOfMemory".repeat(1000)); // 无限增加内存
}
```

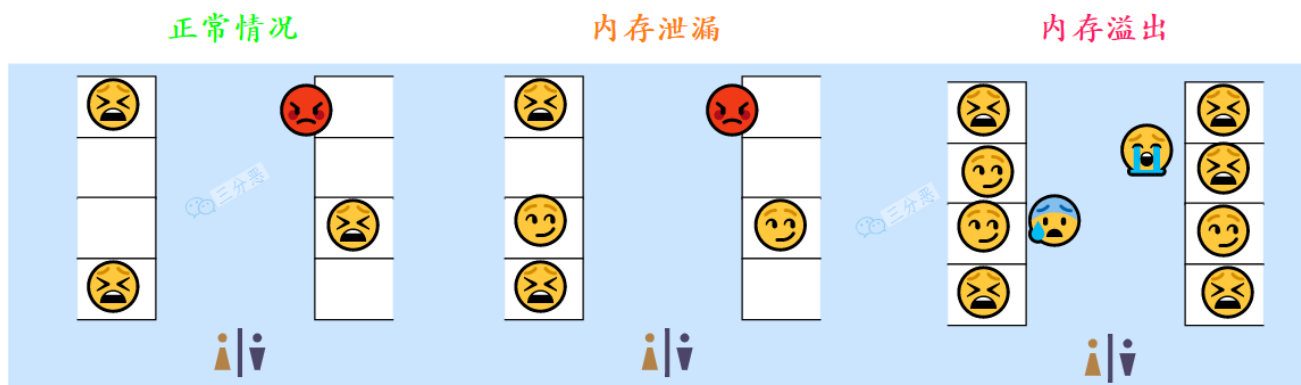
可能是因为堆、元空间、栈或直接内存不足导致的。可以通过优化内存配置、减少对象分配来解决。

内存泄漏是指程序在使用完内存后，未能及时释放，导致占用的内存无法再被使用。随着时间的推移，内存泄漏会导致可用内存逐渐减少，最终导致内存溢出。

内存泄漏通常是因为长期存活的对象持有短期存活对象的引用，又没有及时释放，从而导致短期存活对象无法被回收而导致的。

```
class MemoryLeakExample {
    private static List<Object> staticList = new ArrayList<>();
    public void addObject() {
        staticList.add(new Object()); // 对象不会被回收
    }
}
```

用一个比较有味道的比喻来形容就是，内存溢出是排队去蹲坑，发现没坑了；内存泄漏，就是有人占着茅坑不拉屎，导致坑位不够用。



1. [Java 面试指南 \(付费\)](#) 收录的京东面经同学 1 Java 技术一面面试原题：说说 OOM 的原因
2. [Java 面试指南 \(付费\)](#) 收录的快手面经同学 1 部门主站技术部面试原题：了解 OOM 吗？

18.能手写内存溢出的例子吗？

可以。

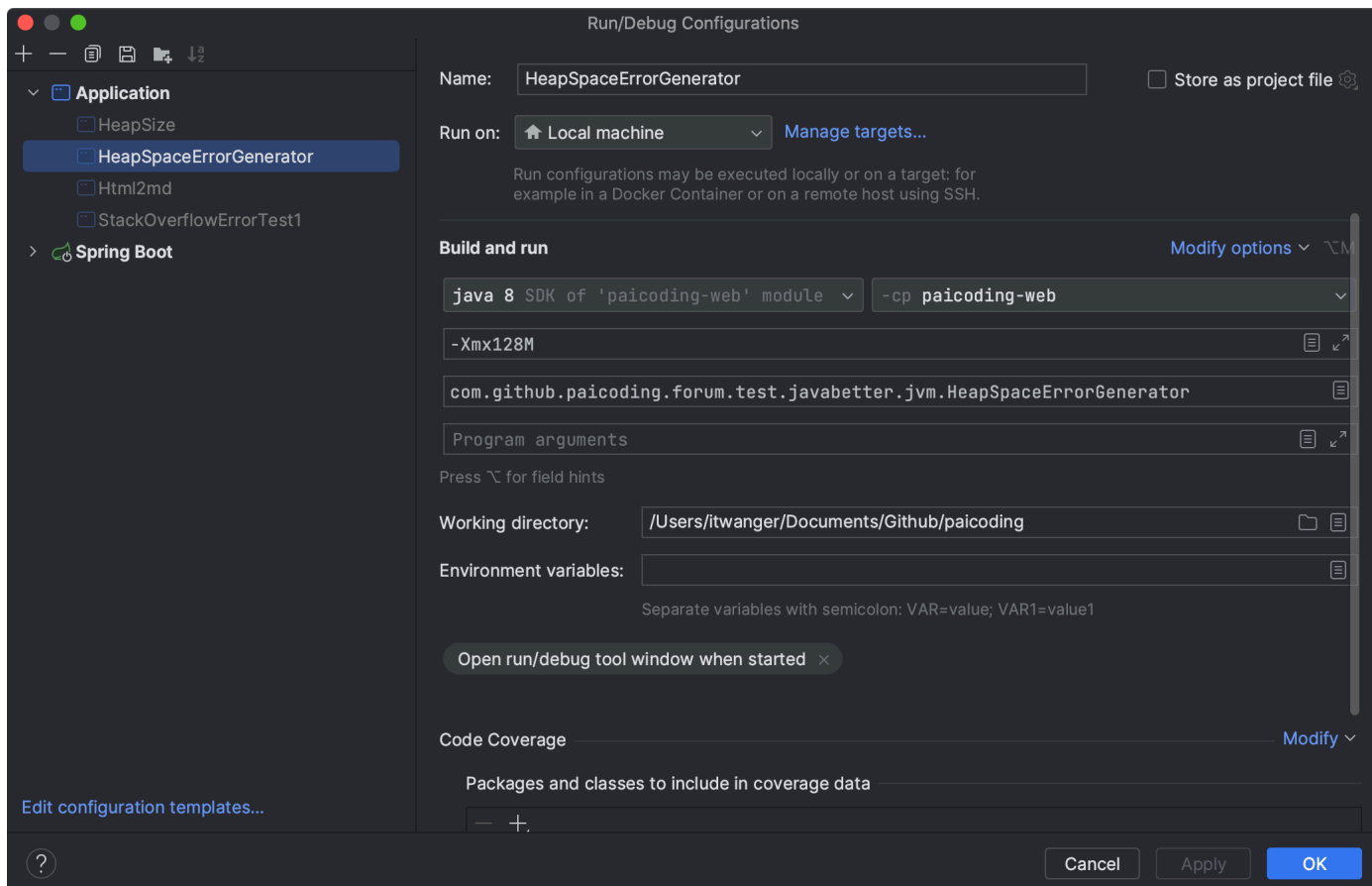
我就拿最常见的堆内存溢出来完成吧，堆内存溢出通常是因为创建了大量的对象，且长时间无法被垃圾收集器回收，导致的。

```
class HeapSpaceErrorGenerator {
    public static void main(String[] args) {
        // 第一步，创建一个大的容器
        List<byte[]> bigObjects = new ArrayList<>();
        try {
            // 第二步，循环写入数据
            while (true) {
                // 第三步，创建一个大对象，一个大约 10M 的数组
                byte[] bigObject = new byte[10 * 1024 * 1024];
                // 第四步，将大对象添加到容器中
                bigObjects.add(bigObject);
            }
        } catch (OutOfMemoryError e) {
            System.out.println("OutOfMemoryError 发生在 " + bigObjects.size() + " 对象后");
            throw e;
        }
    }
}
```

很快就会发生内存溢出。

这就相当于一个房子里，不断堆积不能被回收的杂物，那么房子很快就会被堆满了。

也可以通过 VM 参数设置堆内存大小为 `-Xmx128M`，然后运行程序，出现的内存溢出的时间会更快。



可以看到，堆内存溢出发生在 11 个对象后。

```
/Library/Java/JavaVirtualMachines/jdk1.8.0_301.jdk/Contents/Home/bin/java ...
OutOfMemoryError 发生在 11 对象后
Exception in thread "main" java.lang.OutOfMemoryError: Create breakpoint : Java heap space
    at com.github.paicoding.forum.test.javabetter.jvm.HeapSpaceErrorGenerator.main(HeapSpaceErrorGenerator.java:12)

Process finished with exit code 1
```

1. [Java 面试指南（付费）](#) 收录的京东面经同学 1 Java 技术一面面试原题：说说 OOM 的原因
2. [Java 面试指南（付费）](#) 收录的快手面经同学 1 部门主站技术部面试原题：Java 哪些内存区域会发生 OOM？为什么？

memo：2025 年 1 月 14 日修改到此

19.内存泄漏可能由哪些原因导致呢？

比如说：

①、静态的集合中添加的对象越来越多，但却没有及时清理；静态变量的生命周期与应用程序相同，如果静态变量持有对象的引用，这些对象将无法被 GC 回收。

```
class OOM {
    static List list = new ArrayList();

    public void oomTests(){
        Object obj = new Object();

        list.add(obj);
    }
}
```

②、单例模式下对象持有的外部引用无法及时释放；单例对象在整个应用程序的生命周期中存活，如果单例对象持有其他对象的引用，这些对象将无法被回收。

```
class Singleton {
    private static final Singleton INSTANCE = new Singleton();
    private List<Object> objects = new ArrayList<>();

    public static Singleton getInstance() {
        return INSTANCE;
    }
}
```

③、数据库、IO、Socket 等连接资源没有及时关闭；

```
try {
    Connection conn = null;
    Class.forName("com.mysql.jdbc.Driver");
    conn = DriverManager.getConnection("url", "", "");
    Statement stmt = conn.createStatement();
    ResultSet rs = stmt.executeQuery("....");
} catch (Exception e) {

} finally {
    //不关闭连接
}
```

④、ThreadLocal 的引用未被清理，线程退出后仍然持有对象引用；在线程执行完后，要调用 ThreadLocal 的 remove 方法进行清理。

```
ThreadLocal<Object> threadLocal = new ThreadLocal<>();
threadLocal.set(new Object()); // 未清理
```

20.有没有处理过内存泄漏问题？

推荐阅读：

1. [一次内存溢出的排查优化实战](#)
2. [JVM 性能监控工具之命令行篇](#)

3. [JVM 性能监控工具之可视化篇](#)

有。

当时在做[技术派](#)项目的时候，由于 ThreadLocal 没有及时清理导致出现了内存泄漏问题。

我用可视化的监控工具 VisualVM，配合 JDK 自带的 jstack 等命令行工具进行了排查。

大致的过程我回想了一下，主要有 7 个步骤：

第一步，使用 `jps -l` 查看运行的 Java 进程 ID。

```
[root@VM-0-5-tencentos ~]# jps -l
391195 paicoding-web-0.0.1-SNAPSHOT.jar
4170448 sun.tools.jps.Jps
```

第二步，使用 `top -p [pid]` 查看进程使用 CPU 和内存占用情况。

```
top - 09:00:43 up 270 days, 21:29, 1 user, load average: 0.01, 0.02, 0.00
Tasks: 1 total, 0 running, 1 sleeping, 0 stopped, 0 zombie
%Cpu(s): 0.0 us, 0.0 sy, 0.0 ni,100.0 id, 0.0 wa, 0.0 hi, 0.0 si, 0.0 st
MiB Mem : 3670.2 total, 210.3 free, 2429.9 used, 1030.0 buff/cache
MiB Swap: 0.0 total, 0.0 free, 0.0 used. 753.4 avail Mem
```

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
391195	admin	20	0	4281324	1.0g	16052	S	0.0	27.9	19:53.58	java

第三步，使用 `top -Hp [pid]` 查看进程下的所有线程占用 CPU 和内存情况。

```
top - 09:01:55 up 270 days, 21:30, 1 user, load average: 0.00, 0.01, 0.00
Threads: 52 total, 0 running, 52 sleeping, 0 stopped, 0 zombie
%Cpu(s): 0.0 us, 3.3 sy, 0.0 ni, 96.7 id, 0.0 wa, 0.0 hi, 0.0 si, 0.0 st
MiB Mem : 3670.2 total, 209.3 free, 2430.7 used, 1030.2 buff/cache
MiB Swap: 0.0 total, 0.0 free, 0.0 used. 752.6 avail Mem
```

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
391195	admin	20	0	4281324	1.0g	16052	S	0.0	27.9	0:00.00	java
391196	admin	20	0	4281324	1.0g	16052	S	0.0	27.9	0:13.78	java
391197	admin	20	0	4281324	1.0g	16052	S	0.0	27.9	0:06.05	GC task thread#
391198	admin	20	0	4281324	1.0g	16052	S	0.0	27.9	0:06.12	GC task thread#
391199	admin	20	0	4281324	1.0g	16052	S	0.0	27.9	0:20.28	VM Thread
391200	admin	20	0	4281324	1.0g	16052	S	0.0	27.9	0:00.03	Reference Handl
391201	admin	20	0	4281324	1.0g	16052	S	0.0	27.9	0:00.04	Finalizer
391202	admin	20	0	4281324	1.0g	16052	S	0.0	27.9	0:00.00	Signal Dispatch
391203	admin	20	0	4281324	1.0g	16052	S	0.0	27.9	1:53.88	C2 CompilerThre
391204	admin	20	0	4281324	1.0g	16052	S	0.0	27.9	0:21.92	C1 CompilerThre
391205	admin	20	0	4281324	1.0g	16052	S	0.0	27.9	0:00.00	Service Thread
391206	admin	20	0	4281324	1.0g	16052	S	0.0	27.9	2:09.43	VM Periodic Tas
391281	admin	20	0	4281324	1.0g	16052	S	0.0	27.9	0:06.82	mysql-cj-abando
391292	admin	20	0	4281324	1.0g	16052	S	0.0	27.9	0:08.83	pool-1-thread-1
391293	admin	20	0	4281324	1.0g	16052	S	0.0	27.9	0:08.62	I/O dispatcher
391294	admin	20	0	4281324	1.0g	16052	S	0.0	27.9	0:08.87	I/O dispatcher
391295	admin	20	0	4281324	1.0g	16052	S	0.0	27.9	0:18.74	Catalina-utilit

第四步，抓取线程栈：`jstack -F 29452 > 29452.txt`，可以多抓几次做个对比。

29452 为 pid，顺带作为文件名。

```
[root@VM-0-5-tencentos logs]# jstack -F 391197 > 391197.txt
```

```
[root@VM-0-5-tencentos logs]# ls
```

```
391197.txt  arthas  arthas-cache
```

看看有没有线程死锁、死循环或长时间等待这些问题。

```
Attaching to process ID 391197, please wait...
```

```
Debugger attached successfully.
```

```
Server compiler detected.
```

```
JVM version is 25.382-b05
```

```
Deadlock Detection:
```

```
No deadlocks found.
```

```
Thread 4180746: (state = BLOCKED)
```

```
- sun.misc.Unsafe.park(boolean, long) @bci=0 (Compiled frame; information may be imprecise)
- java.util.concurrent.locks.LockSupport.park(java.lang.Object) @bci=14, line=175 (Compiled frame)
- java.util.concurrent.SynchronousQueue$TransferStack.awaitFulfill(java.util.concurrent.SynchronousQueue$TransferStack, boolean, long) @bci=144, line=458 (Compiled frame)
- java.util.concurrent.SynchronousQueue$TransferStack.transfer(java.lang.Object, boolean, long) @bci=102, line=362 (Compiled frame)
- java.util.concurrent.SynchronousQueue.take() @bci=7, line=924 (Compiled frame)
- java.util.concurrent.ThreadPoolExecutor.getTask() @bci=149, line=1074 (Compiled frame)
- java.util.concurrent.ThreadPoolExecutor.runWorker(java.util.concurrent.ThreadPoolExecutor$Worker) @bci=26, line=1130 (Compiled frame)
- java.util.concurrent.ThreadPoolExecutor$Worker.run() @bci=5, line=624 (Compiled frame)
- java.lang.Thread.run() @bci=11, line=750 (Compiled frame)
```

```
Thread 4180745: (state = BLOCKED)
```

```
- sun.misc.Unsafe.park(boolean, long) @bci=0 (Compiled frame; information may be imprecise)
- java.util.concurrent.locks.LockSupport.park(java.lang.Object) @bci=14, line=175 (Compiled frame)
```

第五步，可以使用 `jstat -gcutil [pid] 5000 10` 每隔 5 秒输出 GC 信息，输出 10 次，查看 **YGC** 和 **Full GC** 次数。

```
[root@VM-0-5-tencentos logs]# jstat -gcutil 391195 5000 10
```

S0	S1	E	O	M	CCS	YGC	YGCT	FGC	FGCT	GCT
0.00	58.10	11.39	20.53	92.52	89.51	981	7.433	4	0.789	8.222
0.00	58.10	11.40	20.53	92.52	89.51	981	7.433	4	0.789	8.222
0.00	58.10	11.40	20.53	92.52	89.51	981	7.433	4	0.789	8.222

通常会出现 YGC 不增加或增加缓慢，而 Full GC 增加很快。

或使用 `jstat -gccause [pid] 5000` 输出 GC 摘要信息。

```
[root@VM-0-5-tencentos logs]# jstat -gccause 391195 5000
```

S0	S1	E	O	M	CCS	YGC	YGCT	FGC	FGCT	GCT	LGCC	GCC
0.00	58.10	34.32	20.53	92.52	89.51	981	7.433	4	0.789	8.222	Allocation Failure	No GC
0.00	58.10	34.33	20.53	92.52	89.51	981	7.433	4	0.789	8.222	Allocation Failure	No GC
0.00	58.10	34.33	20.53	92.52	89.51	981	7.433	4	0.789	8.222	Allocation Failure	No GC

或使用 `jmap -heap [pid]` 查看堆的摘要信息，关注老年代内存使用是否达到阈值，若达到阈值就会执行 Full GC。

```
^C[root@VM-0-5-tencentos logs]# jmap -heap 391195
Attaching to process ID 391195, please wait...
Debugger attached successfully.
Server compiler detected.
JVM version is 25.382-b05
```

```
using thread-local object allocation.
Parallel GC with 2 thread(s)
```

Heap Configuration:

MinHeapFreeRatio	= 0
MaxHeapFreeRatio	= 100
MaxHeapSize	= 1610612736 (1536.0MB)
NewSize	= 268435456 (256.0MB)
MaxNewSize	= 268435456 (256.0MB)
OldSize	= 1342177280 (1280.0MB)
NewRatio	= 2
SurvivorRatio	= 8
MetaspaceSize	= 21807104 (20.796875MB)
CompressedClassSpaceSize	= 1073741824 (1024.0MB)
MaxMetaspaceSize	= 17592186044415 MB
G1HeapRegionSize	= 0 (0.0MB)

Heap Usage:

PS Young Generation

如果发现 `Full GC` 次数太多，就很大概率存在内存泄漏了。

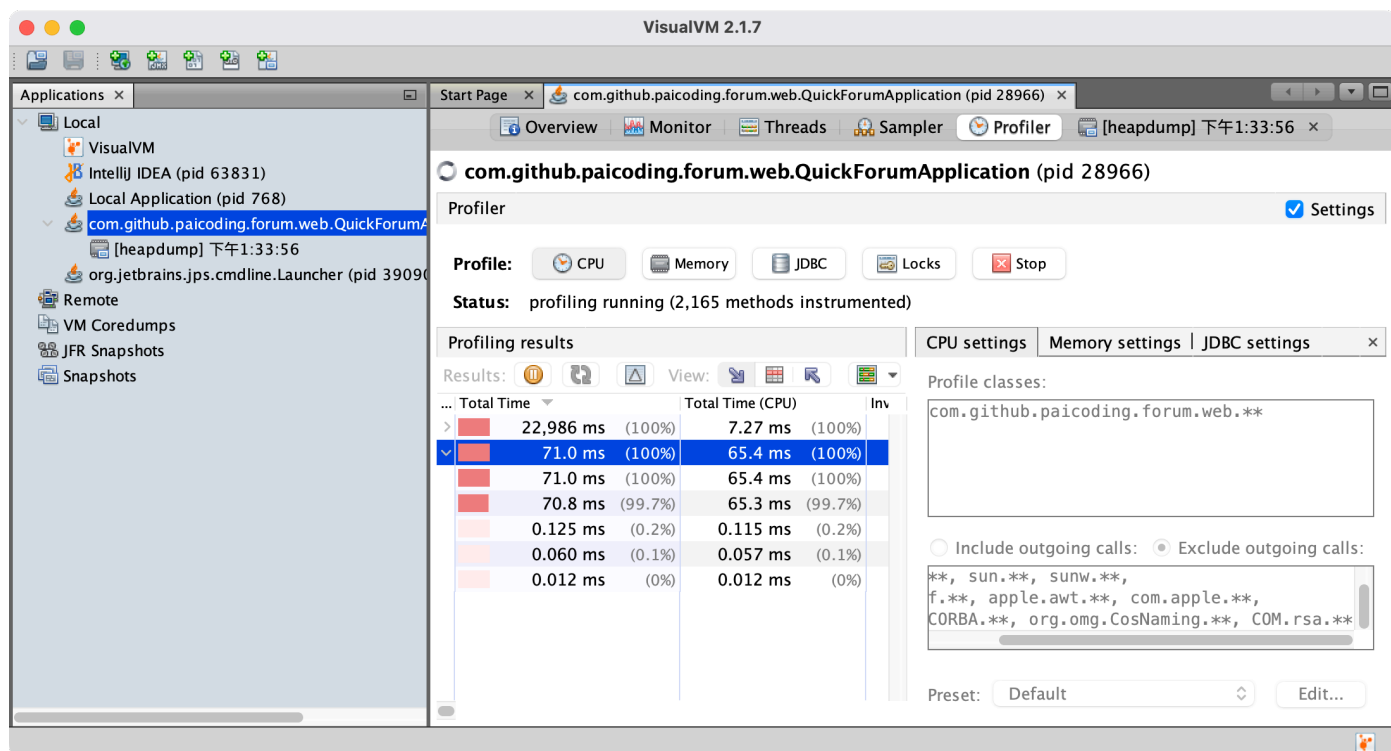
第六步，生成 `dump` 文件，然后借助可视化工具分析哪个对象非常多，基本就能定位到问题根源了。

执行命令 `jmap -dump:format=b,file=heap.hprof 10025` 会输出进程 10025 的堆快照信息，保存到文件 `heap.hprof` 中。


```
~/Documents/GitHub/HelloWorld git:(master) ±127 (0.318s)
/Library/Java/JavaVirtualMachines/jdk-11.0.8.jdk/Contents/Home/bin/
jmap -dump:format=b,file=heap.hprof 10025
Heap dump file created
```

```
~/Documents/GitHub/HelloWorld git:(master) ±127 (0.083s)
ls
Example.class HelloWorld.iml JDK11.java JDK8.java out
Example.java JDK11.class JDK8.class heap.hprof src
```

第七步，使用图形化工具分析，如 JDK 自带的 **VisualVM**，从菜单 > 文件 > 装入 dump 文件。



然后在结果观察内存占用最多的对象，找到内存泄漏的源头。

1. [Java 面试指南（付费）](#) 收录的京东同学 10 后端实习一面的原题：什么是内存泄露
2. [Java 面试指南（付费）](#) 收录的快手面经同学 1 部门主站技术部面试原题：Java 哪些内存区域会发生 OOM？为什么？
3. [Java 面试指南（付费）](#) 收录的美团面经同学 4 一面面试原题：内存泄漏怎么排查

21. 有没有处理过内存溢出问题？

有。

当时在做[技术派](#)的时候，由于上传的文件过大，没有正确处理，导致一下子撑爆了内存，程序直接崩溃了。

我记得是通过导出堆转储文件进行分析发现的。

第一步，使用 jmap 命令手动生成 Heap Dump 文件：

```
jmap -dump:format=b,file=heap.hprof <pid>
```

然后使用 MAT、JProfiler 等工具进行分析，查看内存中的对象占用情况。

一般来说：

如果生产环境的内存还有很多空余，可以适当增大堆内存大小来解决，例如 `-Xmx4g` 参数。

或者检查代码中是否存在内存泄漏，如未关闭的资源、长生命周期的对象等。

之后，在本地进行压力测试，模拟高负载情况下的内存表现，确保修改有效，且没有引入新的问题。

1. [Java 面试指南（付费）](#) 收录的华为面经同学 9 Java 通用软件开发一面面试原题：如何排查 OOM？

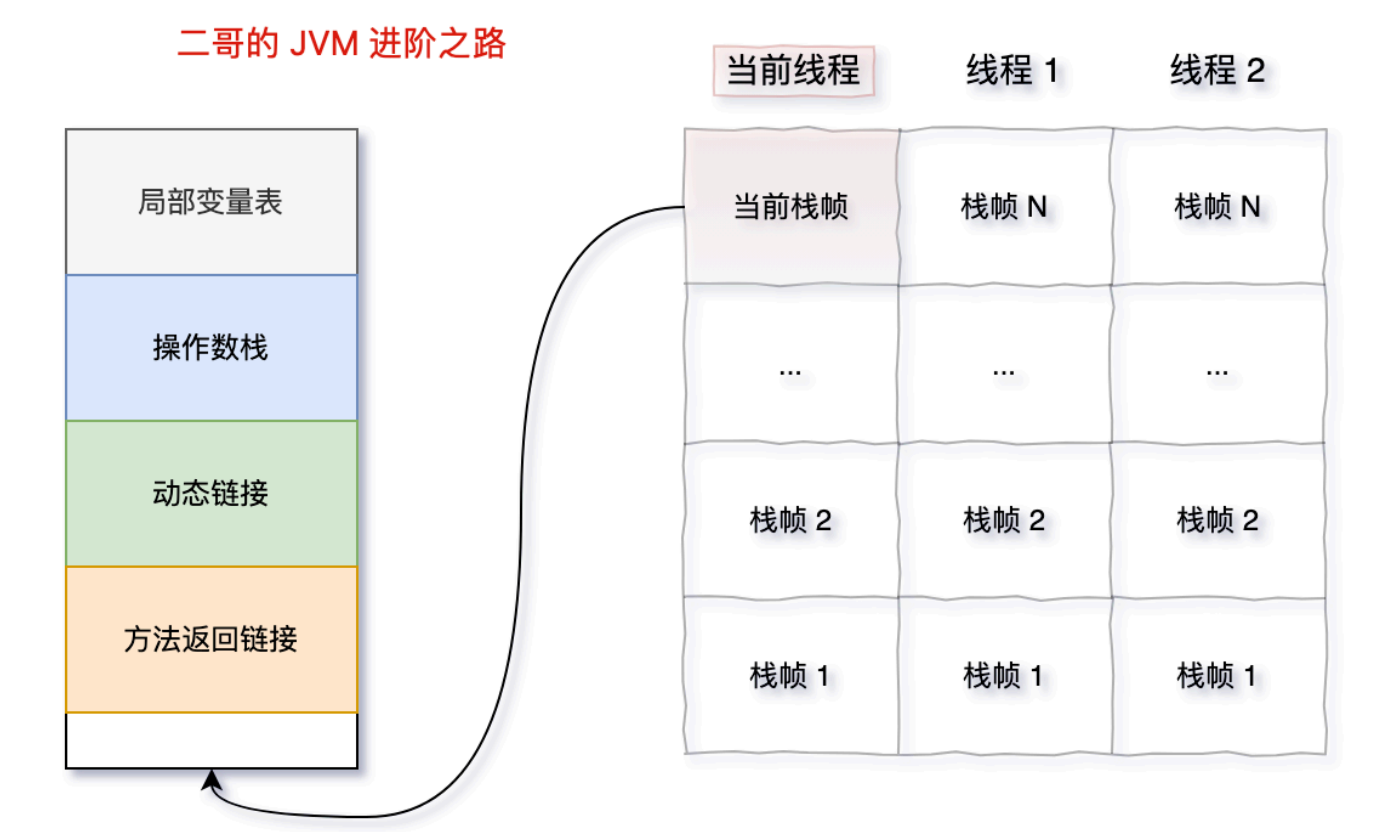
2. [Java 面试指南（付费）](#) 收录的荣耀面经同学 4 面试原题：有没遇到内存泄露，溢出的情况，怎么发生和处理的？

22.什么情况下会发生栈溢出？（补充）

2024 年 10 月 16 日增补

栈溢出发生在程序调用栈的深度超过 JVM 允许的最大深度时。

栈溢出的本质是因为线程的栈空间不足，导致无法再为新的栈帧分配内存。



当一个方法被调用时，JVM 会在栈中分配一个栈帧，用于存储该方法的执行信息。如果方法调用嵌套太深，栈帧不断压入栈中，最终会导致栈空间耗尽，抛出 `StackOverflowError`。

最常见的栈溢出场景就是递归调用，尤其是没有正确的终止条件下，会导致递归无限进行。

```
class StackOverflowExample {  
    public static void recursiveMethod() {  
        // 没有终止条件的递归调用  
        recursiveMethod();  
    }  
  
    public static void main(String[] args) {  
        recursiveMethod(); // 导致栈溢出  
    }  
}
```

另外，如果方法中定义了特别大的局部变量，栈帧会变得很大，导致栈空间更容易耗尽。

```
public class LargeLocalVariables {  
    public static void method() {  
        int[] largeArray = new int[1000000]; // 大量局部变量  
        method(); // 递归调用  
    }  
  
    public static void main(String[] args) {  
        method(); // 导致栈溢出  
    }  
}
```

1. [Java 面试指南 \(付费\)](#) 收录的 OPPO 面经同学 1 面试原题：什么情况下会发生栈溢出？

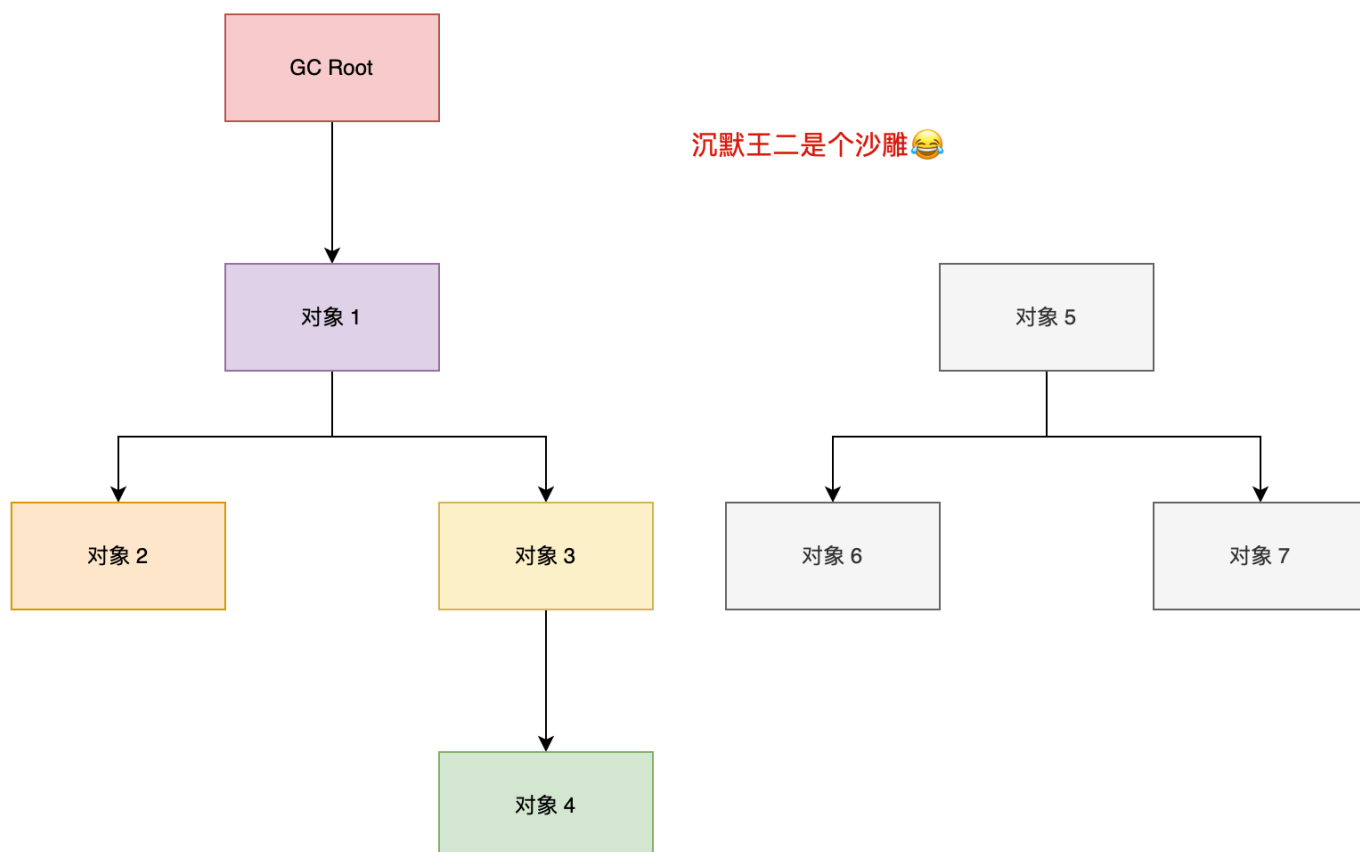
三、垃圾收集

23. 🌟 讲讲 JVM 的垃圾回收机制（补充）

本题是增补的内容，by 2024 年 03 月 09 日；参照：[深入理解 JVM 的垃圾回收机制](#)

垃圾回收就是对内存堆中已经死亡的或者长时间没有使用的对象进行清除或回收。

JVM 在做 GC 之前，会先搞清楚什么是垃圾，什么不是垃圾，通常会通过可达性分析算法来判断对象是否存活。



在确定了哪些垃圾可以被回收后，垃圾收集器（如 CMS、G1、ZGC）要做的事情就是进行垃圾回收，可以采用标记清除算法、复制算法、标记整理算法、分代收集算法等。

[技术派](#)项目使用的 JDK 8，采用的是 CMS 垃圾收集器。

```
java -XX:+UseConcMarkSweepGC \
  -XX:+UseParNewGC \
  -XX:CMSInitiatingOccupancyFraction=75 \
  -XX:+UseCMSInitiatingOccupancyOnly \
  -jar your-application.jar
```

垃圾回收的过程是什么？

Java 的垃圾回收过程主要分为标记存活对象、清除无用对象、以及内存压缩/整理三个阶段。不同的垃圾回收器在执行这些步骤时会采用不同的策略和算法。

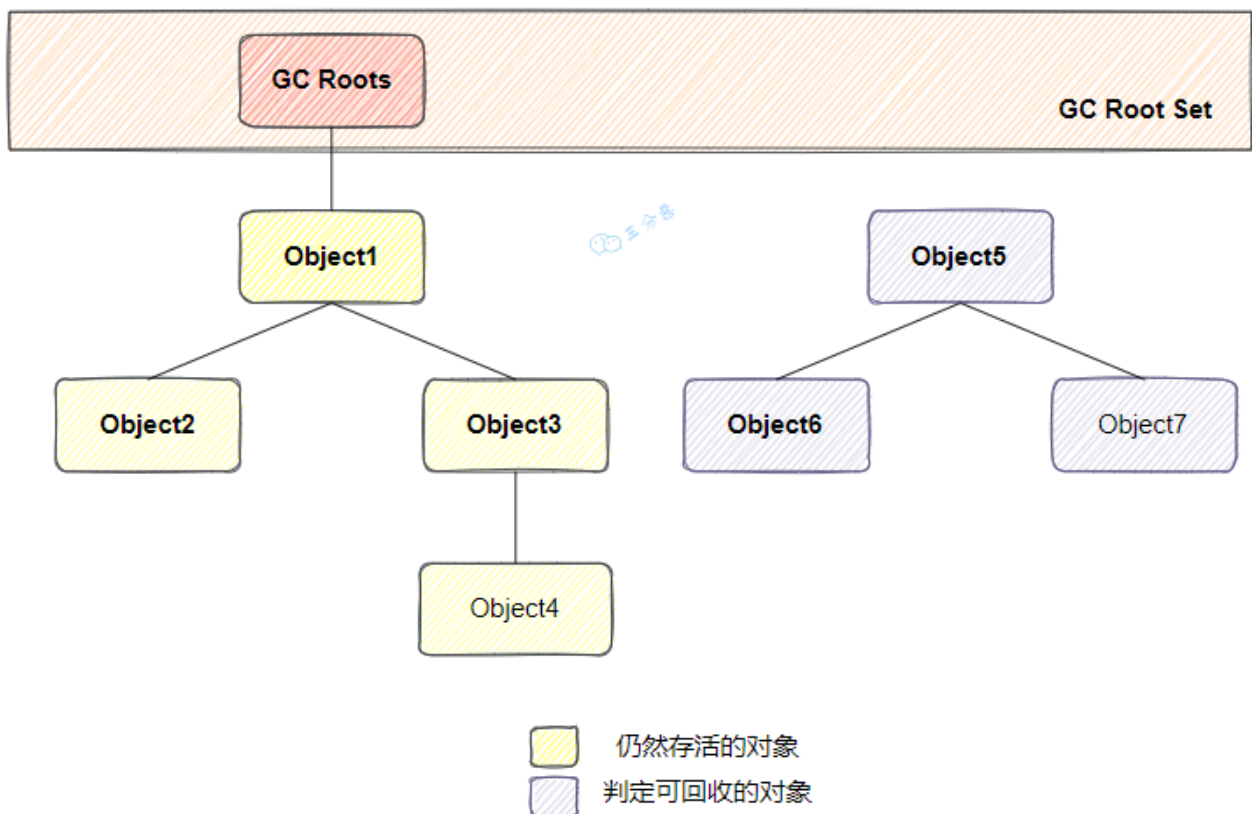
1. [Java 面试指南（付费）](#) 收录的华为 OD 技术一面遇到的一道原题。
2. [Java 面试指南（付费）](#) 收录的美团面经同学 2 Java 后端技术一面面试原题：了解 GC 吗？不可达判断知道吗？
3. [Java 面试指南（付费）](#) 收录的腾讯面经同学 26 暑期实习微信支付面试原题：JVM 垃圾删除
4. [Java 面试指南（付费）](#) 收录的得物面经同学 8 一面面试原题：Java 中垃圾回收的原理
5. [Java 面试指南（付费）](#) 收录的快手同学 2 一面面试原题：JVM 了解吗？内存回收机制说一下？
6. [Java 面试指南（付费）](#) 收录的 OPPO 面经同学 1 面试原题：垃圾回收的过程是什么？
7. [Java 面试指南（付费）](#) 收录的 vivo 面经同学 10 技术一面面试原题：说一下 GC，有哪些方法

8. [Java 面试指南 \(付费\)](#) 收录的荣耀面经同学 4 面试原题：对垃圾回收的理解？
9. [Java 面试指南 \(付费\)](#) 收录的字节跳动同学 17 后端技术面试原题：垃圾回收机制 为什么要学jvm 内存泄漏场景
10. [Java 面试指南 \(付费\)](#) 收录的腾讯面经同学 27 云后台技术一面试原题：GC？ 怎么样去识别垃圾？
11. [Java 面试指南 \(付费\)](#) 收录的理想汽车面经同学 2 一面试原题：说说你对GC的了解？
12. [Java 面试指南 \(付费\)](#) 收录的腾讯面经同学 29 Java 后端一面原题：JVM垃圾回收机制？

24. 🌟 如何判断对象仍然存活？

Java 通过可达性分析算法来判断一个对象是否还存活。

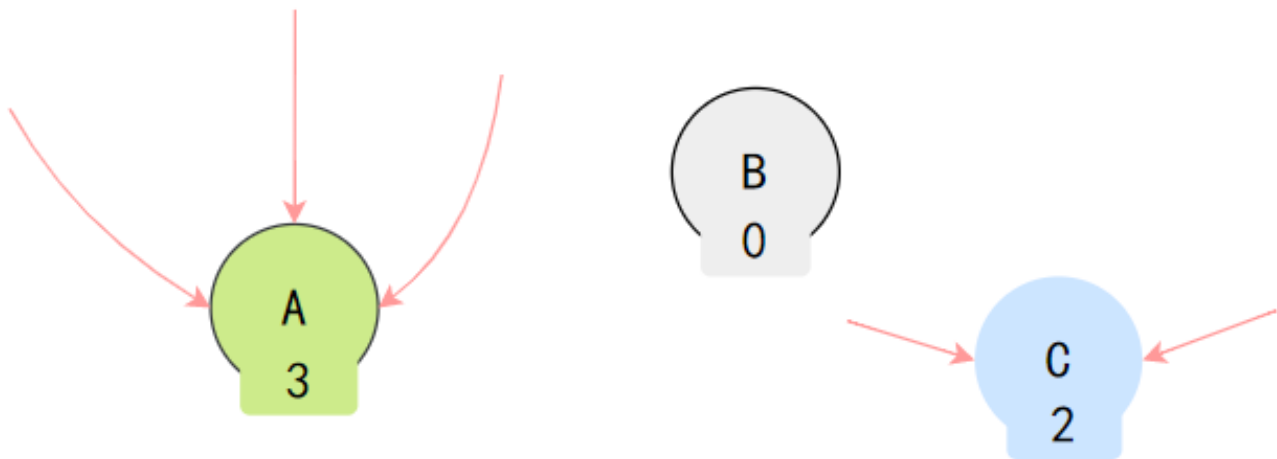
通过一组名为“GC Roots”的根对象，进行递归扫描，无法从根对象到达的对象就是“垃圾”，可以被回收。



这也是 G1、CMS 等主流垃圾收集器使用的主要算法。

什么是引用计数法？

每个对象有一个引用计数器，记录引用它的次数。当计数器为零时，对象可以被回收。



引用计数法无法解决循环引用的问题。例如，两个对象互相引用，但不再被其他对象引用，它们的引用计数都不为零，因此不会被回收。

做可达性分析的时候，应该有哪些前置性的操作？

在进行垃圾回收之前，JVM 会暂停所有正在执行的应用线程。

这是因为可达性分析过程必须确保在执行分析时，内存中的对象关系不会被应用线程修改。如果不暂停应用线程，可能会出现对象引用的改变，导致垃圾回收过程中判断对象是否可达的结果不一致，从而引发严重的内存错误或数据丢失。

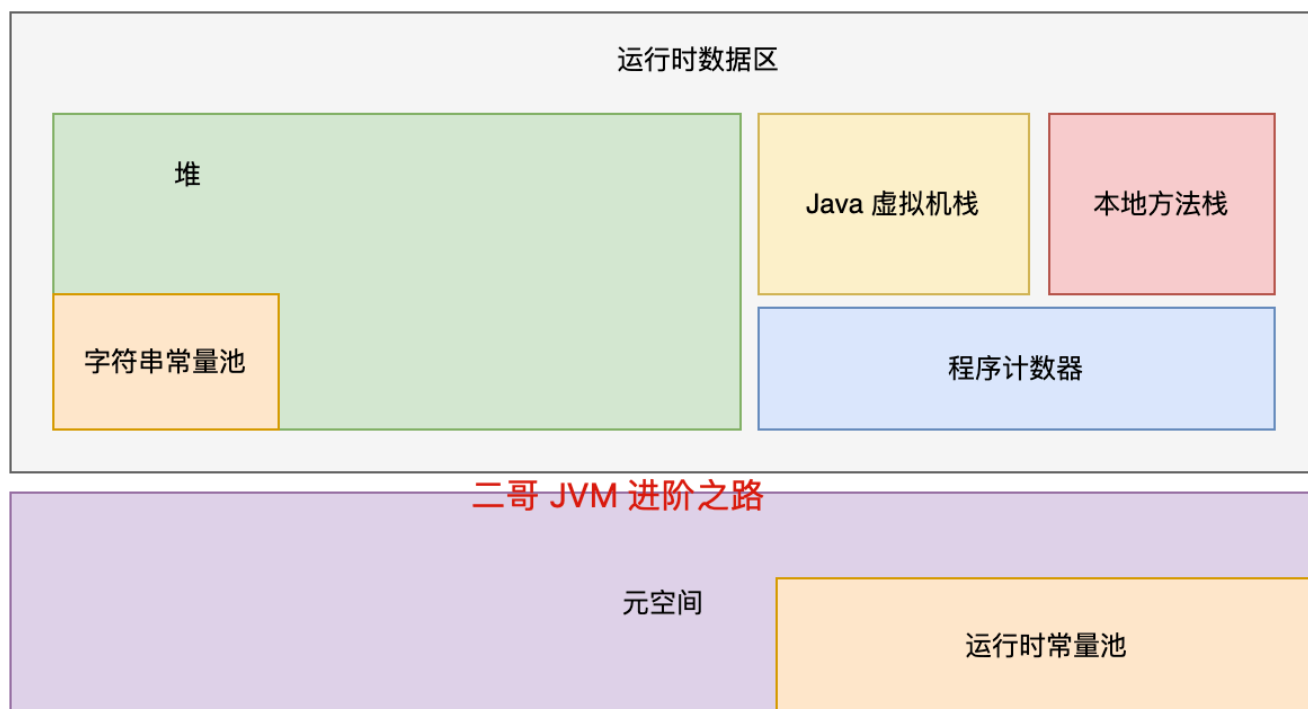
1. [Java 面试指南（付费）](#) 收录的京东面经同学 7 京东到家面试原题：如何判断一个对象是否可以回收
2. [Java 面试指南（付费）](#) 收录的快手同学 2 一面面试原题：做可达性分析的时候，应该有哪些前置性的操作？
3. [Java 面试指南（付费）](#) 收录的京东面经同学 9 面试原题：什么样的对象算作垃圾对象
4. [Java 面试指南（付费）](#) 收录的同学 D 小米一面原题：gc中判断对象可回收的方式有哪些

25.Java 中可作为 GC Roots 的引用有哪几种？

1. 推荐阅读：[深入理解垃圾回收机制](#)
2. 推荐阅读：[R 大的所谓“GC roots”](#)

所谓的 GC Roots，就是一组必须活跃的引用，它们是程序运行时的起点，是一切引用链的源头。在 Java 中，GC Roots 包括以下几种：

- 虚拟机栈中的引用（方法的参数、局部变量等）
- 本地方法栈中 JNI 的引用
- 类静态变量
- 运行时常量池中的常量（String 或 Class 类型）



说说虚拟机栈中的引用？

来看下面这段代码：

```
public class StackReference {
    public void greet() {
        Object localVar = new Object(); // 这里的 localVar 是一个局部变量，存在于虚拟机栈中
        System.out.println(localVar.toString());
    }

    public static void main(String[] args) {
        new StackReference().greet();
    }
}
```

在 greet 方法中，localVar 是一个局部变量，存在于虚拟机栈中，可以被认为是 GC Roots。

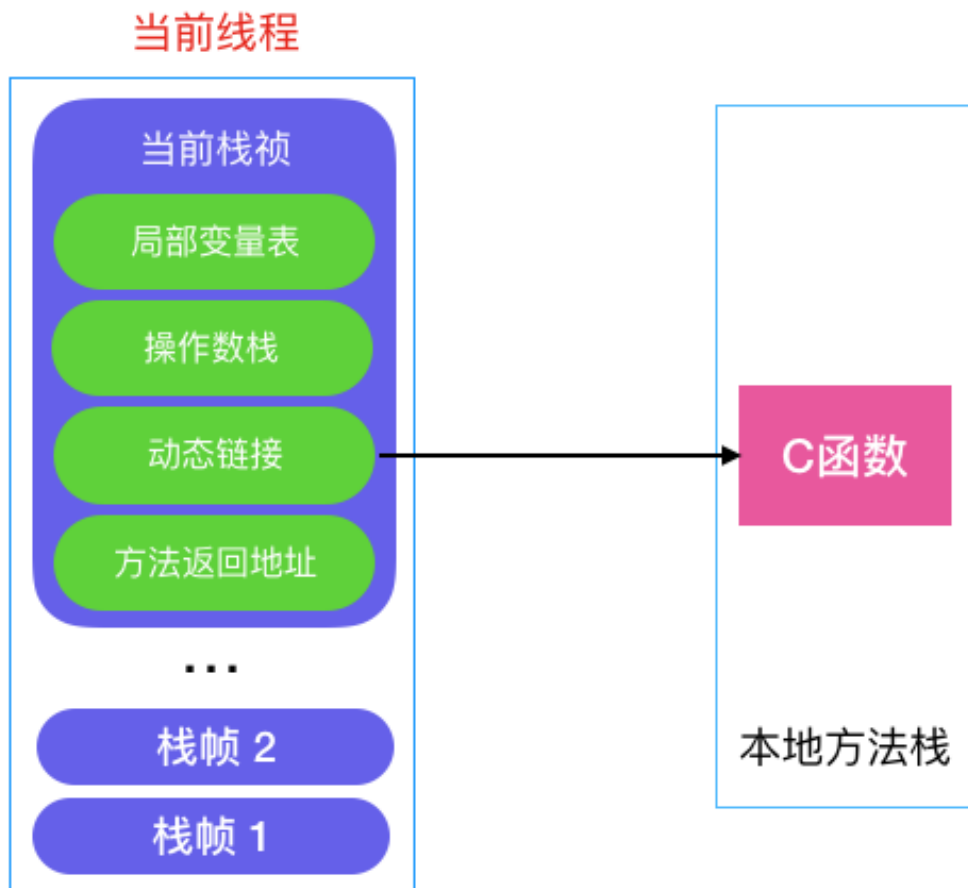
在 greet 方法执行期间，localVar 引用的对象是活跃的，因为它是从 GC Roots 可达的。

当 greet 方法执行完毕后，localVar 的作用域结束，localVar 引用的 Object 对象不再由任何 GC Roots 引用（假设没有其他引用指向这个对象），因此它将有资格作为垃圾被回收掉 😊。

说说本地方法栈中 JNI 的引用？

Java 通过 JNI 提供了一种机制，允许 Java 代码调用本地代码（通常是 C 或 C++ 编写的代码）。

当调用 Java 方法时，虚拟机会创建一个栈帧并压入虚拟机栈，而当它调用本地方法时，虚拟机会通过动态链接直接调用指定的本地方法。



JNI 引用是在 Java 本地接口代码中创建的引用，这些引用可以指向 Java 堆中的对象。

```
// 假设的JNI方法
public native void nativeMethod();

// 假设在C/C++中实现的本地方法
/*
 * Class:      NativeExample
 * Method:     nativeMethod
 * Signature: ()V
 */
JNIEXPORT void JNICALL Java_NativeExample_nativeMethod(JNIEnv *env, jobject thisObj) {
    jobject localRef = (*env)->NewObject(env, ...); // 在本地方法栈中创建JNI引用
    // localRef 引用的Java对象在本地方法执行期间是活跃的
}
```

在本地代码中，localRef 是对 Java 对象的一个 JNI 引用，它在本地方法执行期间保持 Java 对象活跃，可以被认为是 GC Roots。

一旦 JNI 方法执行完毕，除非这个引用是全局的，否则它指向的对象将会被作为垃圾回收掉（假设没有其他地方再引用这个对象）。

说说类静态变量？

来看下面这段代码：

```
public class StaticFieldReference {
    private static Object staticVar = new Object(); // 类静态变量

    public static void main(String[] args) {
        System.out.println(staticVar.toString());
    }
}
```

StaticFieldReference 类中的 staticVar 引用了一个 Object 对象，这个引用存储在元空间，可以被认为是 GC Roots。

只要 StaticFieldReference 类未被卸载，staticVar 引用的对象都不会被垃圾回收。如果 StaticFieldReference 类被卸载（这通常发生在其类加载器被垃圾回收时），那么 staticVar 引用的对象也将有资格被垃圾回收（如果没有其他引用指向这个对象）。

说说运行时常量池中的常量？

来看这段代码：

```
class ConstantPoolReference {
    public static final String CONSTANT_STRING = "Hello, World"; // 常量，存在于运行时常量池中
    public static final Class<?> CONSTANT_CLASS = Object.class; // 类类型常量

    public static void main(String[] args) {
        System.out.println(CONSTANT_STRING);
        System.out.println(CONSTANT_CLASS.getName());
    }
}
```

在 ConstantPoolReference 中，CONSTANT_STRING 和 CONSTANT_CLASS 作为常量存储在运行时常量池。它们可以用来作为 GC Roots。

这些常量引用的对象（字符串"Hello, World"和 Object.class 类对象）在常量池中，只要包含这些常量的 ConstantPoolReference 类未被卸载，这些对象就不会被垃圾回收。

1. [Java 面试指南（付费）](#) 收录的帆软同学 3 Java 后端一面的原题：哪些对象可以作为 GC Roots
2. [Java 面试指南（付费）](#) 收录的腾讯面经同学 27 云后台技术一面面试原题：GC Root?
3. [Java 面试指南（付费）](#) 收录的同学 D 小米一面原题：那些对象可以作为gc root

26.finalize()方法了解吗？

垃圾回收就是古代的秋后问斩，`finalize()` 就是刀下留人，在人犯被处决之前，还要做最后一次审计，青天大老爷会看看有没有什么冤情，需不需要刀下留人。

垃圾收集



finalize



如果对象在进行可达性分析后发现没有与 GC Roots 相连接的引用链，那它将会被第一次标记，随后进行一次筛选。

筛选的条件是对象是否有必要执行 `finalize()` 方法。

如果对象在 `finalize()` 中成功拯救自己——只要重新与引用链上的任何一个对象建立关联即可。

譬如把自己（`this` 关键字）赋值给某个类变量或者对象的成员变量，那在第二次标记时它就“逃过一劫”；但是如果没抓住这个机会，那么对象就真的要被回收了。

27. 🌟 垃圾收集算法了解吗？

垃圾收集算法主要有三种，分别是标记-清除算法、标记-复制算法和标记-整理算法。

说说标记-清除算法？

标记-清除 算法分为两个阶段：

- **标记：** 标记所有需要回收的对象
- **清除：** 回收所有被标记的对象

标记-清除算法



优点是实现简单，缺点是回收过程中会产生内存碎片。

说说标记-复制算法？

标记-复制 算法可以解决标记-清除算法的内存碎片问题，因为它将内存空间划分为两块，每次只使用其中一块。当这一块内存用完了，就将还存活着的对象复制到另外一块上面，然后清理掉这一块。

标记-复制算法



缺点是浪费了一半的内存空间。

说说标记-整理算法？

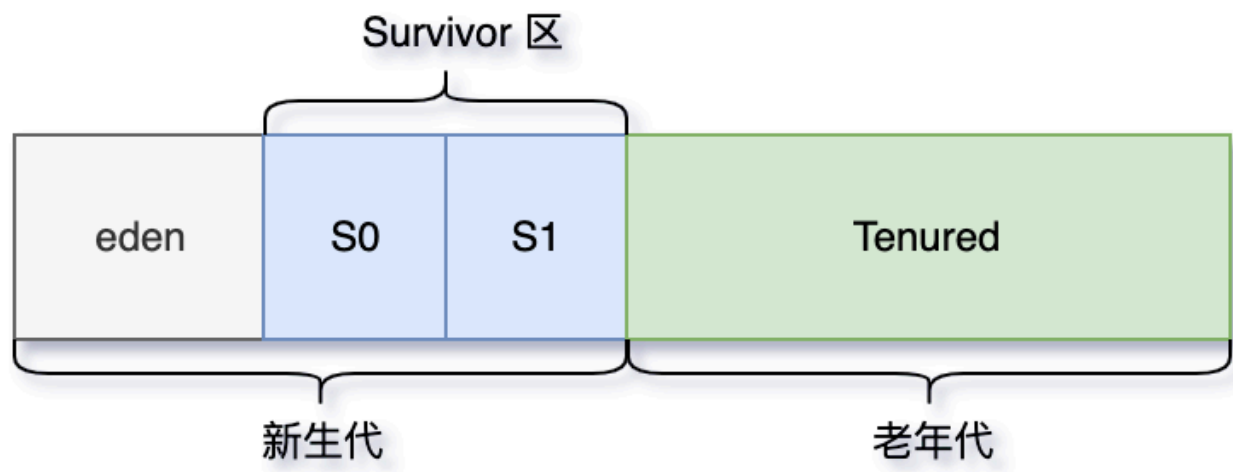
标记-整理 算法是标记-清除复制算法的升级版，它不再划分内存空间，而是将存活的对象向内存的一端移动，然后清理边界以外的内存。



缺点是移动对象的成本比较高。

说说分代收集算法？

分代收集 算法是目前主流的垃圾收集算法，它根据对象存活周期的不同将内存划分为几块，一般分为新生代和老年代。



新生代用复制算法，因为大部分对象生命周期短。老年代用标记-整理算法，因为对象存活率较高。

为什么要用分代收集呢？

分代收集算法的核心思想是根据对象的生命周期优化垃圾回收。

新生代的对象生命周期短，使用复制算法可以快速回收。老年代的对象生命周期长，使用标记-整理算法可以减少移动对象的成本。

标记复制的标记过程和复制过程会不会停顿？

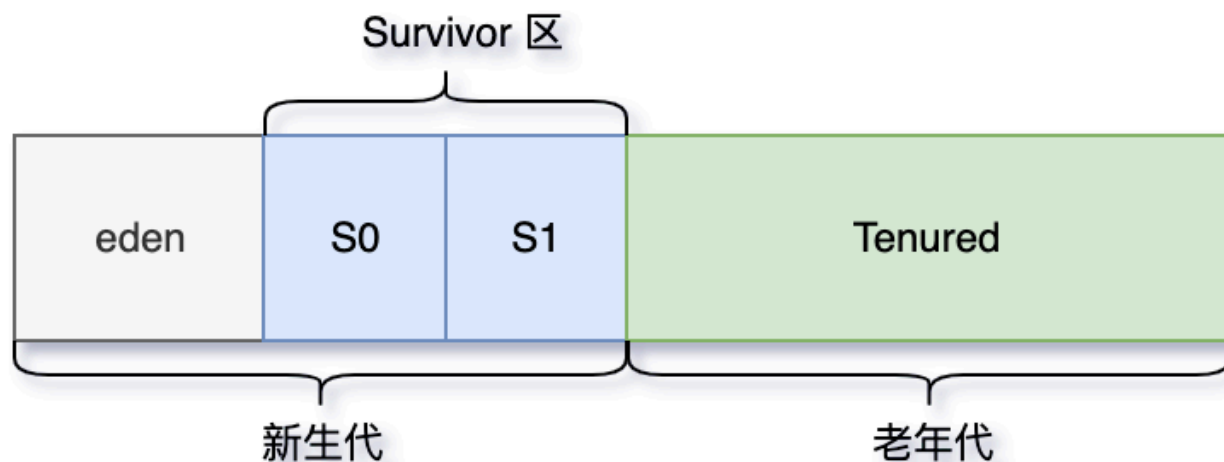
在标记-复制算法中，标记阶段和复制阶段都会触发STW。

- 标记阶段停顿是为了保证对象的引用关系不被修改。
- 复制阶段停顿是防止对象在复制过程中被修改。

1. [Java 面试指南（付费）](#) 收录的字节跳动面经同学 1 Java 后端技术一面面试原题：垃圾回收算法了解多少？
2. [Java 面试指南（付费）](#) 收录的小米面经同学 F 面试原题：垃圾回收的算法及详细介绍
3. [Java 面试指南（付费）](#) 收录的腾讯面经同学 27 云后台技术一面面试原题：回收的方法？分代收集算法里面具体是怎么回收的？为什么要用分代收集呢？
4. [Java 面试指南（付费）](#) 收录的百度同学 4 面试原题：Gc 算法有哪些？
5. [Java 面试指南（付费）](#) 收录的京东面经同学 9 面试原题：问了垃圾回收算法，针对问了每个算法的优缺点
6. [Java 面试指南（付费）](#) 收录的同学 D 小米一面原题：gc垃圾回收算法有哪些

28.Minor GC、Major GC、Mixed GC、Full GC 都是什么意思？

Minor GC 也称为 Young GC，是指发生在年轻代的垃圾收集。年轻代包含 Eden 区以及两个 Survivor 区。



Major GC 也称为 Old GC，主要指的是发生在老年代的垃圾收集。是 CMS 的特有行为。

Mixed GC 是 G1 垃圾收集器特有的一种 GC 类型，它在一次 GC 中同时清理年轻代和部分老年代。

Full GC 是最彻底的垃圾收集，涉及整个 Java 堆和方法区。它是最耗时的 GC，通常在 JVM 压力很大时发生。

FULL gc怎么去清理的？

Full GC 会从 GC Root 出发，标记所有可达对象。新生代使用复制算法，清空 Eden 区。老年代使用标记-整理算法，回收对象并消除碎片。

停顿时间较长，会影响系统响应性能。

1. [Java 面试指南 \(付费\)](#) 收录的阿里面经同学 5 阿里妈妈 Java 后端技术一面面试原题：full gc 和 young gc 的区别
2. [Java 面试指南 \(付费\)](#) 收录的腾讯面经同学 27 云后台技术一面面试原题：FULL gc怎么去清理的？

29.Young GC 什么时候触发？

如果 Eden 区没有足够的空间时，就会触发 Young GC 来清理新生代。

1. [Java 面试指南 \(付费\)](#) 收录的百度同学 4 面试原题：什么时候会触发 GC？

30.什么时候会触发 Full GC？

在进行 Young GC 的时候，如果发现 老年代可用的连续内存空间 < 新生代历次 Young GC 后升入老年代的对象总和的平均大小，说明本次 Young GC 后升入老年代的对象大小，可能超过了老年代当前可用的内存空间，就会触发 Full GC。

执行 Young GC 后老年代没有足够的内存空间存放转入的对象，会立即触发一次 Full GC。

`System.gc()`、`jmap -dump` 等命令会触发 full gc。

空间分配担保是什么？

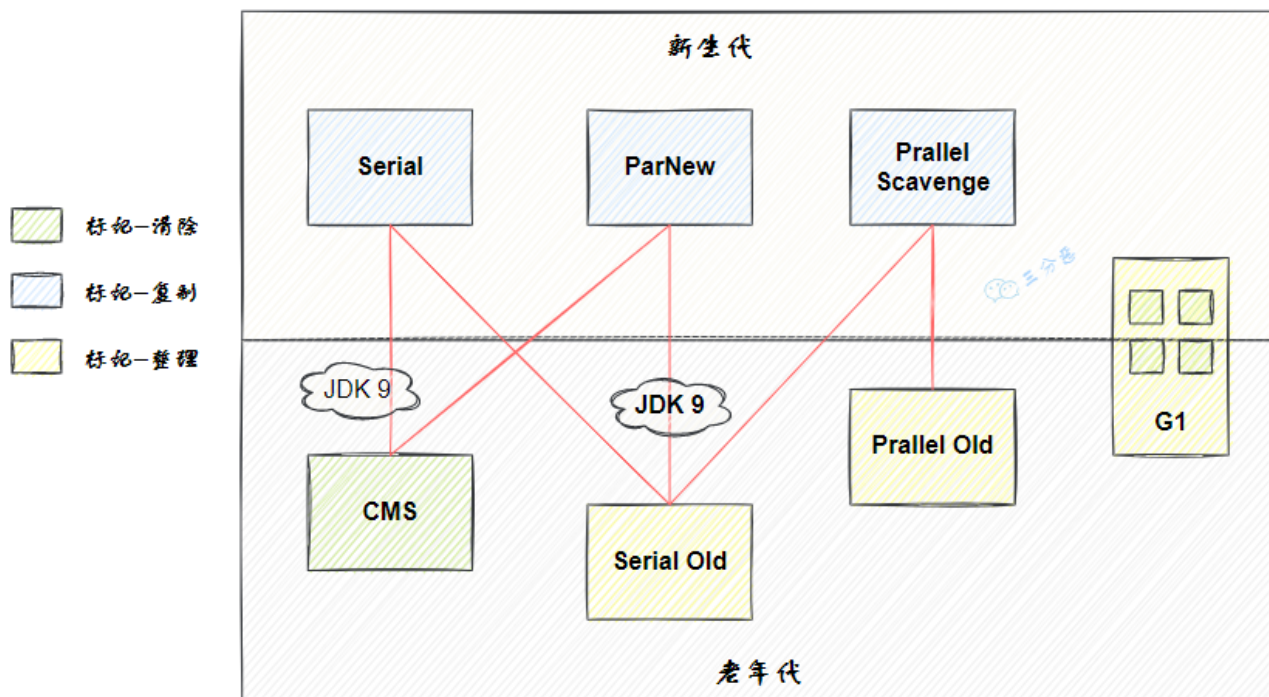
空间分配担保是指在进行 Minor GC 前，JVM 会确保老年代有足够的空间存放从新生代晋升的对象。如果老年代空间不足，可能会触发 Full GC。

1. [Java 面试指南 \(付费\)](#) 收录的快手同学 4 一面原题：如何判断死亡对象？GC Roots有哪些？空间分配担保是什么？

31.🌟知道哪些垃圾收集器？

推荐阅读：[深入理解 JVM 的垃圾收集器：CMS、G1、ZGC](#)

JVM 的垃圾收集器主要分为两大类：分代收集器和分区收集器，分代收集器的代表是 CMS，分区收集器的代表是 G1 和 ZGC。



CMS 是第一个关注 GC 停顿时间的垃圾收集器，JDK 1.5 时引入，JDK9 被标记弃用，JDK14 被移除。

G1 在 JDK 1.7 时引入，在 JDK 9 时取代 CMS 成为了默认的垃圾收集器。

ZGC 是 JDK11 推出的一款低延迟垃圾收集器，适用于大内存低延迟服务的内存管理和回收，在 128G 的大堆下，最大停顿时间才 1.68 ms，性能远胜于 G1 和 CMS。

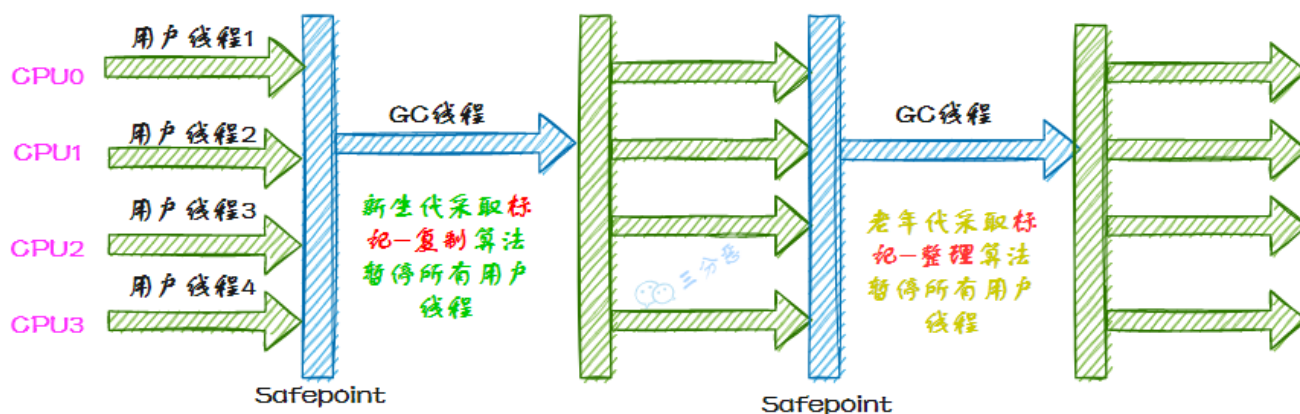
说说 Serial 收集器？

Serial 收集器是最基础、历史最悠久的收集器。

如同它的名字（串行），它是一个单线程工作的收集器，使用一个处理器或一条收集线程去完成垃圾收集工作。并且进行垃圾收集时，必须暂停其他所有工作线程，直到垃圾收集结束——这就是所谓的“Stop The World”。

Serial/Serial Old 收集器的运行过程如图：

Serial/Serial Old 收集器运行示意图

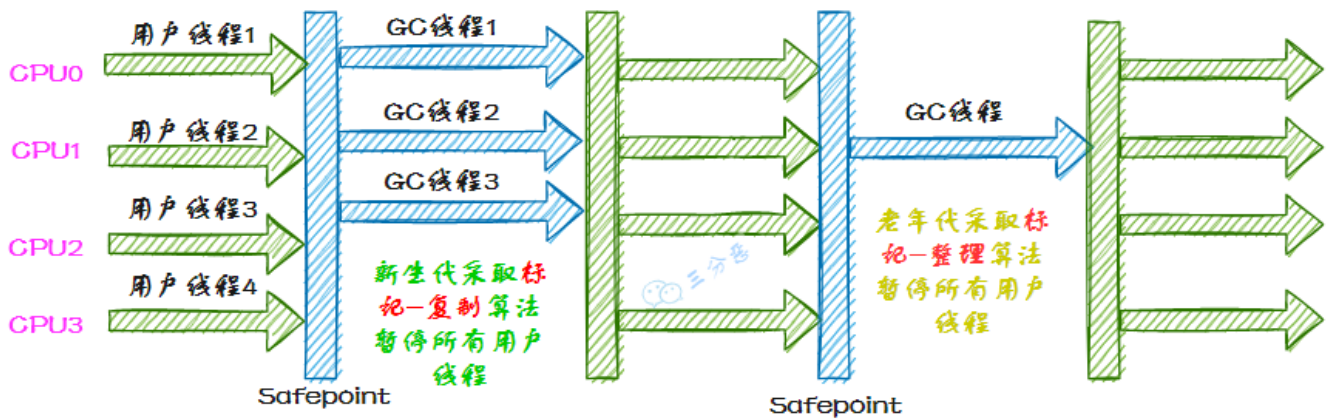


说说 ParNew 收集器?

ParNew 收集器实质上是 Serial 收集器的多线程并行版本，使用多条线程进行垃圾收集。

ParNew/Serial Old 收集器运行示意图如下：

ParNew/Serial Old 收集器运行示意图



说说 Parallel Scavenge 收集器?

Parallel Scavenge 收集器是一款新生代收集器，基于标记-复制算法实现，也能够并行收集。和 ParNew 有些类似，但 Parallel Scavenge 主要关注的是垃圾收集的吞吐量——所谓吞吐量，就是 CPU 用于运行用户代码的时间和总消耗时间的比值，比值越大，说明垃圾收集的占比越小。

$$\text{吞吐量} = \frac{\text{运行用户代码的时间}}{\text{运行垃圾收集时间} + \text{运行用户代码的时间}}$$

根据对象存活周期的不同会将内存划分为几块，一般是把 Java 堆分为新生代和老年代，这样就可以根据各个年代的特点采用最适当的收集算法。

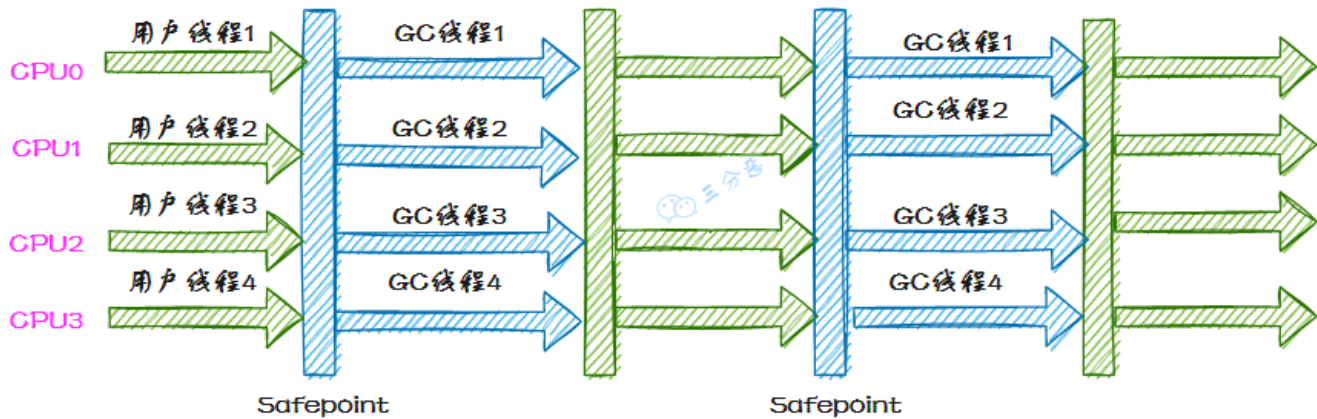
说说 Serial Old 收集器?

Serial Old 是 Serial 收集器的老年代版本，它同样是一个单线程收集器，使用标记-整理算法。

说说 Parallel Old 收集器?

Parallel Old 是 Parallel Scavenge 收集器的老年代版本，基于标记-整理算法实现，使用多条 GC 线程在 STW 期间同时进行垃圾回收。

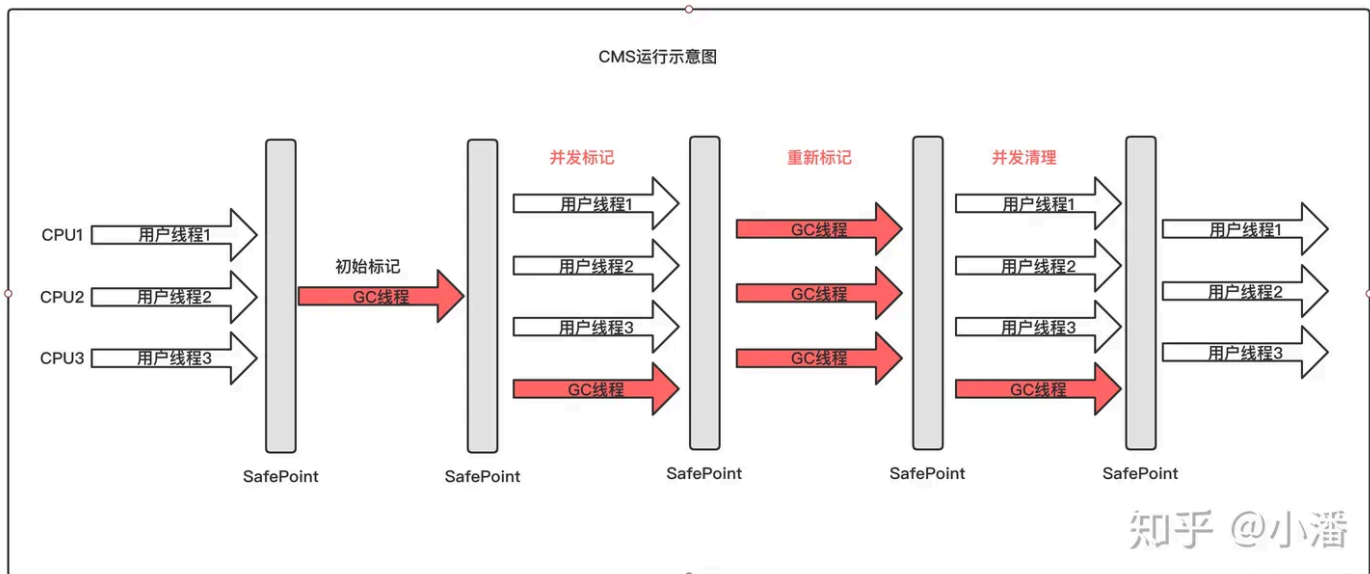
Parallel Scavenge/Parallel Old收集器运行示意图



说说 CMS 收集器？

CMS 在 JDK 1.5 时引入，JDK 9 时被标记弃用，JDK 14 时被移除。

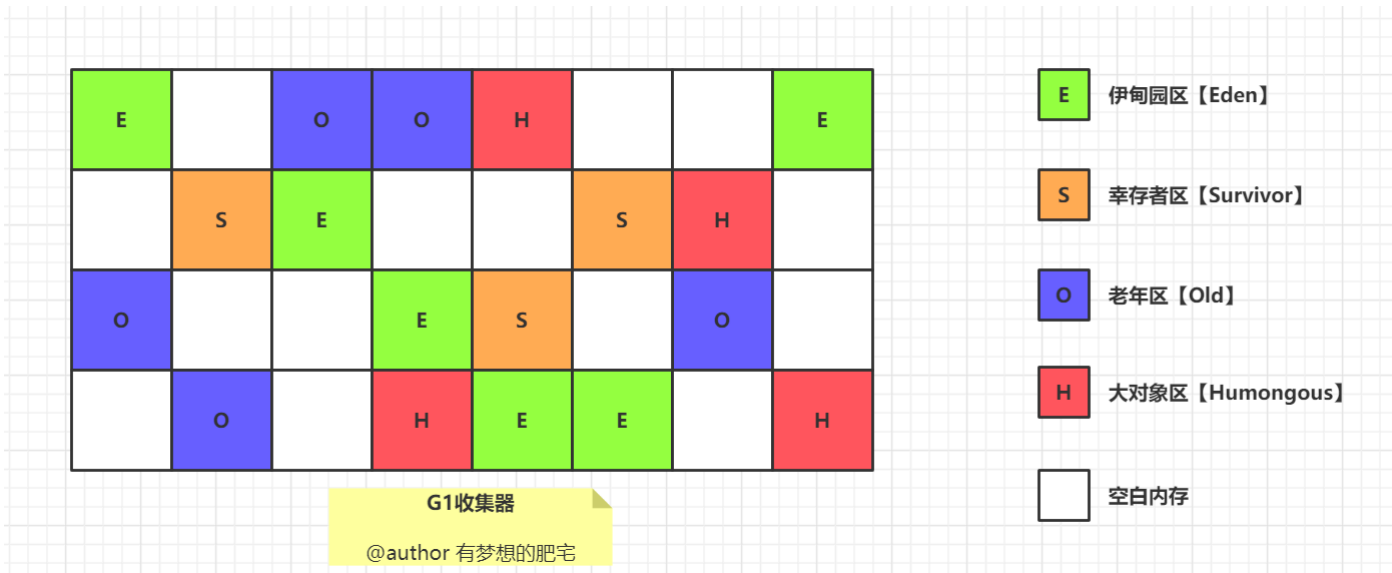
CMS 是一种低延迟的垃圾收集器，采用标记-清除算法，分为初始标记、并发标记、重新标记和并发清除四个阶段，优点是垃圾回收线程和应用线程同时运行，停顿时间短，适合延迟敏感的应用，但容易产生内存碎片，可能触发 Full GC。



说说 G1 收集器？

G1 在 JDK 1.7 时引入，在 JDK 9 时取代 CMS 成为默认的垃圾收集器。

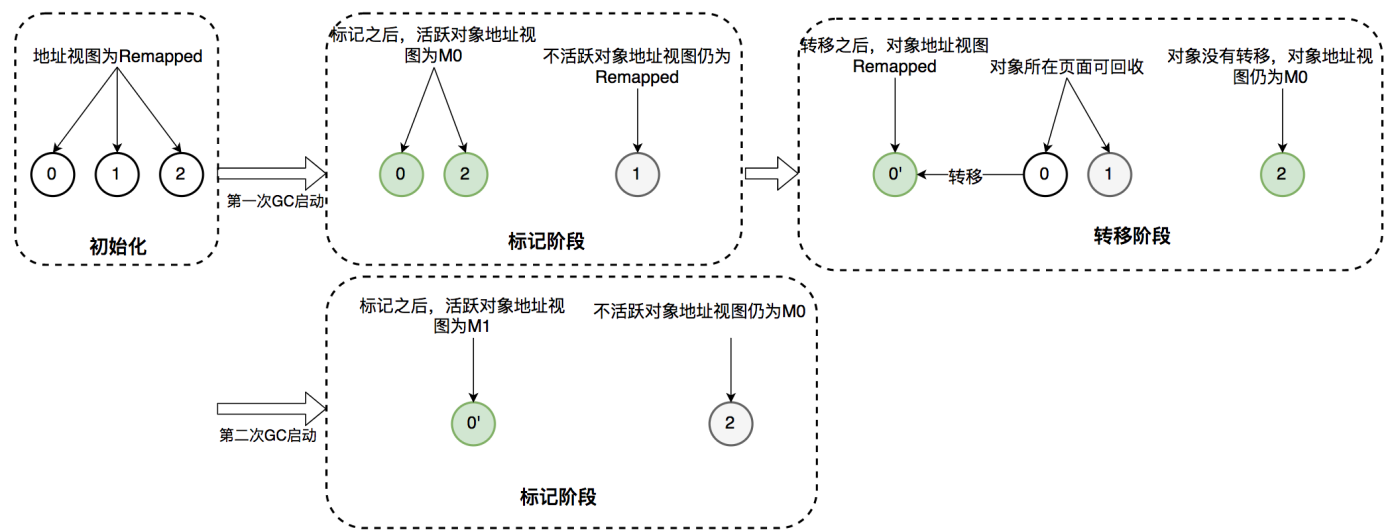
G1 是一种面向大内存、高吞吐场景的垃圾收集器，它将堆划分为多个小的 Region，通过标记-整理算法，避免了内存碎片问题。优点是停顿时间可控，适合大堆场景，但调优较复杂。



说说 ZGC 收集器？

ZGC 是 JDK 11 时引入的一款低延迟的垃圾收集器，最大特点是将垃圾收集的停顿时间控制在 10ms 以内，即使在 TB 级别的堆内存下也能保持较低的停顿时间。

它通过并发标记和重定位来避免大部分 Stop-The-World 停顿，主要依赖指针染色来管理对象状态。



- **标记对象的可达性：**通过在指针上增加标记位，不需要额外的标记位即可判断对象的存活状态。
- **重定位状态：**在对象被移动时，可以通过指针染色来更新对象的引用，而不需要等待全局同步。

适用于需要超低延迟的场景，比如金融交易系统、电商平台。

垃圾回收器的作用是什么？

垃圾回收器的核心作用是自动管理 Java 应用程序的运行时内存。它负责识别哪些内存是不再被应用程序使用的，并释放这些内存以便重新使用。

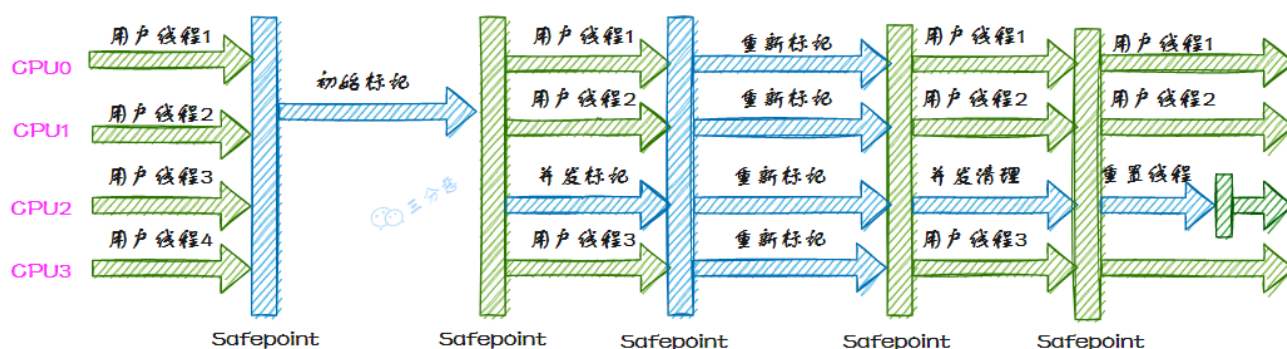
这一过程减少了程序员手动管理内存的负担，降低了内存泄漏和溢出错误的风险。

1. [Java 面试指南（付费）](#) 收录的滴滴同学 2 技术二面的原题：了解哪些垃圾回收器，只能回收一个代（新生代、老年代）吗，使用的 jdk 版本
2. [Java 面试指南（付费）](#) 收录的京东同学 10 后端实习一面的原题：垃圾回收器的作用是什么

3. [Java 面试指南 \(付费\)](#) 收录的携程面经同学 10 Java 暑期实习一面面试原题：有哪些垃圾回收器，选一个讲一下垃圾回收的流程
4. [Java 面试指南 \(付费\)](#) 收录的京东同学 4 云实习面试原题：常见的 7 个 GC 回收器
5. [Java 面试指南 \(付费\)](#) 收录的美团面经同学 15 点评后端技术面试原题：讲一下知道的垃圾回收器，问知不知道ZGC回收器（不知道）
6. [Java 面试指南 \(付费\)](#) 收录的阿里云面经同学 22 面经：cms和g1的区别
7. [Java 面试指南 \(付费\)](#) 收录的京东面经同学 9 面试原题：怎么理解并发和并行，Parallel Old和CMS有什么区别？

32. 🌟能详细说一下 CMS 的垃圾收集过程吗？

CMS收集器运行示意图



CMS 使用标记-清除算法进行垃圾收集，分 4 大步：

- **初始标记：**标记所有从 GC Roots 直接可达的对象，这个阶段需要 STW，但速度很快。
- **并发标记：**从初始标记的对象出发，遍历所有对象，标记所有可达的对象。这个阶段是并发进行的。
- **重新标记：**完成剩余的标记工作，包括处理并发阶段遗留下来的少量变动，这个阶段通常需要短暂的 STW 停顿。
- **并发清除：**清除未被标记的对象，回收它们占用的内存空间。

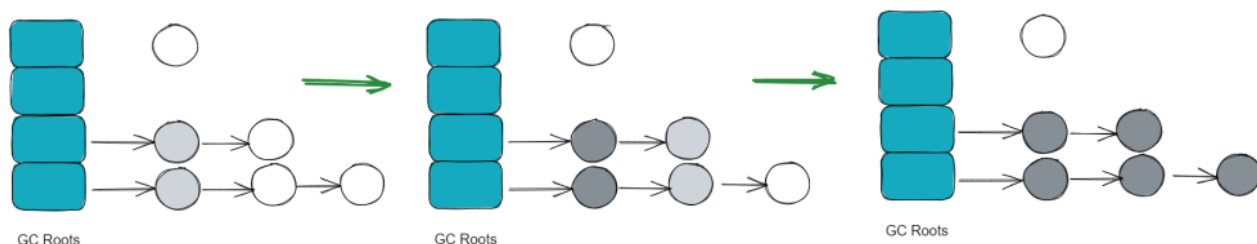
你提到了remark，那它remark具体是怎么执行的？三色标记法？

是的，remark 阶段通常会结合三色标记法来执行，确保在并发标记期间所有存活对象都被正确标记。目的是修正并发标记阶段中可能遗漏的对象引用变化。

在 remark 阶段，垃圾收集器会停止应用线程，以确保在这个阶段不会有引用关系的进一步变化。这种暂停通常很短暂。remark 阶段主要包括以下操作：

1. 处理写屏障记录的引用变化：在并发标记阶段，应用程序可能会更新对象的引用（比如一个黑色对象新增了对一个白色对象的引用），这些变化通过写屏障记录下来。在 remark 阶段，GC 会处理这些记录，确保所有可达对象都正确地标记为灰色或黑色。
2. 扫描灰色对象：再次遍历灰色对象，处理它们的所有引用，确保引用的对象正确标记为灰色或黑色。
3. 清理：确保所有引用关系正确处理，灰色对象标记为黑色，白色对象保持不变。这一步完成后，所有存活对象都应当是黑色的。

什么是三色标记法？



三色标记法用于标记对象的存活状态，它将对象分为三类：

1. 白色（White）：尚未访问的对象。垃圾回收结束后，仍然为白色的对象会被认为是不可达的对象，可以回收。
2. 灰色（Gray）：已经访问到但未标记完其引用的对象。灰色对象是需要进一步处理的。
3. 黑色（Black）：已经访问到并且其所有引用对象都已经标记过。黑色对象是完全处理过的，不需要再处理。

三色标记法的工作流程：

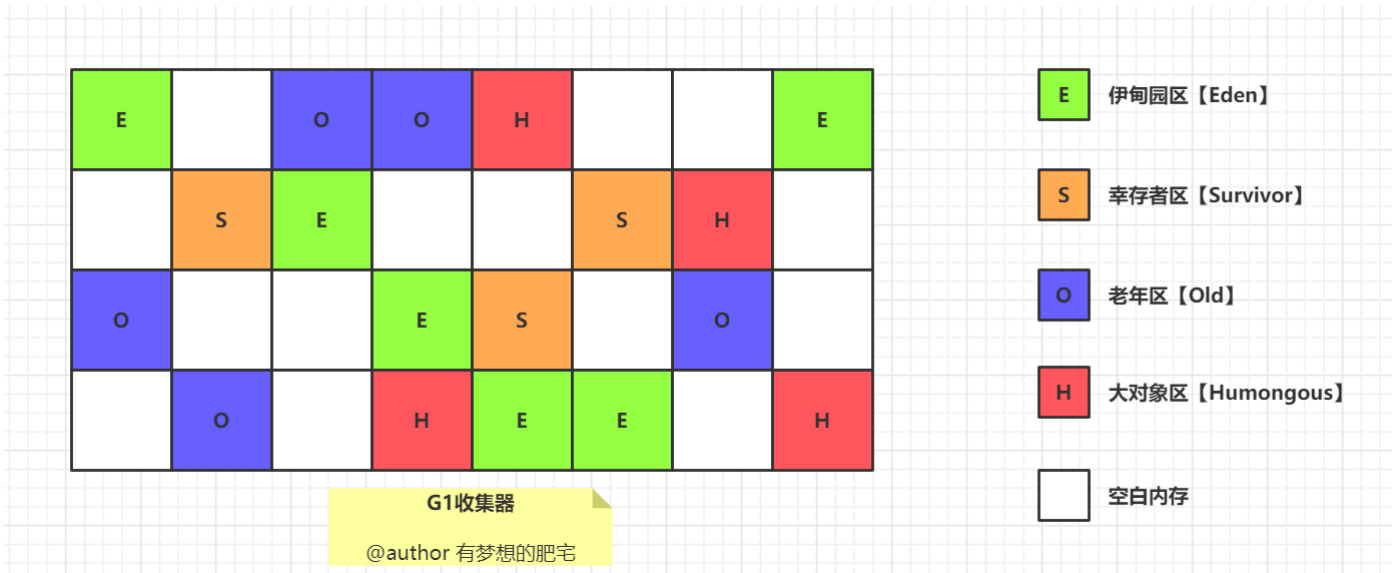
- ①、初始标记（Initial Marking）：从 GC Roots 开始，标记所有直接可达的对象为灰色。
 - ②、并发标记（Concurrent Marking）：在此阶段，标记所有灰色对象引用的对象为灰色，然后将灰色对象自身标记为黑色。这个过程是并发的，和应用线程同时进行。
- 此阶段的一个问题是，应用线程可能在并发标记期间修改对象的引用关系，导致一些对象的标记状态不准确。
- ③、重新标记（Remarking）：重新标记阶段的目标是处理并发标记阶段遗漏的引用变化。为了确保所有存活对象都被正确标记，remark 需要在 STW 暂停期间执行。
 - ④、使用写屏障（Write Barrier）来捕捉并发标记阶段应用线程对对象引用的更新。通过遍历这些更新的引用来修正标记状态，确保遗漏的对象不会被错误地回收。

推荐阅读：[小道哥的三色标记](#)

1. [Java 面试指南（付费）](#) 收录的携程面经同学 10 Java 暑期实习一面面试原题：有哪些垃圾回收器，选一个讲一下垃圾回收的流程
2. [Java 面试指南（付费）](#) 收录的携程面经同学 1 Java 后端技术一面面试原题：对象创建到销毁，内存如何分配的，（类加载和对象创建过程，CMS，G1 内存清理和分配）
3. [Java 面试指南（付费）](#) 收录的收钱吧面经同学 1 Java 后端一面面试原题：CMS用了什么垃圾回收算法？你提到了remark，那它remark具体是怎么执行的？三色标记法？
4. [Java 面试指南（付费）](#) 收录的京东面经同学 9 面试原题：问了CMS垃圾回收器

33. 🌟 G1 垃圾收集器了解吗？

G1 在 JDK 1.7 时引入，在 JDK 9 时取代 CMS 成为默认的垃圾收集器。



G1 把 Java 堆划分为多个大小相等的独立区域Region，每个区域都可以扮演新生代或老年代的角色。

同时，G1 还有一个专门为大对象设计的 Region，叫 Humongous 区。

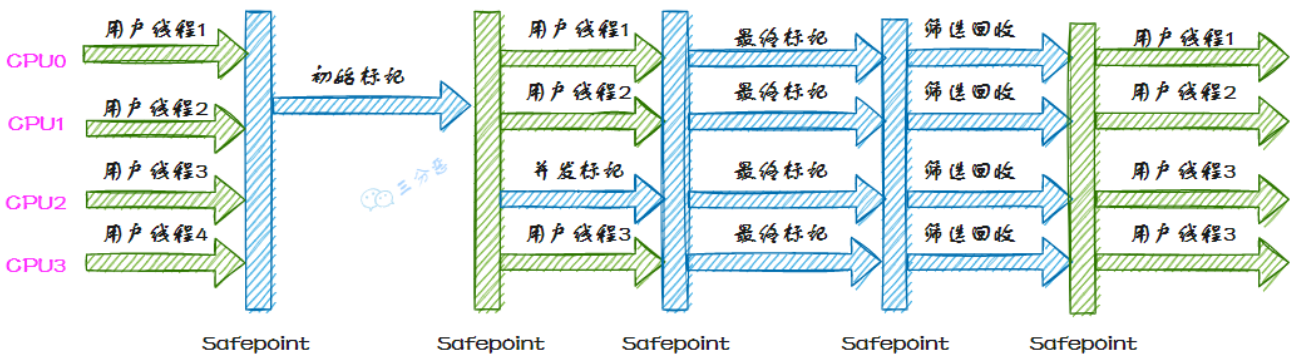
大对象的判定规则是，如果一个大对象超过了一个 Region 大小的 50%，比如每个 Region 是 2M，只要一个对象超过了 1M，就会被放入 Humongous 中。

这种区域化管理使得 G1 可以更灵活地进行垃圾收集，只回收部分区域而不是整个新生代或老年代。

G1 收集器的运行过程大致可划分为这几个步骤：

- ①、**并发标记**，G1 通过并发标记的方式找出堆中的垃圾对象。并发标记阶段与应用线程同时执行，不会导致应用线程暂停。
 - ②、**混合收集**，在并发标记完成后，G1 会计算出哪些区域的回收价值最高（也就是包含最多垃圾的区域），然后优先回收这些区域。这种回收方式包括了部分新生代区域和老年代区域。
- 选择回收成本低而收益高的区域进行回收，可以提高回收效率和减少停顿时间。
- ③、**可预测的停顿**，G1 在垃圾回收期间仍然需要「Stop the World」。不过，G1 在停顿时间上添加了预测机制，用户可以 JVM 启动时指定期望停顿时间，G1 会尽可能地在這個时间内完成垃圾回收。

G1收集器运行示意图



1. [Java 面试指南（付费）](#) 收录的京东面经同学 1 Java 技术一面面试原题：说说 G1 垃圾回收器的原理

2. [Java 面试指南 \(付费\)](#) 收录的携程面经同学 1 Java 后端技术一面面试原题：对象创建到销毁，内存如何分配的，（类加载和对象创建过程，CMS，G1 内存清理和分配）
3. [Java 面试指南 \(付费\)](#) 收录的百度同学 4 面试原题：G1 垃圾回收器了解吗？
4. [Java 面试指南 \(付费\)](#) 收录的理想汽车面经同学 2 一面面试原题：了解过G1垃圾回收器吗？

34.有了 CMS，为什么还要引入 G1？

特性	CMS	G1
设计目标	低停顿时间	可预测的停顿时间
并发性	是	是
内存碎片	是，容易产生碎片	否，通过区域划分和压缩减少碎片
收集代数	年轻代和老年代	整个堆，但区分年轻代和老年代
并发阶段	并发标记、并发清理	并发标记、并发清理、并发回收
停顿时间预测	较难预测	可配置停顿时间目标
容易出现的问题	内存碎片、Concurrent Mode Failure	较少出现长时间停顿

CMS 适用于对延迟敏感的应用场景，主要目标是减少停顿时间，但容易产生内存碎片。

G1 则提供了更好的停顿时间预测和内存压缩能力，适用于大内存和多核处理器环境。

1. [Java 面试指南 \(付费\)](#) 收录的快手面经同学 5 面试原题：CMS 垃圾收集器和 G1 垃圾收集器什么区别

35.你们线上用的什么垃圾收集器？

我们生产环境中采用了设计比较优秀的 G1 垃圾收集器，因为它不仅能满足低停顿的要求，而且解决了 CMS 的浮动垃圾问题、内存碎片问题。

G1 非常适合大内存、多核处理器的环境。

以上是比较符合面试官预期的回答，但实际上，大多数情况下我们可能还是使用的 JDK 8 默认垃圾收集器。

可以通过以下命令查看当前 JVM 的垃圾收集器：

```
java -XX:+PrintCommandLineFlags -version
```

```
~ (0.3s)
java -XX:+PrintCommandLineFlags -version
-XX:InitialHeapSize=268435456 -XX:MaxHeapSize=4294967296 -XX:+PrintCommandLineFlags -XX:+UseCompressedClassPointers -XX:+UseCompressedOops -XX:+UseParallelGC
java version "1.8.0_301"
Java(TM) SE Runtime Environment (build 1.8.0_301-b09)
Java HotSpot(TM) 64-Bit Server VM (build 25.301-b09, mixed mode)
```

`UseParallelGC = Parallel Scavenge + Parallel Old`，表示新生代用 `Parallel Scavenge` 收集器，老年代使用 `Parallel Old` 收集器。

因此你也可以这样回答：

我们系统的业务相对复杂，但并发量并不是特别高，所以我们选择了适用于多核处理器、能够并行处理垃圾回收任务，且能提供高吞吐量的 `Parallel GC`。

但这个说法不讨喜，你也可以回答：

我们系统采用的是 CMS 收集器，能够最大限度减少应用暂停时间。

工作中项目使用的什么垃圾回收算法？

我们生产环境中采用了设计比较优秀的 G1 垃圾收集器，G1 采用的是分区式标记-整理算法，将堆划分为多个区域，按需回收，适用于大内存和多核环境，能够同时考虑吞吐量和暂停时间。

或者：

我们系统采用的是 CMS 收集器，CMS 采用的是标记-清除算法，能够并发标记和清除垃圾，减少暂停时间，适用于对延迟敏感的应用。

又或者：

我们系统采用的是 Parallel 收集器，Parallel 采用的是年轻代使用复制算法，老年代使用标记-整理算法，适用于高吞吐量要求的应用。

1. [Java 面试指南（付费）](#) 收录的华为 OD 面经同学 3 技术二面面试原题：工作中项目使用的什么垃圾回收算法

36.垃圾收集器应该如何选择？

如果应用程序只需要一个很小的内存空间（大约 100 MB），或者对停顿时间没有特殊的要求，可以选择 Serial 收集器。

如果优先考虑应用程序的峰值性能，并且没有时间要求，或者可以接受 1 秒或更长的停顿时间，可以选择 Parallel 收集器。

如果响应时间比吞吐量优先级高，或者垃圾收集暂停必须保持在大约 1 秒以内，可以选择 CMS/ G1 收集器。

如果响应时间是高优先级的，或者堆空间比较大，可以选择 ZGC 收集器。

memo：2025 年 1 月 16 日修改至此。

四、JVM 调优

37.用过哪些性能监控的命令行工具？

操作系统层面，我用过 `top`、`vmstat`、`iostat`、`netstat` 等命令，可以监控系统整体的资源使用情况，比如说内存、CPU、IO 使用情况、网络使用情况。

JDK 自带的命令行工具层面，我用过 `jps`、`jstat`、`jinfo`、`jmap`、`jhat`、`jstack`、`jcmd` 等，可以查看 JVM 运行时信息、内存使用情况、堆栈信息等。

你一般都怎么用jmap？

①、我一般会使用 `jmap -heap <pid>` 查看堆内存摘要，包括新生代、老年代、元空间等。

```
^C[root@VM-0-5-tencentos logs]# jmap -heap 391195
Attaching to process ID 391195, please wait...
Debugger attached successfully.
Server compiler detected.
JVM version is 25.382-b05
```

```
using thread-local object allocation.
Parallel GC with 2 thread(s)
```

Heap Configuration:

MinHeapFreeRatio	= 0
MaxHeapFreeRatio	= 100
MaxHeapSize	= 1610612736 (1536.0MB)
NewSize	= 268435456 (256.0MB)
MaxNewSize	= 268435456 (256.0MB)
OldSize	= 1342177280 (1280.0MB)
NewRatio	= 2
SurvivorRatio	= 8
MetaspaceSize	= 21807104 (20.796875MB)
CompressedClassSpaceSize	= 1073741824 (1024.0MB)
MaxMetaspaceSize	= 17592186044415 MB
G1HeapRegionSize	= 0 (0.0MB)

Heap Usage:

PS Young Generation

②、或者使用 `jmap -histo <pid>` 查看对象分布。


```
/Library/Java/JavaVirtualMachines/jdk-11.0.8.jdk/Contents/Home/bin/jmap -histo 10025
```

num	#instances	#bytes	class name (module)
1:	871	2392376	[I (java.base@11.0.8)
2:	9519	548304	[B (java.base@11.0.8)
3:	8038	192912	java.lang.String (java.base@11.0.8)
4:	4713	150816	java.util.HashMap\$Node (java.base@11.0.8)
5:	2111	121328	[Ljava.lang.Object; (java.base@11.0.8)
6:	860	105264	java.lang.Class (java.base@11.0.8)
7:	408	77576	[Ljava.util.HashMap\$Node; (java.base@11.0.8)
8:	45	56200	[C (java.base@11.0.8)
9:	1294	41408	java.util.concurrent.ConcurrentHashMap\$Node (java.base@11.0.8)
10:	661	34616	[Ljava.lang.String; (java.base@11.0.8)
11:	513	28728	jdk.internal.org.objectweb.asm.Item (java.base@11.0.8)
12:	614	24560	java.util.LinkedHashMap\$Entry (java.base@11.0.8)
13:	57	18960	[Ljava.util.concurrent.ConcurrentHashMap\$Node; (java.base@11.0.8)
14:	358	17184	java.util.HashMap (java.base@11.0.8)
15:	18	16800	[Ljdk.internal.org.objectweb.asm.Item; (java.base@11.0.8)
16:	379	12128	java.lang.invoke.MethodType\$ConcurrentWeakInternSet\$WeakEntry (java.base@11.0.8)
17:	221	10608	java.lang.invoke.MemberName (java.base@11.0.8)
18:	261	10440	java.lang.invoke.MethodType (java.base@11.0.8)
19:	432	10368	java.lang.StringBuilder (java.base@11.0.8)
20:	41	9184	jdk.internal.org.objectweb.asm.MethodWriter (java.base@11.0.8)

③、还有生成堆转储文件：`jmap -dump:format=b,file=<path> <pid>`。

```
~/Documents/GitHub/HelloWorld git:(master) ±127 (0.318s)
/Library/Java/JavaVirtualMachines/jdk-11.0.8.jdk/Contents/Home/bin/
jmap -dump:format=b,file=heap.hprof 10025
Heap dump file created

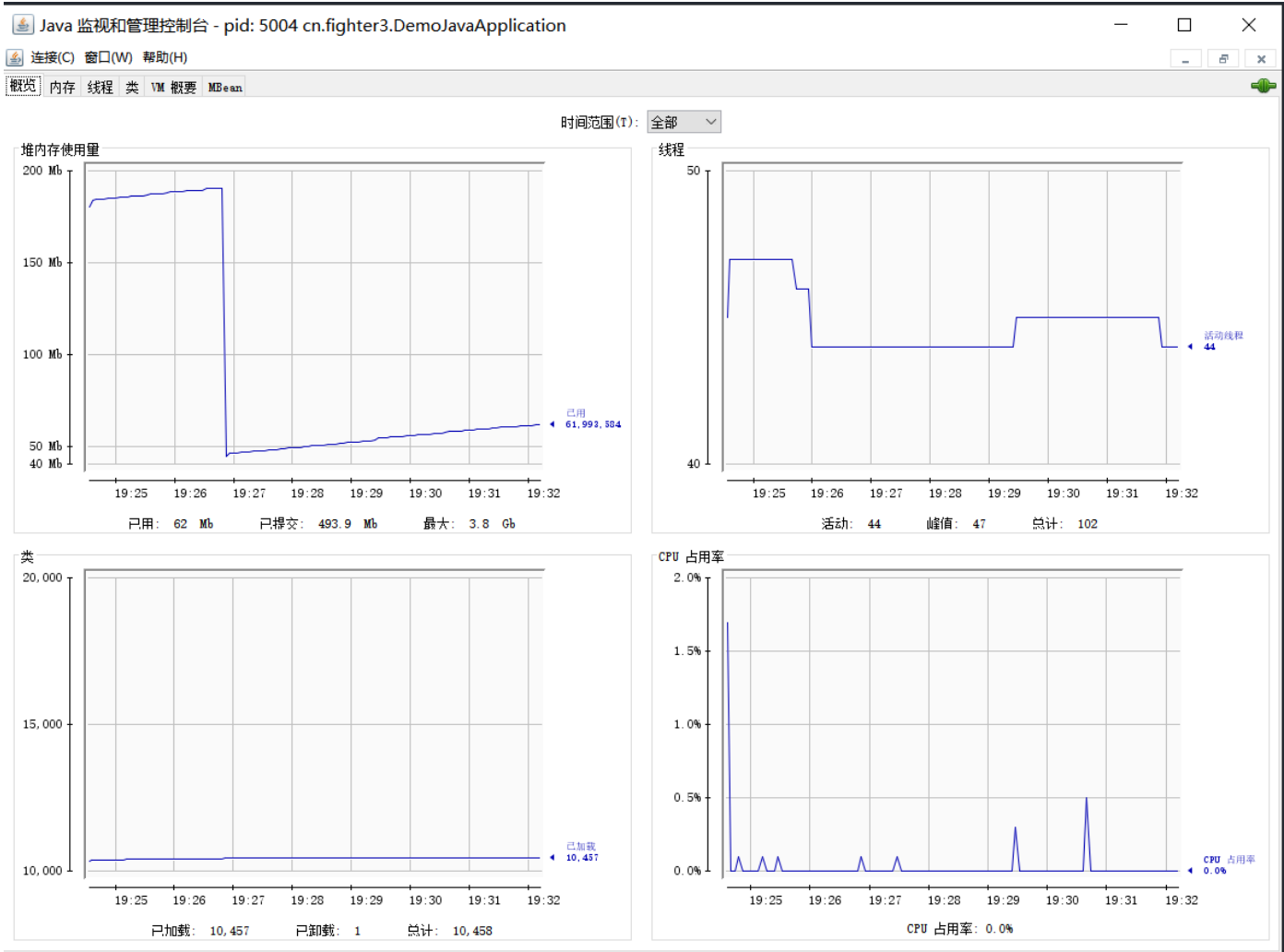
~/Documents/GitHub/HelloWorld git:(master) ±127 (0.083s)
ls
Example.class  HelloWorld.iml  JDK11.java      JDK8.java      out
Example.java   JDK11.class    JDK8.class      heap.hprof     src
```

1. [Java 面试指南 \(付费\)](#) 收录的哔哩哔哩同学 1 二面面试原题：你是如何使用jmap，你用过哪些命令？

38.了解哪些可视化的性能监控工具？

我自己用过的可视化工具主要有：

①、JConsole：JDK 自带的监控工具，可以用来监视 Java 应用程序的运行状态，包括内存使用、线程状态、类加载、GC 等。



②、VisualVM：一个基于 NetBeans 的可视化工具，在很长一段时间内，VisualVM 都是 Oracle 官方主推的故障处理工具。集成了多个 JDK 命令行工具的功能，非常友好。

插件

更新 可用插件 (18) 已下载 已安装 设置

检查最新版本(F)

搜索(S):

安装	名称	类别	源
☒	VisualVM-Glassfish	Application ...	社区提供的插件
☐	VisualVM-Extensions	Platform	社区提供的插件
☐	Startup Profiler	Profiling	社区提供的插件
☐	BTrace Workbench	Profiling	社区提供的插件
☐	VisualVM-Security	Security	社区提供的插件
☐	Visual GC	Tools	社区提供的插件
☐	VisualVM-BufferMonitor	Tools	社区提供的插件
☐	Threads Inspector	Tools	社区提供的插件
☐	VisualVM-JConsole	Tools	社区提供的插件
☐	VisualVM-MBeans	Tools	社区提供的插件
☐	KillApplication	Tools	社区提供的插件
☐	Tracer-Jvmstat Probes	Tracer	社区提供的插件
☐	Tracer-Monitor Probes	Tracer	社区提供的插件
☐	Tracer-Swing Probes	Tracer	社区提供的插件
☐	Tracer-IO Probes	Tracer	社区提供的插件
☐	Tracer-Collections Probes	Tracer	社区提供的插件
☐	Tracer-JVM Probes	Tracer	社区提供的插件
☐	OQL Syntax Support	UI	社区提供的插件

VisualVM-Glassfish

社区提供的插件

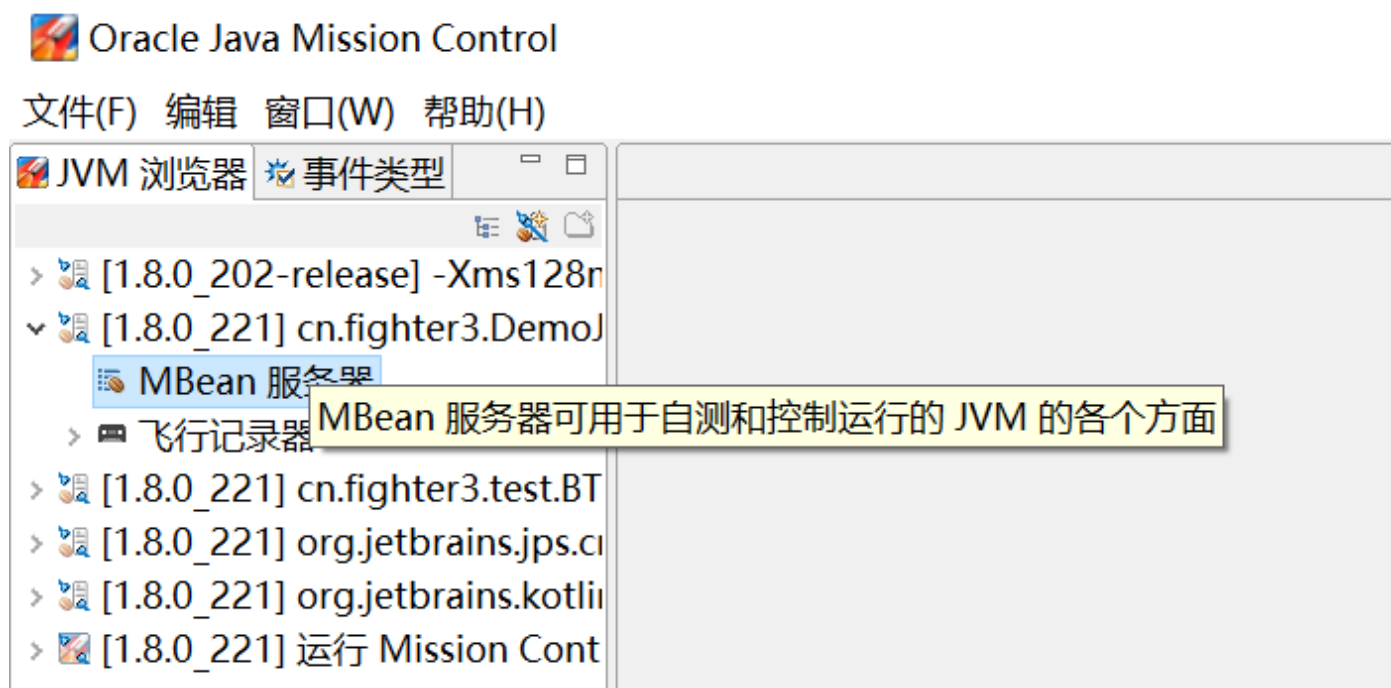
版本: 1.5
作者: Jaroslav Bachorik
日期: 16-11-7
源: Java VisualVM 插件中心
主页: <https://visualvm.github.io>

插件描述

A sample plugin giving an overview of advanced monitoring capabilities of VisualVM. Enhances monitoring of GlassFish application server by adding specialized overview, new tab for monitoring HTTP Service and the ability to visually select and monitor any of the deployed web applications

安装(I)

③、Java Mission Control: JMC 最初是 JRockit VM 中的诊断工具，但在 Oracle JDK7 Update 40 以后，就绑定到了 HotSpot VM 中。不过后来又又被 Oracle 开源出来作为一个单独的产品。



用过哪些第三方的工具?

- ①、**MAT**: 一个 Java 堆内存分析工具，主要用于分析和查找 Java 堆中的内存泄漏和内存消耗问题；可以从 Java 堆转储文件中分析内存使用情况，并提供丰富的报告，如内存泄漏疑点、最大对象和 GC 根信息；支持通过图形界面查询对象，以及检查对象间的引用关系。
- ②、**GChisto**: GC 日志分析工具，可以帮助我们优化垃圾收集行为和调整 GC 性能。
- ③、**JProfiler**: 一个全功能的商业化 Java 性能分析工具，提供 CPU、内存和线程的实时分析。
- ④、**arthas**: 阿里巴巴开源的 Java 诊断工具，主要用于线上的应用诊断；支持在不停机的情况下进行诊断；可以提供包括 JVM 信息查看、监控、Trace 命令、反编译等功能。
- ⑤、**async-profiler**: 一个低开销的性能分析工具，支持生成火焰图，适用于复杂性能问题的分析。

1. [Java 面试指南 \(付费\)](#) 收录的华为面经同学 9 Java 通用软件开发一面面试原题：如何查看当前 Java 程序里哪些对象正在使用，哪些对象已经被释放

39.JVM 的常见参数配置知道哪些?

配置堆内存大小的参数有哪些?

- `-Xms`: 初始堆大小
- `-Xmx`: 最大堆大小
- `-XX:NewSize=n`: 设置年轻代大小
- `-XX:NewRatio=n`: 设置年轻代和年老代的比值。如: n 为 3 表示年轻代和年老代比值为 1:3, 年轻代占总和的 $1/4$
- `-XX:SurvivorRatio=n`: 年轻代中 Eden 区与两个 Survivor 区的比值。如 $n=3$ 表示 Eden 占 3 Survivor 占 2, 一个 Survivor 区占整个年轻代的 $1/5$

配置 GC 收集器的参数有哪些？

- `-XX:+UseSerialGC`：设置串行收集器
- `-XX:+UseParallelGC`：设置并行收集器
- `-XX:+UseParalledlOldGC`：设置并行老年代收集器
- `-XX:+UseConcMarkSweepGC`：设置并发收集器

配置并行收集的参数有哪些？

- `-XX:MaxGCPauseMillis=n`：设置最大垃圾回收停顿时间
- `-XX:GCTimeRatio=n`：设置垃圾回收时间占程序运行时间的比例
- `-XX:+CMSIncrementalMode`：设置增量模式，适合单 CPU 环境
- `-XX:ParallelGCThreads=n`：设置并行收集器的线程数

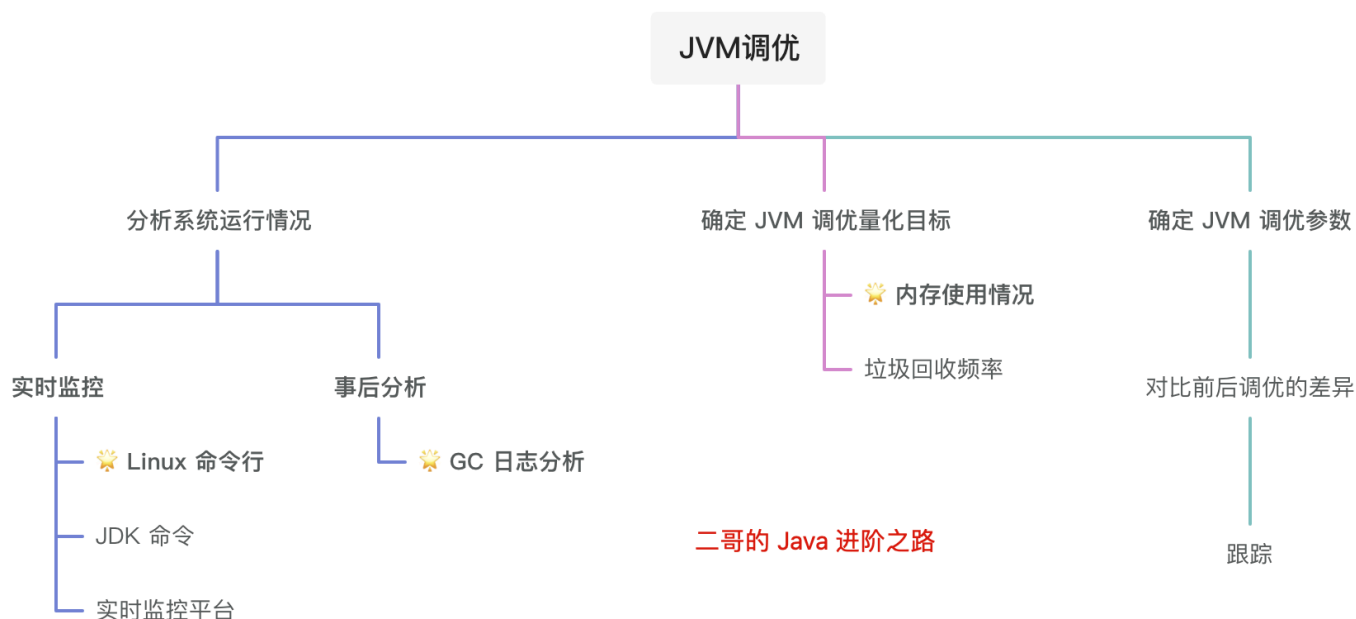
打印 GC 回收的过程日志信息的参数有哪些？

- `-XX:+PrintGC`：输出 GC 日志
- `-XX:+PrintGCDetails`：输出 GC 详细日志
- `-XX:+PrintGCTimeStamps`：输出 GC 的时间戳（以基准时间的形式）
- `-Xloggc:filename`：日志文件的输出路径

40.做过 JVM 调优吗？

做过。

JVM 调优是一个复杂的过程，调优的对象包括堆内存、垃圾收集器和 JVM 运行时参数等。



如果堆内存设置过小，可能会导致频繁的垃圾回收。所以在[技术派实战项目](#)中，启动 JVM 的时候配置了 `-Xms` 和 `-Xmx` 参数，让堆内存最大可用内存为 2G（我用的丐版服务器）。

在项目运行期间，我会使用 JVisualVM 定期观察和分析 GC 日志，如果发现频繁的 Full GC，我会特意关注一下老年代的使用情况。

接着，通过分析 Heap dump 寻找内存泄漏的源头，看看是否有未关闭的资源，长生命周期的大对象等。

之后进行代码优化，比如说减少大对象的创建、优化数据结构的使用方式、减少不必要的对象持有等。

1. [Java 面试指南 \(付费\)](#) 收录的华为面经同学 6 Java 通用软件开发一面面试题：说说你对 JVM 调优的了解

41.CPU 占用过高怎么排查？

CPU飙高排查



首先，使用 top 命令查看 CPU 占用情况，找到占用 CPU 较高的进程 ID。

```
top
```

```
top - 07:27:05 up 1:57, 3 users, load average: 0.00, 0.00, 0.00
Tasks: 150 total, 1 running, 149 sleeping, 0 stopped, 0 zombie
Cpu(s): 0.0%us, 0.3%sy, 0.0%ni, 99.7%id, 0.0%wa, 0.0%hi, 0.0%si, 0.0%st
Mem: 1003020k total, 899340k used, 103680k free, 22248k buffers
Swap: 2031612k total, 536k used, 2031076k free, 505856k cached
```

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
14291	root	20	0	254m	8192	4996	S	0.3	0.8	0:04.63	vmtoolsd
37600	niuben	20	0	15028	1272	948	R	0.3	0.1	0:01.60	top
1	root	20	0	19352	1516	1212	S	0.0	0.2	0:00.91	init
2	root	20	0	0	0	0	S	0.0	0.0	0:00.00	kthreadd
3	root	RT	0	0	0	0	S	0.0	0.0	0:00.00	migration/0
4	root	20	0	0	0	0	S	0.0	0.0	0:00.02	ksoftirqd/0
5	root	RT	0	0	0	0	S	0.0	0.0	0:00.00	stopper/0
6	root	RT	0	0	0	0	S	0.0	0.0	0:00.00	watchdog/0
7	root	20	0	0	0	0	S	0.0	0.0	0:02.28	events/0
8	root	20	0	0	0	0	S	0.0	0.0	0:00.00	events/0
9	root	20	0	0	0	0	S	0.0	0.0	0:00.00	events_long/0
10	root	20	0	0	0	0	S	0.0	0.0	0:00.00	events_power_ef
11	root	20	0	0	0	0	S	0.0	0.0	0:00.00	cgroup
12	root	20	0	0	0	0	S	0.0	0.0	0:00.00	khelper

接着，使用 jstack 命令查看对应进程的线程堆栈信息。

```
jstack -l <pid> > thread-dump.txt
```

上面👉这个命令会将所有线程的堆栈信息输出到 thread-dump.txt 文件中。

然后再使用 top 命令查看进程中线程的占用情况，找到占用 CPU 较高的线程 ID。

```
top -H -p <pid>
```

```
[root@19test ~]# top -H -p 31357
top - 23:13:44 up 113 days, 9:23,  2 users,  load average: 0.00, 0.00, 0.00
Tasks: 84 total,  0 running, 84 sleeping,  0 stopped,  0 zombie
Cpu(s): 0.1%us, 0.0%sy, 0.0%ni, 99.9%id, 0.0%wa, 0.0%hi, 0.0%si, 0.0%st
Mem:   8061376k total, 7914040k used, 147336k free, 142232k buffers
Swap: 3145720k total, 1167148k used, 1978572k free, 706504k cached
```

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
31357	root	20	0	5711m	1.7g	15m	S	0.0	21.6	0:00.16	java
31359	root	20	0	5711m	1.7g	15m	S	0.0	21.6	0:00.28	java
31360	root	20	0	5711m	1.7g	15m	S	0.0	21.6	0:01.63	java
31361	root	20	0	5711m	1.7g	15m	S	0.0	21.6	0:01.48	java
31362	root	20	0	5711m	1.7g	15m	S	0.0	21.6	0:01.48	java
31363	root	20	0	5711m	1.7g	15m	S	0.0	21.6	0:01.61	java
31364	root	20	0	5711m	1.7g	15m	S	0.0	21.6	0:03.95	java
31365	root	20	0	5711m	1.7g	15m	S	0.0	21.6	0:03.32	java
31366	root	20	0	5711m	1.7g	15m	S	0.0	21.6	0:00.06	java
31367	root	20	0	5711m	1.7g	15m	S	0.0	21.6	0:00.06	java
31368	root	20	0	5711m	1.7g	15m	S	0.0	21.6	0:00.00	java
31369	root	20	0	5711m	1.7g	15m	S	0.0	21.6	0:00.00	java
31370	root	20	0	5711m	1.7g	15m	S	0.0	21.6	0:46.57	java
31371	root	20	0	5711m	1.7g	15m	S	0.0	21.6	0:50.12	java
31372	root	20	0	5711m	1.7g	15m	S	0.0	21.6	0:11.58	java
31373	root	20	0	5711m	1.7g	15m	S	0.0	21.6	0:00.00	java
31374	root	20	0	5711m	1.7g	15m	S	0.0	21.6	0:00.00	java
31375	root	20	0	5711m	1.7g	15m	S	0.0	21.6	0:00.00	java

注意，top 命令显示的线程 ID 是十进制的，而 jstack 输出的是十六进制的，所以需要将线程 ID 转换为十六进制。

```
printf "%x\n" PID
```

接着在 jstack 的输出中搜索这个十六进制的线程 ID，找到对应的堆栈信息。

```
"Thread-5" #21 prio=5 os_prio=0 tid=0x00007f812c018800 nid=0x1a85 runnable
[0x00007f811c000000]
  java.lang.Thread.State: RUNNABLE
    at com.example.MyClass.myMethod(MyClass.java:123)
    at ...
```


最后，根据堆栈信息定位到具体的业务方法，查看是否有死循环、频繁的垃圾回收、资源竞争导致的上下文频繁切换等问题。

1. [Java 面试指南（付费）](#) 收录的阿里面经同学 1 闲鱼后端一面的原题：上线的业务出了问题怎么调试，比如某个线程 cpu 占用率高，怎么看堆栈信息
2. [Java 面试指南（付费）](#) 收录的快手同学 4 一面原题：服务器的CPU占用持续升高，有哪些排查问题的手段？排查后发现是项目产生了内存泄露，如何确定问题出在哪里？

42.内存飙升问题怎么排查？

内存飙升一般是因为创建了大量的 Java 对象导致的，如果持续飙升则说明垃圾回收跟不上对象创建的速度，或者内存泄漏导致对象无法回收。

排查的方法主要分为以下几步：

第一，先观察垃圾回收的情况，可以通过 `jstat -gc PID 1000` 查看 GC 次数和时间。

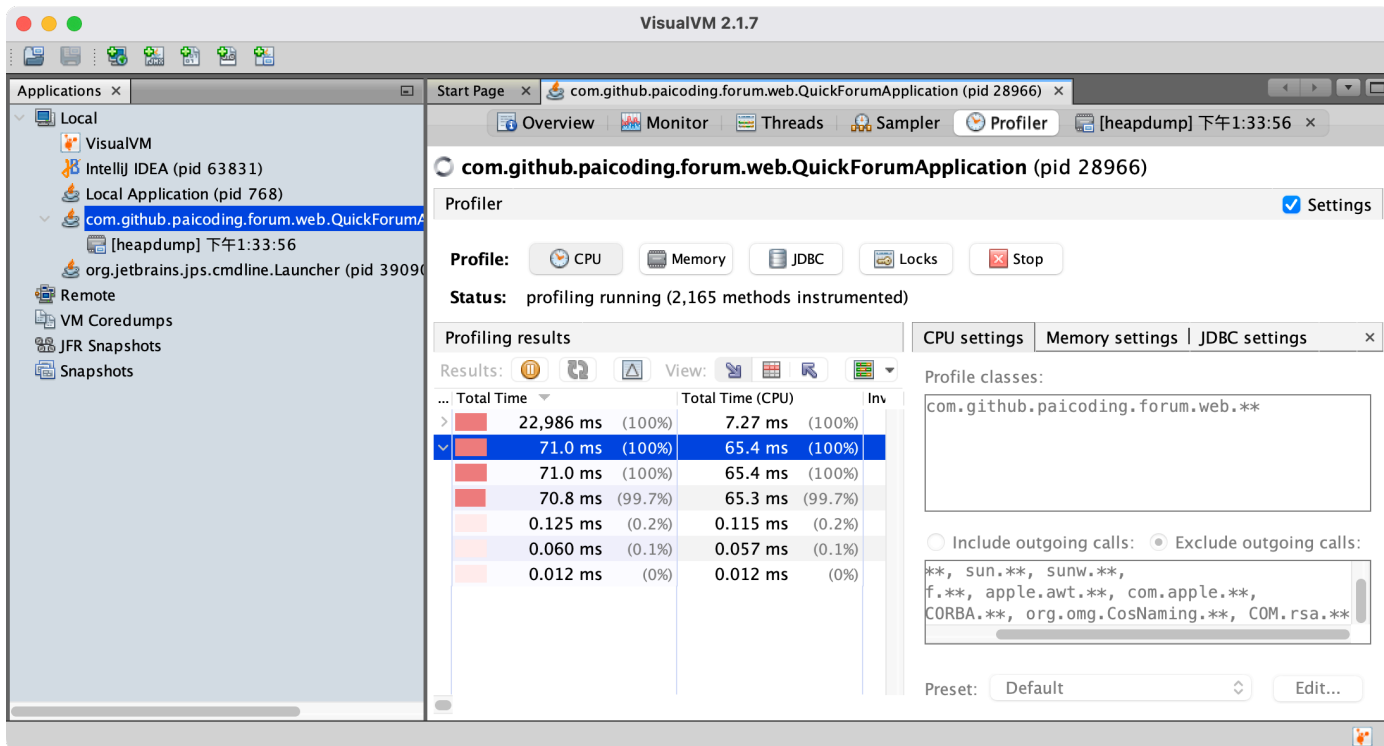
或者使用 `jmap -histo PID | head -20` 查看堆内存占用空间最大的前 20 个对象类型。

第二步，通过 jmap 命令 dump 出堆内存信息。

```
~/Documents/GitHub/HelloWorld git:(master) ±127 (0.318s)
/Library/Java/JavaVirtualMachines/jdk-11.0.8.jdk/Contents/Home/bin/
jmap -dump:format=b,file=heap.hprof 10025
Heap dump file created
```

```
~/Documents/GitHub/HelloWorld git:(master) ±127 (0.083s)
ls
Example.class  HelloWorld.iml  JDK11.java      JDK8.java      out
Example.java   JDK11.class    JDK8.class      heap.hprof      src
```

第三步，使用可视化工具分析 dump 文件，比如说 VisualVM，找到占用内存高的对象，再找到创建该对象的业务代码位置，从代码和业务场景中定位具体问题。



1. [Java 面试指南 \(付费\)](#) 收录的联想面经同学 7 面试原题：怎么定位线上的内存问题。

43. 频繁 minor gc 怎么办？

频繁的 Minor GC 通常意味着新生代中的对象频繁地被垃圾回收，可能是因为新生代空间设置的过小，或者是因为程序中存在大量的短生命周期对象（如临时变量）。

可以使用 GC 日志进行分析，查看 GC 的频率和耗时，找到频繁 GC 的原因。

```
-XX:+PrintGCDetails -Xloggc:gc.log
```

或者使用监控工具查看堆内存的使用情况，特别是新生代（Eden 和 Survivor 区）的使用情况。

如果是因为新生代空间不足，可以通过 `-Xmn` 增加新生代的大小，减缓新生代的填满速度。

```
java -Xmn256m your-app.jar
```

如果对象需要长期存活，但频繁从 Survivor 区晋升到老年代，可以通过 `-XX:SurvivorRatio` 参数调整 Eden 和 Survivor 的比例。默认比例是 8:1，表示 8 个空间用于 Eden，1 个空间用于 Survivor 区。

```
-XX:SurvivorRatio=6
```

调整为 6 的话，会减少 Eden 区的大小，增加 Survivor 区的大小，以确保对象在 Survivor 区中存活的时间足够长，避免过早晋升到老年代。

1. [Java 面试指南 \(付费\)](#) 收录的京东面经同学 8 面试原题：young GC 频繁如何排查？修改哪些参数？

44. 频繁 Full GC 怎么办？

频繁的 Full GC 通常意味着老年代中的对象频繁地被垃圾回收，可能是因为老年代空间设置的过小，或者是因为程序中存在大量的长生命周期对象。

该怎么排查 Full GC 频繁问题？

我厂会通过专门的性能监控系统，查看 GC 的频率和堆内存的使用情况，然后根据监控数据分析 GC 的原因。

如果是小厂，可以这么回复。

我一般会使用 JDK 的自带工具，包括 jmap、jstat 等。

```
# 查看堆内存各区域的使用率以及gc情况
jstat -gcutil -h20 pid 1000
# 查看堆内存中的存活对象，并按空间排序
jmap -histo pid | head -n20
# dump堆内存文件
jmap -dump:format=b,file=heap pid
```

或者使用一些可视化的工具，比如 VisualVM、JConsole 等，查看堆内存的使用情况。

假如是因为大对象直接分配到老年代导致的 Full GC 频繁，可以通过 `-XX:PretenureSizeThreshold` 参数设置大对象直接进入老年代的阈值。

或者将大对象拆分成小对象，减少大对象的创建。比如说分页。

假如是因为内存泄漏导致的频繁 Full GC，可以通过分析堆内存 dump 文件找到内存泄漏的对象，再找到内存泄漏的代码位置。

假如是因为长生命周期的对象进入到了老年代，要及时释放资源，比如说 ThreadLocal、数据库连接、IO 资源等。

假如是因为 GC 参数配置不合理导致的频繁 Full GC，可以通过调整 GC 参数来优化 GC 行为。或者直接更换更适合的 GC 收集器，如 G1、ZGC 等。

1. [Java 面试指南（付费）](#) 收录的得物面经同学 8 一面面试原题：Java 中 full gc 频繁，有哪些原因

五、类加载机制

45. 🌟 了解类的加载机制吗？（补充）

2024 年 03 月 29 日增补

了解。

JVM 的操作对象是 Class 文件，JVM 把 Class 文件中描述类的数据结构加载到内存中，并对数据进行校验、解析和初始化，最终转化成可以被 JVM 直接使用的类型，这个过程被称为类加载机制。

其中最重要的三个概念就是：类加载器、类加载过程和双亲委派模型。

- **类加载器**：负责加载类文件，将类文件加载到内存中，生成 Class 对象。
- **类加载过程**：包括加载、验证、准备、解析和初始化等步骤。
- **双亲委派模型**：当一个类加载器接收到类加载请求时，它会把请求委派给父——类加载器去完成，依次递归，直到最顶层的类加载器，如果父——类加载器无法完成加载请求，子类加载器才会尝试自己去加载。

1. [Java 面试指南 \(付费\)](#) 收录的小米暑期实习同学 E 一面面试原题：你了解类的加载机制吗？
2. [Java 面试指南 \(付费\)](#) 收录的美团面经同学 3 Java 后端技术一面面试原题：java 的类加载机制 双亲委派机制 这样设计的原因是什么

46.类加载器有哪些？

主要有四种：

- ①、**启动类加载器**，负责加载 JVM 的核心类库，如 rt.jar 和其他核心库位于 `JAVA_HOME/jre/lib` 目录下的类。
- ②、**扩展类加载器**，负责加载 `JAVA_HOME/jre/lib/ext` 目录下，或者由系统属性 `java.ext.dirs` 指定位置的类库，由 `sun.misc.Launcher$ExtClassLoader` 实现。
- ③、**应用程序类加载器**，负责加载 classpath 的类库，由 `sun.misc.Launcher$AppClassLoader` 实现。

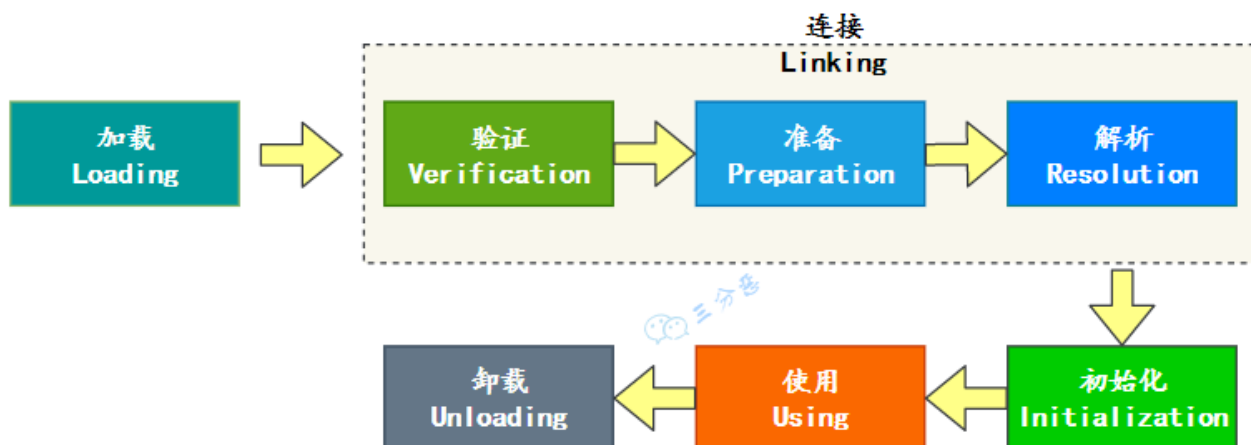
我们编写的任何类都是由应用程序类加载器加载的，除非显式使用自定义类加载器。

- ④、**用户自定义类加载器**，通常用于加载网络上的类、执行热部署（动态加载和替换应用程序的组件），或者为了安全考虑，从不同的源加载类。

通过继承 `java.lang.ClassLoader` 类来实现。

47.能说一下类的生命周期吗？

一个类从被加载到虚拟机内存中开始，到从内存中卸载，整个生命周期需要经过七个阶段：加载、验证、准备、解析、初始化、使用 and 卸载。



48.🌟类装载的过程知道吗？

推荐阅读：[一文彻底搞懂Java类加载机制](#)

知道。

类装载过程包括三个阶段：载入、链接和初始化。

- ①、**载入**：将类的二进制字节码加载到内存中。

②、链接可以细分为三个小的阶段：

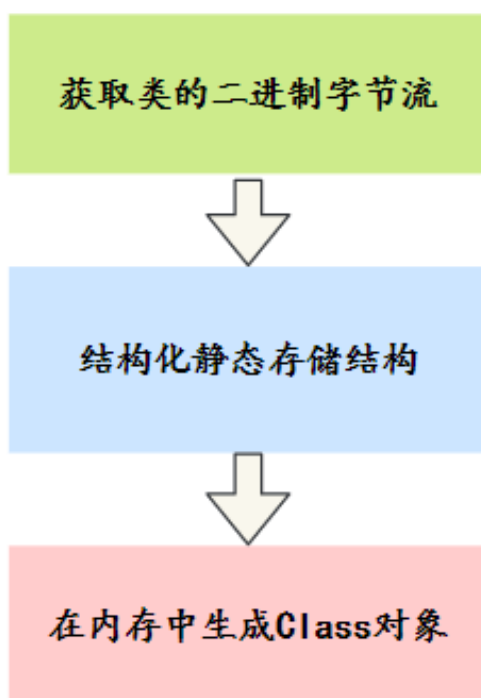
- 验证：检查类文件格式是否符合 JVM 规范
- 准备：为类的静态变量分配内存并设置默认值。
- 解析：将符号引用替换为直接引用。

③、初始化：执行静态代码块和静态变量初始化。

在准备阶段，静态变量已经被赋过默认初始值了，在初始化阶段，静态变量将被赋值为代码期望赋的值。比如说 `static int a = 1;`，在准备阶段，`a` 的值为 0，在初始化阶段，`a` 的值为 1。

换句话说，初始化阶段是在执行类的构造方法，也就是 `javap` 中看到的 `<clinit>()`。

载入过程 JVM 会做什么？



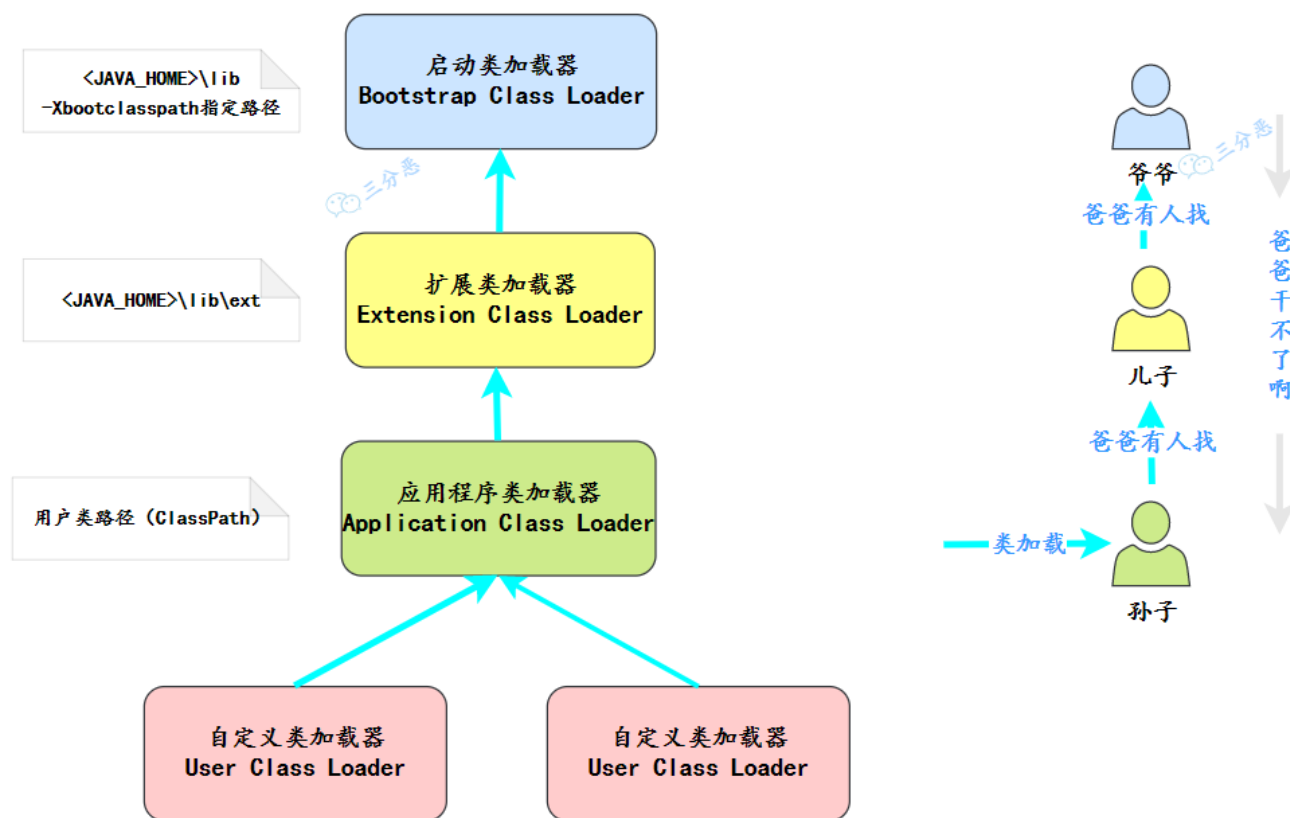
- 1) 通过一个类的全限定名来获取定义此类的二进制字节流。
- 2) 将这个字节流所代表的静态存储结构转化为方法区的运行时数据结构。
- 3) 在内存中生成一个代表这个类的 `java.lang.Class` 对象，作为这个类的访问入口。

1. [Java 面试指南（付费）](#) 收录的小米暑期实习同学 E 一面面试原题：你了解类的加载机制吗？
2. [Java 面试指南（付费）](#) 收录的美团面经同学 16 暑期实习一面面试原题：讲一下类加载过程，双亲委派模型，双亲委派的好处
3. [Java 面试指南（付费）](#) 收录的美团面经同学 18 成都到家面试原题：类加载过程
4. [Java 面试指南（付费）](#) 收录的快手同学 4 一面原题：类装载的执行过程？双亲委派模式是什么？为什么使用这种模式？

memo：2025 年 1 月 17 日修改至此。

49. 🌟 什么是双亲委派模型？

双亲委派模型要求类加载器在加载类时，先委托父加载器尝试加载，只有父加载器无法加载时，子加载器才会加载。



这个过程会一直向上递归，也就是说，从子加载器到父加载器，再到更上层的加载器，一直到最顶层的启动类加载器。

启动类加载器会尝试加载这个类。如果它能够加载这个类，就直接返回；如果它不能加载这个类，就会将加载任务返回给委托它的子加载器。

子加载器尝试加载这个类。如果子加载器也无法加载这个类，它就会继续向下传递这个加载任务，依此类推。

直到某个加载器能够加载这个类，或者所有加载器都无法加载这个类，最终抛出 `ClassNotFoundException`。

1. [Java 面试指南（付费）](#) 收录的小米暑期实习同学 E 一面面试原题：你了解类的加载机制吗？
2. [Java 面试指南（付费）](#) 收录的阿里云面经同学 22 面经：双亲委派机制

49. 为什么要用双亲委派模型？

- ①、避免类的重复加载：父加载器加载的类，子加载器无需重复加载。
- ②、保证核心类库的安全性：如 `java.lang.*` 只能由 `Bootstrap ClassLoader` 加载，防止被篡改。

1. [Java 面试指南（付费）](#) 收录的美团面经同学 16 暑期实习一面面试原题：讲一下类加载过程，双亲委派模型，双亲委派的好处

50. 如何破坏双亲委派机制？

重写 `ClassLoader` 的 `loadClass()` 方法。

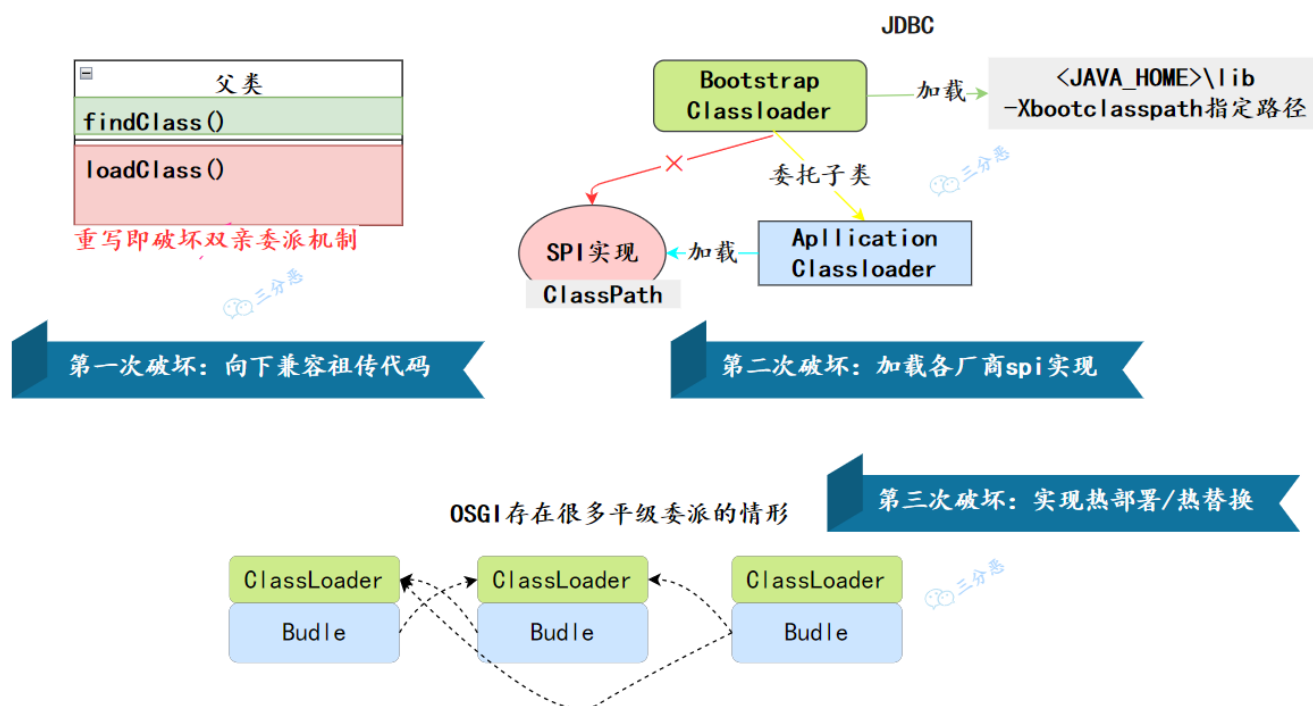
如果不想打破双亲委派模型，就重写 `ClassLoader` 类中的 `findClass()` 方法，那些无法被父类加载器加载的类最终会通过这个方法被加载。

memo：2025 年 1 月 18 日修改至此。

51. 有哪些破坏双亲委派模型的典型例子？

我了解的有两种：

- 第一种：SPI 机制加载 JDBC 驱动。
- 第二种：热部署框架。



说说SPI 机制？

SPI 是 Java 的一种扩展机制，用于加载和注册第三方类库，常见于 JDBC、JNDI 等框架。

双亲委派模型会优先让父类加载器加载类，而 SPI 需要动态加载子类加载器中的实现。

根据双亲委派模型，`java.sql.Driver` 类应该由父加载器加载，但父类加载器无法加载由子类加载器定义的驱动类，如 MySQL 的 `com.mysql.cj.jdbc.Driver`。

那么只能使用 SPI 机制通过 `META-INF/services` 文件指定服务提供者的实现类。

```
ClassLoader cl = Thread.currentThread().getContextClassLoader();
Enumeration<Driver> drivers = ServiceLoader.load(Driver.class, cl).iterator();
```

`DriverManager` 使用了线程上下文类加载器来加载 SPI 的实现类，从而允许子类加载器加载具体的 JDBC 驱动。

说说热部署？

热部署是指在不重启服务器的情况下更新应用程序代码，需要替换旧版本的类，但旧版本的类可能由父加载器加载。

如 Spring Boot 的 DevTools 通常会自定义类加载器，优先加载新的类版本。

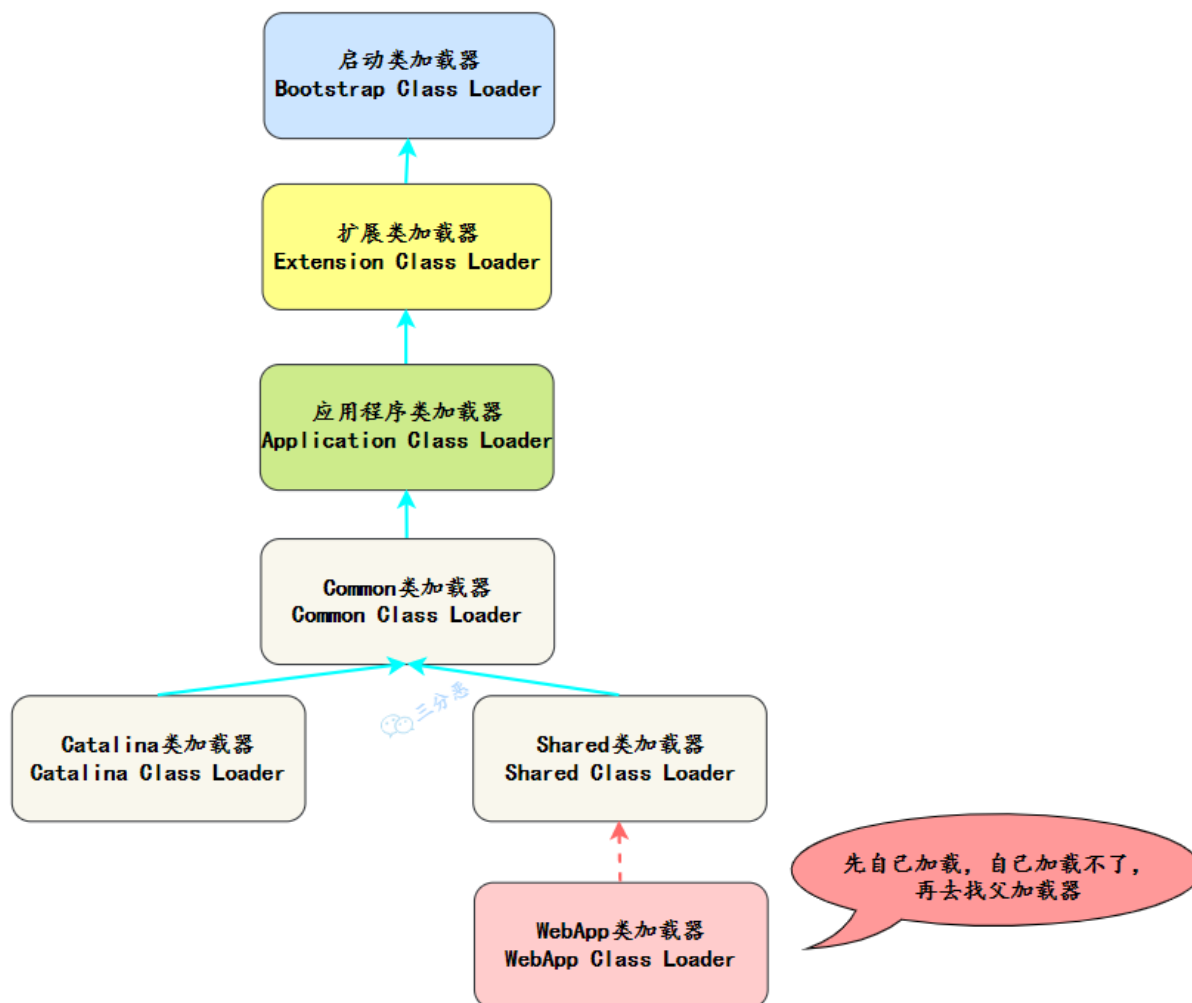
memo：2025 年 1 月 19 日修改至此。

52.Tomcat 的类加载机制了解吗？

了解。

Tomcat 基于双亲委派模型进行了一些扩展，主要的类加载器有：

- Bootstrap ClassLoader：加载 Java 的核心类库；
- Catalina ClassLoader：加载 Tomcat 的核心类库；
- Shared ClassLoader：加载共享类库，允许多个 Web 应用共享某些类库；
- WebApp ClassLoader：加载 Web 应用程序的类库，支持多应用隔离和优先加载应用自定义的类库（破坏了双亲委派模型）。



53.你觉得应该怎么实现一个热部署功能？

热部署是指在不重启服务器的情况下，动态加载、更新或卸载应用程序的组件，比如类、配置文件等。

需要在类加载器的基础上，实现类的重新加载。

我的思路是：

第一步，使用文件监控机制，如 Java NIO 的 WatchService 来监控类文件或配置文件的变化。当监控到文件变更时，触发热部署流程。

```
class FileWatcher {

    public static void watchDirectoryPath(Path path) {
        // 检查路径是否是有效目录
        if (!isDirectory(path)) {
            System.err.println("Provided path is not a directory: " + path);
            return;
        }

        System.out.println("Starting to watch path: " + path);

        // 获取文件系统的 WatchService
        try (WatchService watchService = path.getFileSystem().newWatchService()) {
            // 注册目录监听服务，监听创建、修改和删除事件
            path.register(watchService, ENTRY_CREATE, ENTRY_MODIFY, ENTRY_DELETE);

            while (true) {
                WatchKey key;
                try {
                    // 阻塞直到有事件发生
                    key = watchService.take();
                } catch (InterruptedException e) {
                    System.out.println("WatchService interrupted, stopping directory
watch.");

                    Thread.currentThread().interrupt();
                    break;
                }

                // 处理事件
                for (WatchEvent<?> event : key.pollEvents()) {
                    processEvent(event);
                }

                // 重置 key，如果失败则退出
                if (!key.reset()) {
                    System.out.println("WatchKey no longer valid. Exiting watch loop.");
                    break;
                }
            }
        } catch (IOException e) {
            System.err.println("An error occurred while setting up the WatchService: " +
e.getMessage());
            e.printStackTrace();
        }
    }
}
```

```

    }
}

private static boolean isDirectory(Path path) {
    return Files.isDirectory(path, LinkOption.NOFOLLOW_LINKS);
}

private static void processEvent(WatchEvent<?> event) {
    WatchEvent.Kind<?> kind = event.kind();

    // 处理事件类型
    if (kind == OVERFLOW) {
        System.out.println("Event overflow occurred. Some events might have been lost.");
        return;
    }

    @SuppressWarnings("unchecked")
    Path fileName = ((WatchEvent<Path>) event).context();
    System.out.println("Event: " + kind.name() + ", File affected: " + fileName);
}

public static void main(String[] args) {
    // 设置监控路径为当前目录
    Path pathToWatch = Paths.get(".");
    watchDirectoryPath(pathToWatch);
}
}

```

第二步，创建一个自定义类加载器，继承 `java.lang.ClassLoader`，并重写 `findClass()` 方法，用来加载新的类文件。

```

class HotSwapClassLoader extends ClassLoader {
    public HotSwapClassLoader() {
        super(ClassLoader.getSystemClassLoader());
    }

    @Override
    protected Class<?> findClass(String name) throws ClassNotFoundException {
        // 加载指定路径下的类文件字节码
        byte[] classBytes = loadClassData(name);
        if (classBytes == null) {
            throw new ClassNotFoundException(name);
        }
        // 调用defineClass将字节码转换为Class对象
        return defineClass(name, classBytes, 0, classBytes.length);
    }

    private byte[] loadClassData(String name) {
        // 实现从文件系统或其他来源加载类文件的字节码
        // ...
    }
}

```

```
        return null;
    }
}
```

友情提示：IntelliJ IDEA 提供了热部署功能，当我们修改了代码后，IDEA 会自动保存并编译，如果是 Web 项目，还可以在 Chrome 浏览器中装一个 LiveReload 插件，一旦编译完成，页面就会自动刷新看到最新的效果。对于测试或者调试来说，非常方便。

1. [Java 面试指南（付费）](#) 收录的小米暑期实习同学 E 一面面试原题：那你知道类的热更新的？

54. 说说解释执行和编译执行的区别（补充）

2024 年 03 月 08 日增补

先说解释和编译的区别：

- 解释：将源代码逐行转换为机器码。
- 编译：将源代码一次性转换为机器码。

一个是逐行，一个是一次性，再来说说解释执行和编译执行的区别：

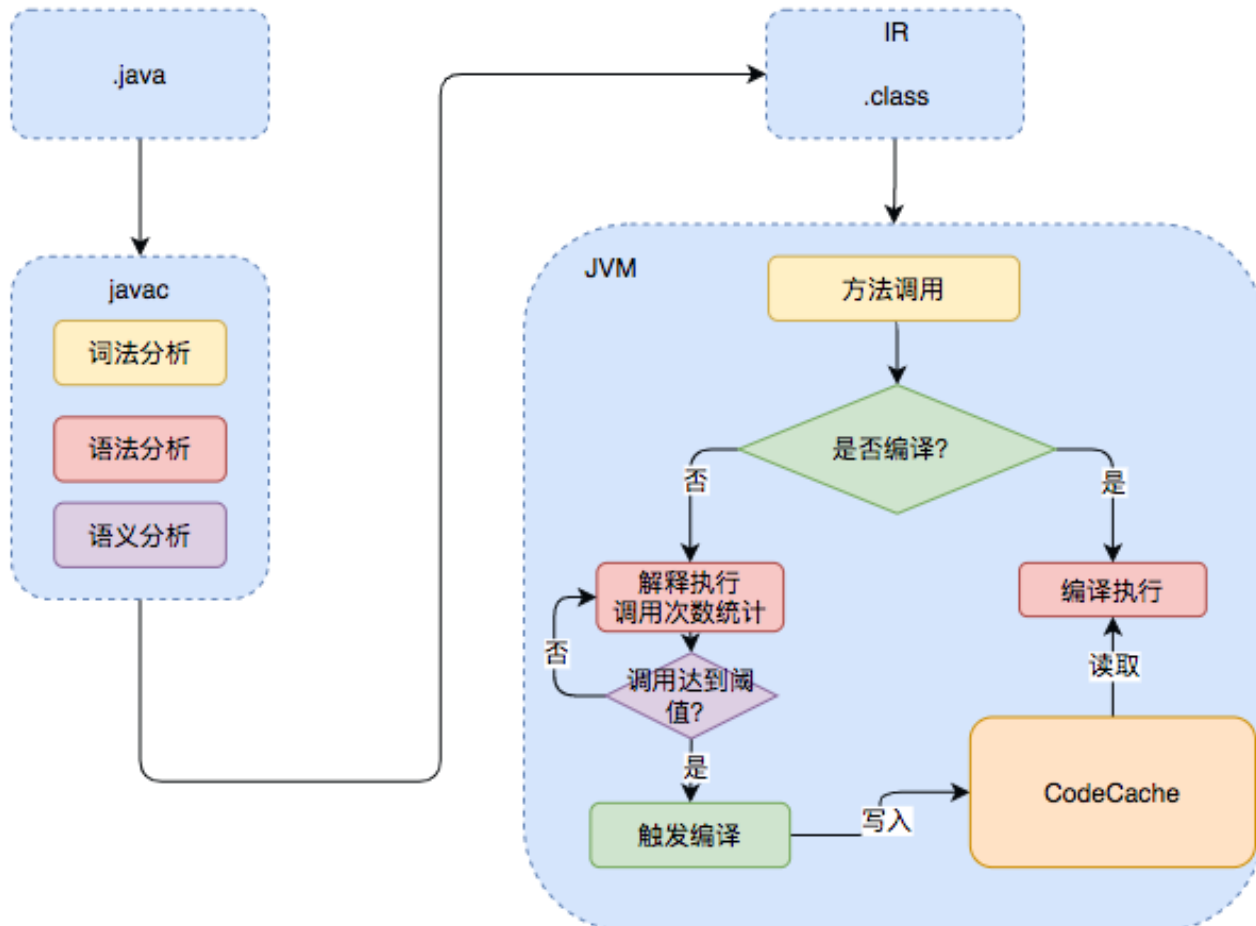
- 解释执行：程序运行时，将源代码逐行转换为机器码，然后执行。
- 编译执行：程序运行前，将源代码一次性转换为机器码，然后执行。

Java 一般被称为“解释型语言”，因为 Java 代码在执行前，需要先将源代码编译成字节码，然后在运行时，再由 JVM 的解释器“逐行”将字节码转换为机器码，然后执行。

这也是 Java 被诟病“慢”的主要原因。

但 JIT 的出现打破了这种刻板印象，JVM 会将热点代码（即运行频率高的代码）编译后放入 CodeCache，当下次执行再遇到这段代码时，会从 CodeCache 中直接读取机器码，然后执行。

因此，Java 的执行效率得到了大幅提升。



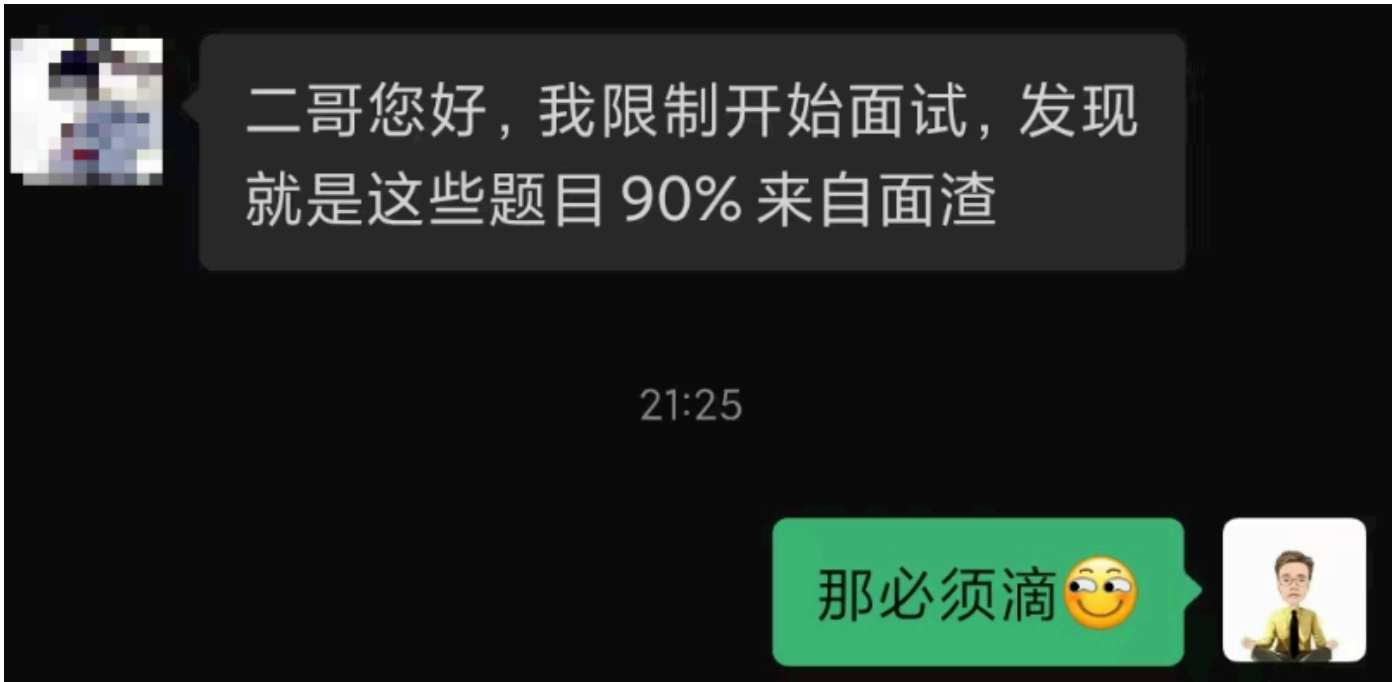
1. [Java 面试指南 \(付费\)](#) 收录的腾讯 Java 后端实习一面原题：说说 Java 解释执行的流程。

memo：2025 年 1 月 21 日修改至此。

面渣逆袭 JVM 篇第二版终于整理完了，说一点心里话。



网上的八股其实不少，这样可以给大家提供更多的选择，但面渣逆袭的含金量懂的都懂。



面渣逆袭第二版是在星球嘉宾三分恶的初版基础上，加入了二哥自己的思考，加入了 1000 多份真实面经之后的结果，并且从从 24 届到 25 届，帮助了很多小伙伴。未来的 26、27 届，也将因此受益，从而拿到心仪的 offer。能帮助到大家，我很欣慰，并且在重制面渣逆袭的过程中，我也成长了很多，很多薄弱的基础环节都得到了加强。



花舞洛伊人 2 进阶牛

11-19 21:46 内蒙古农业大学 Java 发布于内蒙古

八股文看谁的

如题，javaguide，二哥面渣，小林code，看谁的🤔



银行究极大孝子 6 大橘已定 🐼 潜力领航者

南京理工大学 Java

全是广告哥，我觉得二哥面渣/小林就足够了，你不一定要只看一家，一个八股问题，看其他博客，集百家之长

👍 36 💬 回复 ➦ 分享 发布于 09-03 14:44 江苏



1589494222 3 出师牛 🐼 潜力领航者 楼主 回复

中肯的

👍 1 💬 回复 发布于 09-03 20:55 北京 来自Android客户端



流连~ 5 飞黄腾达

深圳技术大学 Java

二哥的面渣逆袭，还有掘金竹子爱熊猫的专栏，个人感觉还是不错的

👍 32 💬 回复 ➦ 分享 发布于 09-01 14:46 广东 来自Android客户端



陕西 4天前



双非硕测开开了17×15拒了🤔二哥八股文无敌



沉默王二 作者 4天前



感谢认可，八股是不是吊打面试官😊



； 陕西 4天前



回复 沉默王二：包吊打的👍

很多时候，我觉得自己是一个佛系的人，不愿意和别人争个高低，也不愿意去刻意宣传自己的作品。

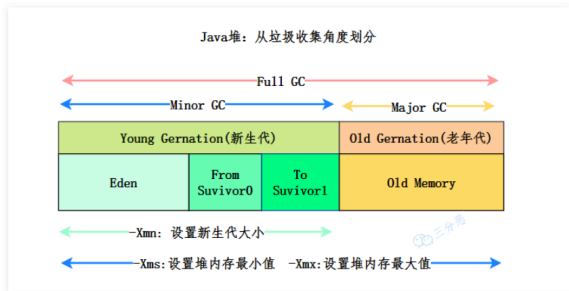
我喜欢静待花开。

如果你觉得面渣逆袭还不错，可以告诉学弟学妹们有这样一份免费的学习资料，帮我做个口碑。

我还会继续优化，也不确定第三版什么时候会来，但我会尽力。

愿大家都有一个光明的未来。

这次仍然是三个版本，亮白、暗黑和 epub 版本。给大家展示其中一个 epub 版本吧，有些小伙伴很急需这个版本，所以也满足大家了。



随着 **JIT 编译器** 的发展和逃逸技术的逐渐成熟，“所有的对象都会分配到堆上”就不再那么绝对了。

从 JDK 7 开始，JVM 默认开启了逃逸分析，意味着如果某些方法中的对象引用没有被返回或者没有在方法体外使用，也就是未逃逸出去，那么对象可以直接在栈上分配内存。

堆和栈的区别是什么？

堆属于线程共享的内存区域，几乎所有 new 出来的对象都会堆上分配，生命周期不由单个方法调用所决定，可以在方法调用结束后继续存在，直到不再被任何变量引用，最后被垃圾收集器回收。

栈属于线程私有的内存区域，主要存储局部变量、方法参数、对象引用等，通常随着方法调用的结束而自动释放，不需要垃圾收集器处理。

介绍一下方法区？

方法区并不真实存在，属于 Java 虚拟机规范中的一个逻辑概念，用于存储已被 JVM 加载的类信息、常量、静态变量、即时编译器编译后的代码缓存等。

在 HotSpot 虚拟机中，方法区的实现称为永久代 PermGen，但在 Java 8 及之后的版本中，已经被元空间 Metaspace 所替代。

变量存在堆栈的什么位置？

对于局部变量，它存储在当前方法栈帧中的局部变量表中。当方法执行完毕，栈帧被回收，局部变量也会被释放。

```
public void method() {  
    int localVar = 100; // 局部变量，存储在栈帧中的  
}
```

对于静态变量来说，它存储在 Java 虚拟机规范中的方法区中，在 Java 7 中是永久带，在 Java8 及以后是元空间。

```
public class StaticVarDemo {  
    public static int staticVar = 100; // 静态变  
}
```

1. **Java 面试指南（付费）** 收录的京东同学 10 后端实习一面的原题：堆和栈的区别是什

由于 PDF 没办法自我更新，所以需要最新版的小伙伴，可以微信搜【沉默王二】，或者扫描/长按识别下面的二维码，关注二哥的公众号，回复【222】即可拉取最新版本。



- [面渣逆袭 Spring 篇](#) 🍷
- [面渣逆袭 Redis 篇](#) 🍷
- [面渣逆袭 MyBatis 篇](#) 🍷
- [面渣逆袭 MySQL 篇](#) 🍷
- [面渣逆袭操作系统篇](#) 🍷
- [面渣逆袭计算机网络篇](#) 🍷
- [面渣逆袭 RocketMQ 篇](#) 🍷
- [面渣逆袭分布式篇](#) 🍷
- [面渣逆袭微服务篇](#) 🍷
- [面渣逆袭设计模式篇](#) 🍷
- [面渣逆袭 Linux 篇](#) 🍷